Preface

Who This Book Is For

This book is written for:

- Complete beginners with no prior programming experience whatsoever
- Learners who may have experience in other programming languages but are new to Python
- Students who are frustrated with dull, repetitive, and uninteractive methods of learning programming
- Educators who wish to adopt a more problem-driven, narrative-based, and real-world approach to teaching programming

Core Philosophy

At its core, this book is built on a single principle:

Programming is not learned by memorizing concepts—it is learned by solving problems.

Instead of introducing abstract ideas first, this book begins with real problems. You will attempt to solve them using your current understanding. When that understanding is no longer sufficient, a new concept is introduced—not as theory, but as a necessary tool for solving the problem. In this way, concepts are learned through necessity rather than memorization.

How This Book Works

Each chapter follows a consistent structure:

- A realistic, real-world problem is introduced at the beginning
- The chapter develops step-by-step reasoning needed to understand the problem
- Concepts are introduced only when required to solve the problem
- Examples and guided explanations reinforce understanding
- The original problem returns at the end as a complete project

This mirrors how programming is actually learned and applied in real environments.

Narrative Learning System

This book is not only about programming—it is also a guided experience. You take the role of **David**, a developing software engineer working through real tasks and responsibilities. Throughout the journey, you will encounter:

- **Mr. Henderson** — your supervisor, representing expectations, structure, and real-world pressure
- **Feynman** — representing the authors, guiding you through concepts with clarity and intuition

These characters are not decorative. They represent real perspectives found in professional learning environments: confusion, explanation, correction, and growth.

Progression System

Your journey begins at an entry-level position. As you move through the chapters:

- Your responsibilities increase
- Your problems become more complex
- Your thinking becomes more structured
- You grow through experience, not instruction alone

The final outcome of this journey is intentionally not revealed.

Learning Method

This book emphasizes active engagement. You will frequently be asked to:

- Predict program behavior
- Debug incorrect logic
- Construct solutions from partial understanding

Learning happens through thinking, not passive reading.

Visual and Conceptual Design

To support understanding, this book makes extensive use of:

- Diagrams and visual explanations
- Real-world analogies

- Step-by-step reasoning
- Dialogue-based explanations inspired by real student questions

The design is intentionally engaging and sometimes slightly informal to improve clarity and intuition.

What You Will Gain

By the end of this book, you will not only understand Python syntax. You will be able to:

- Understand how programs behave
- Break down and analyze problems
- Construct structured and logical solutions
- Think like a programmer in real-world situations

Table of Contents

Volume 1

Introduction

Chapter 1	Variables and Simple Data Types
Chapter 2	Functions and Program Structure
Chapter 3	Flow Control and Data Structures
Chapter 4	Libraries
Chapter 5	Exceptions
Chapter 6	File I/O

Volume 2 Coming soon

Volume I

Python Basics

Introduction

Purpose of Curiosity

In the mid-1990s, the digital world was a chaotic, unmapped wilderness. Deep within the dormitories of Stanford University, a young man, along with his friend, was obsessed with a single, elegant question: Could the vast, disorganized chaos of the early internet be ranked by treating every link like a recommendation—the same way a scientific paper is judged by how many other researchers cite it?

He wasn't attempting to build an empire. He was simply following a thread of curiosity.

To test his theory, he crafted a small, unassuming program—a "crawler" designed to wander from one page to the next, recording every link it found. Because he lacked the funds for professional equipment, he built the housing for his machine's memory out of Lego bricks. It was a jagged, multi-colored tower of plastic sitting in a dusty room, holding together a vision that would soon redefine human knowledge.

The program was built on a startlingly simple realization: a link is more than just a shortcut; it is a "vote." In its infancy, the project was known by the peculiar name BackRub. A few years later, that name was discarded for another: Google.

The student was Larry Page. It stands as a permanent reminder to every person sitting before a blank screen: the most massive structures in our world are almost always built on the foundation of a small idea that actually worked.

"Who knows? when u finish the book you might say , "If we had been born a little earlier, we might have been the ones to build Google."

What Is Programming?

If you think programming is like what you see in the movies—furiously typing cryptic green text while a "System Override" bar flashes on a screen—let me save you the suspense: it is not.

In fact, real programming often looks more like staring at a screen for three hours without typing a single character. Programming is 90% problem-solving and only 10% typing. It relies on four simple building blocks:

- "Do this, then do that." (Sequence)
- "If this is true, do this; otherwise, do that." (Condition)
- "Do this exactly 27 times." (Iteration)
- "Keep doing this until I say stop." (Loops)

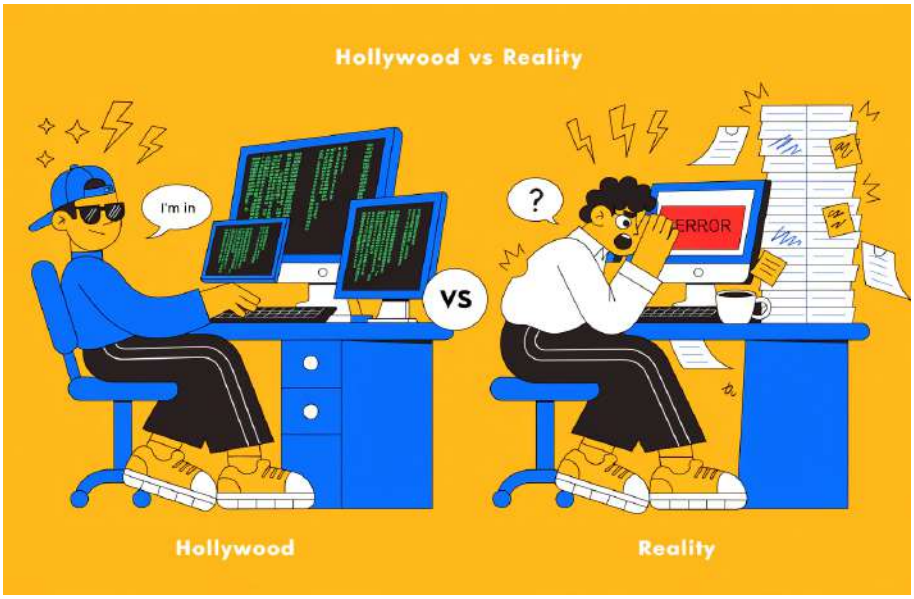


Figure 1: Hollywood vs Reality: What programming actually looks like

Why python ?

Right here, at the start of this book, I'm making a claim: Python is the most human-readable programming language. Now, I'll take it even further—I'm making a bold bet. Python is not just readable; it is easier to understand than learning a human language. Think about it. Can someone learn to speak or write English fluently after reading just one book? Of course not. If that were possible, everyone would do it. But here's my bet: by the time you finish this book, you will be able to "read" and understand Python with ease. And if I'm wrong—then I've lost the bet. You can come back and call me out on it. Now, take a look at the code below. Even if you've never written a single line of Python before, try to understand what it does just by reading it.

password$_{c}heck.py$

```python
# Read the secret password from a file
secret_password = open('SecretPasswordFile.txt').read()

# Ask the user to enter a password
typed_password = input('Enter your password: ')

# Check if the passwords match
if typed_password == secret_password:
    print('Access granted')
else:
    print('Access denied')
```

- The program reads the stored password directly from a file.
- It asks the user to enter a password.
- It compares both passwords.
- If they match, access is granted; otherwise, it is denied.

See? I didn't lie—it really is that readable.

Now, although Python is widely recommended as a first programming language for pedagogical reasons due to its simplicity and readability, this is not the main reason it is chosen in this book. The primary reason is its rapidly increasing importance in modern technology. Its usage and significance are currently growing at an exceptional rate, especially in fields such as artificial intelligence, data science, and scientific computing. In particular, Python's dominance comes from its powerful ecosystem of libraries and frameworks, including:

- **PyTorch:** A leading framework for deep learning and neural network development.
- **TensorFlow:** A widely used platform for machine learning and large-scale AI systems.
- **NumPy:** A core library for numerical computation and array processing.
- **Pandas:** A powerful tool for data analysis and manipulation.

These tools make Python the central language in both industry and research environments, powering applications from large-scale AI systems to scientific simulations. In summary, Python is not only chosen because it is easy to learn, but because its relevance, adoption, and impact are expanding rapidly across modern computing. Now, take a look at the code below.

What Is Python?

Python is a programming language (with a defined set of syntax rules) and also refers to the **Python Interpreter**, the software that reads and executes Python instructions. In simple terms, the language is what you write, and the interpreter is what makes the computer understand and run it.

The name "Python" does not come from the snake. It was actually inspired by the British comedy group *Monty Python*, reflecting the creator's preference for a fun and unconventional approach to programming rather than something overly technical or intimidating. You can download Python from python.org.

It is also important to know that Python is just one of many programming languages in the world. There are many others, each designed with different goals and styles.

The Language Gap

Imagine three entities in a room:

1. **You:** You speak a human-friendly language (Python).
2. **The Computer:** It speaks a complex language of electricity and binary (1s and 0s).
3. **The Interpreter:** The bridge that speaks both perfectly.

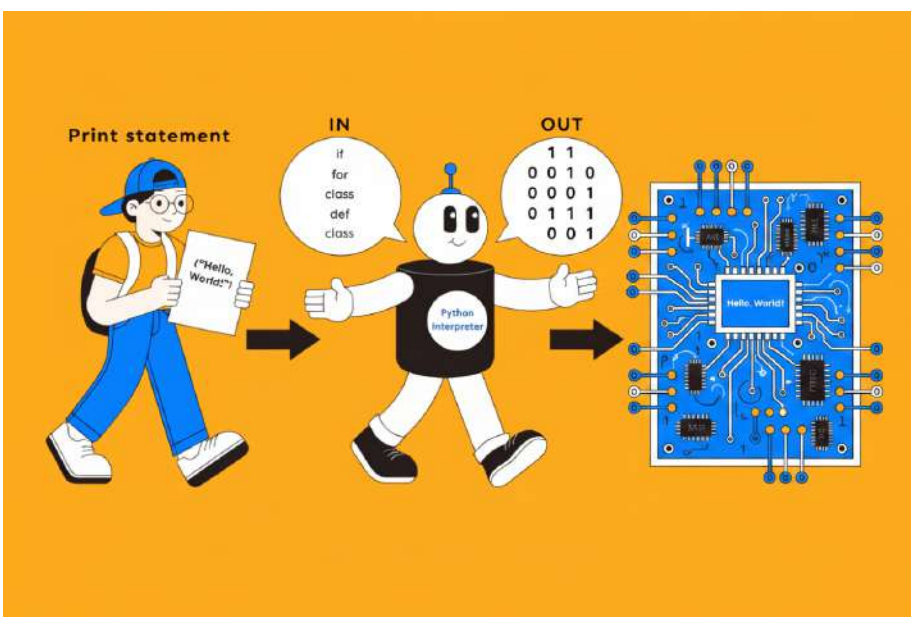


Figure 2: The Language Gap — How the Interpreter translates Python to Machine Code

Python allows you to tell the Interpreter what you want in a way that makes sense to you, and the Interpreter handles the task of explaining it to the machine.

Inspiration

As mentioned in the Introduction, this is the algorithm we talked about. Below is a version of the actual foundational concept used by Google. At this stage, reading this might feel like trying to read Mandarin Chinese. It is dense, it is mathematical, and it is the exact logic that turned a college project into a global empire.

pagerank_formula.py

```python
# Version 1: The Iterative Formula
DAMPING = 0.85
def calculate_rank_formulaic(graph, iterations=10):
    pages = list(graph.keys()); n = len(pages)
    ranks = {page: 1 / n for page in pages}
    for i in range(iterations):
        new_ranks = {}
        for page in pages:
            surfer_jump = (1 - DAMPING) / n
            inbound_votes = sum(ranks[v] / len(graph[v]) for v in graph if page in graph[v])
            new_ranks[page] = surfer_jump + (DAMPING * inbound_votes)
        ranks = new_ranks
    return ranks
```

pagerank_bayesian.py

```python
import random
# Version 2: The Bayesian Network
def calculate_rank_bayesian(graph, samples=10000):
    pages = list(graph.keys()); visit_counts = {page: 0 for page in pages}
    current_page = random.choice(pages)
    for _ in range(samples):
        visit_counts[current_page] += 1
        if random.random() < 0.85 and graph[current_page]:
            current_page = random.choice(graph[current_page])
        else:
            current_page = random.choice(pages)
    return {page: count / samples for page, count in visit_counts.items()}
```

Don't be intimidated; be hungry. By the end of Volume 1, you will be able to write these from scratch.

Chapter 1

Variables and Simple Data Types

When I was in Grade 6, I wrote my first program. It wasn't complex, and it certainly didn't change the world, but it felt like magic. It was a single line of code that nearly every programmer begins with. To be honest, I wrote it in a language called "JavaScript":

hello.js

```
// JavaScript
console.log("Hello world!");
```

Now, after all those years, I invite you to do the same, but in "Python":

hello_world.py

```
print("Hello world!")
```

Even if you have never seen a line of code in your life, you can probably guess what this does. It simply prints the phrase "Hello world!" to your screen. This simplicity is why we start here.

Terminal Output

```
Hello world!
```

1.1 File Extension and Interpreter

When you run a file such as `hello_world.py`, the `.py` extension tells the system that the file contains Python code. This allows the operating system and code editor to recognize it as a Python program. Once the file is executed, it is passed to the **Python Interpreter**. The interpreter is a program that reads your code line by line and translates it into actions the computer can perform. For example, when the interpreter encounters the word `print`, it understands that it should display the value inside the parentheses on the screen. In this way, the interpreter acts as a bridge between human-readable code and machine-executable instructions.

1.2 Storing Values in Variables

Let us extend the program by introducing a variable. Modify the file as follows:

hello$_{w}$*orld.py*

```
message = "Hello my beloved programmers"
print(message)
```

A variable is a named storage location used to hold a value. The value represents the data associated with the variable name.

In this example, the variable `message` stores the string `"Hello my beloved programmers"`. When the Python interpreter executes the first line, it assigns this string to the variable name. On the second line, it retrieves the stored value from `message` and outputs it to the screen.

Terminal Output

```
Hello my beloved programmers
```

Important

Variables allow programs to store data in memory and reuse it throughout execution.

1.3 The "Labeled Box" Analogy

Think of a variable as a labeled box in a computer's memory where a single value can be stored. An **Assignment Statement** consists of a variable name, an equal sign (`=`), and the value to be stored.

Figure 1.1: Variable Initialization: storing a value inside a labeled memory box.

Important

In Python, the single equal sign (`=`) is used for **assignment**, meaning it stores a value inside a variable. This is different from mathematical equality, which uses the double equal sign (`==`) to compare whether two values are the same. This distinction will become important in later sections.

A variable is **initialized** the first time a value is assigned to it.

You can observe this concept in practice using the following example, which models a simple supermarket inventory system:

inventory.py

```python
# Initial stock values in a supermarket inventory system
milk_cartons = 40
egg_boxes = 2

# Using variables in arithmetic expressions
print(milk_cartons + egg_boxes)
print(milk_cartons + egg_boxes + milk_cartons)
```

Terminal Output

```
42
82
```

1.4 Overwriting Variables

A variable in Python is not a permanent vault; it is a temporary storage location. Crucially, a variable can hold **exactly one value** at any given moment. When a new value is assigned to an existing variable, the previous value is discarded and replaced. This process is known as **overwriting**.

Important

When a variable is overwritten, the old data is purged from that specific memory location. Python does not maintain a "history" of previous values. If you need to keep a piece of data for later, you must store it in a different variable before the overwrite occurs.

main.py

```python
container = "Hello"
print(container)

container = "Goodbye"
print(container)
```

Terminal Output

```
Hello Goodbye
```

Figure 1.2: Overwriting: When a new value enters, the old one is discarded.

Problem 1: The Sequence of Execution

Consider the following script. What will be the final value displayed on the screen?

sequence.py

```
score = 10
score = 20
score = 5
print(score)
```

Solution

The output will be **5**. Python executes instructions linearly (top to bottom). The initial value 10 was replaced by 20, which was then immediately replaced by 5. The `print()` function only triggers after the final assignment is complete.

There Is No Dumb Question

David: If the old value is "discarded," where does it go? Is it still taking up space in my computer?
Feynman: That is a brilliant question, David. Python has a system called a **Garbage Collector**. Once a value is no longer linked to any variable name, Python realizes that no human or program can ever use it again. It automatically "recycles" that memory space so your computer stays efficient.

There Is No Dumb Question

David: out of topic but can i ask you one question?
Feynman:go ahead!
David: when I delete a document on my computer? Is the file not actually "gone," but the computer just stops tracking where it is so the space can be used again?
Feynman: You are exactly right, and that is a very sharp observation! When you overwrite a variable or delete a file, the physical bits of data often sit there until something else needs that exact spot.
Think about how terrifying it would be if it worked differently: if every time you deleted a variable or a file, the memory was physically removed or "burned out" so it could never be used again, your computer's memory would shrink byte-by-byte every single day until it eventually died. The **Garbage Collector** in Python (and the File System in your OS) ensures that memory is a renewable resource, not a finite one.

1.5 Dynamic Assignment: The input() Function

Until now, we have been "hard-coding" values—meaning we, the authors of the code, decided what goes in the box. However, real applications are dynamic. The `input()` function allows the **user** to provide the value that will be stored in a variable at runtime.

Important

The `input()` function is a **blocking** command. The Python interpreter will pause all operations and wait indefinitely until the user types their response and presses the **Enter** key.

Problem 2: Interaction and Overwriting

A programmer sets a default value but then asks the user for a change. If the user types "Python" when prompted, what is the output?

user$_i$nput.py

```
language = "C++"
print("Default language is: " + language)

language = input("Enter your favorite language: ")
print("Choice updated to: " + language)
```

Solution

Initially, the `language` box contains "C++". When the `input()` line runs, the interpreter waits for the user. As soon as the user types "Python" and hits Enter, the value "C++" is deleted and replaced by the user's choice.

Terminal Output

```
Default language is:  C++
Enter your favorite language:  Python
Choice updated to:  Python
```

Problem 3: The "Empty Input" Trap

What happens if a user is prompted to enter data but simply presses **Enter** without typing anything? Does the variable keep its old value?

trap.py

```
security_code = 9999
security_code = input("Enter new code: ")
print(security_code)
```

The "Empty String" Reality

No, it does not keep the 9999. Even if the user types nothing, pressing the Enter key sends an **Empty String** (`""`) to the variable. The original value (9999) is overwritten by "nothingness."

There Is No Dumb Question

David: Wait, so the `input()` function is "blind"? It doesn't care if the user actually typed a valid answer or just hit Enter?
Feynman: Exactly. The computer doesn't judge the quality of the data; it just follows

the instruction to *store* it. If you tell it to replace your password with "nothing," it will happily do so. This is why professional programs require "validation" logic to ensure the user didn't leave a box empty.

1.6 Syntax Errors

In the physical world, humans can communicate through context, even if grammar is imperfect. In programming, however, the Python interpreter requires absolute precision. When the interpreter encounters code that violates the formal rules of the language, it triggers a **Syntax Error**.

Think of a Syntax Error as a "grammatical failure" in the machine's language. When this occurs, the program does not run partially; it stops entirely and generates a **Traceback**—a diagnostic report that identifies the exact line where the interpreter became confused.

Important

An error message is not a failure; it is a diagnostic tool. Your computer is not broken; it is simply asking for clarification. Learning to read Tracebacks is the most important skill a programmer can develop.

Problem 1: The Unterminated String

A frequent mistake occurs when a programmer opens a string literal with a quotation mark but fails to close it before the end of the line.

error$_1$.*py*

```
print("Hello my beloved programmers)
```

Solution: Closing the Literal

Python expects every string to be "terminated" or closed. To resolve this, ensure every opening quote has a matching closing quote at the end of the text.

Terminal Output

```
SyntaxError: unterminated string literal (detected at line 1)
```

There Is No Dumb Question

David: Why is Python so picky? If I forget a single quote, it's obvious what I meant. Why can't it just fix it for me?
Feynman: Because "obvious" is a human concept, David. If the interpreter started guessing where your text ended, it might accidentally include your code as part of the text. Absolute strictness is the only way to ensure the computer does *exactly* what you tell it to do, nothing more and nothing less.

Problem 2: The Missing Parenthesis

The `print()` function is a command that requires "delimiters" (parentheses) to contain the data you wish to process. Leaving one out is a frequent oversight.

error$_2$.*py*

```
print("Welcome to Volume 1"
```

Solution: Balancing Delimiters

Parentheses must always be balanced. If you open a set, you must close it.

Terminal Output

```
SyntaxError: unexpected EOF while parsing
```

Note: EOF stands for "End of File"—Python kept looking for the closing bracket until the file ended.

Problem 3: The Missing Separator

When attempting to display two distinct pieces of data—such as a label and a number—placing them side-by-side without a separator causes a collision.

error$_3$.*py*

```
print("The current stock is:" 50)
```

Solution: Using the Comma

To output multiple items in a single line, you must separate them with a comma. This tells Python to treat them as separate entities and automatically adds a space between them.

Terminal Output

```
SyntaxError: invalid syntax
```

Logic Challenge: The Silent Failure

Analyze the code below. It will not "crash," but it will not update the user's name as intended.

logic_check.py

```
1 user_name = "Alice"
2 print(user_name)
3
4 user_name == "Bob"
5 print(user_name)
```

Analysis

The second output remains **Alice**. This is because Line 4 uses `==`. Instead of changing the variable, the program simply asked a question: "Does `user_name` equal Bob?" The answer was "No," but since that answer wasn't stored anywhere, the program moved on without changing anything.

There Is No Dumb Question

David: So you're saying a program can be "wrong" even if there are no error messages?
Feynman: Exactly. That is a **Logic Error**. Those are much harder to find because the computer is happy, but the human is confused. A Syntax Error is the computer telling you it doesn't understand your language; a Logic Error is the computer doing exactly what you said, even if what you said was a mistake.

Important

In Python, the single equal sign (`=`) is for **Assignment** (putting a value in a box). The double equal sign (`==`) is for **Comparison** (asking if two things are the same).

1.6 The Categories of Data

In the physical world, we categorize things to know how to interact with them: you drink a liquid, but you eat a solid. In Python, we do the same with **Data Types**. A data type is a classification that tells Python exactly what a value is and what it is allowed to do with it.

Data Type	Code	Description	Examples
Integers	`int`	Whole numbers	-1, 0, 42, 1000
Floating-point	`float`	Numbers with decimals	-1.25, 0.0, 3.14
Strings	`str`	Series of characters	'a', "Hello!", "42"

1.6.1 Numerical Precision: int vs. float

The two most common numerical types are **Integers** and **Floats**. Even though the mathematical value is the same, Python treats `42` as an integer and `42.0` as a float. To a computer, that decimal point—even if it is just zero—signals that the number must be handled with a different level of precision in the hardware.

Important

The Identity Rule: Every single value in Python belongs to exactly **one** data type. You can check the type of any value by using the `type()` function, such as `type(42)`.

There Is No Dumb Question

David: Wait, if I put quotes around a number, like `"42"`, is it still a number?
Feynman: No, David! The moment you put quotes around it, you have "gift-wrapped" it. The computer doesn't look inside the wrapping to see a number; it just sees the wrapping itself. To Python, `42` is a value for math, but `"42"` is just a piece of text. It's the difference between the number 100 and the word "One-Hundred."

Problem 1: Identifying the Category

Identify the specific data type for each of the following variables based on their values:

types.py

```
value_a = 500
value_b = "500"
value_c = 500.0
value_d = ""
```

Solution

- **value_a:** `int` (A pure whole number).
- **value_b:** `str` (Textual data, because of the quotes).
- **value_c:** `float` (A decimal number).
- **value_d:** `str` (The "Empty String"—it is still text, just with zero characters).

1.7 Operators: The Multi-Talented Symbols

The meaning of an operator changes based on the data types of the values next to it. This concept is known as **Operator Overloading**.

1.7.1 Concatenation and Replication

When the `+` operator is used on two strings, it becomes the **concatenation** operator, joining them together. When the `*` operator is used on a string and an integer, it becomes the **replication** operator.

operators.py

```python
# Concatenation: Glueing text
print("Alice" + "Bob")

# Replication: Repeating text
print("Alice" * 5)
```

Terminal Output

```
AliceBob
AliceAliceAliceAliceAlice
```

There Is No Dumb Question

David: Why can't I just multiply two strings? Like `"Hi" * "Hi"`?
Feynman: Because multiplication is just repeated addition. You can add "Hi" to itself 5 times, but you can't add "Hi" to itself "Hi" times. It has no physical meaning in logic. To the computer, the `*` symbol needs a "scalar" (a count) to know how many times to repeat the pattern in memory.

Problem 2: The Mixing Trap

If you run the following code, will it print a value or cause an error?

mystery.py

```
x = 10
y = "20"
print(x + y)
```

Solution

This results in a **TypeError**. Python is a "Strongly Typed" language—it refuses to guess if you want math ($10 + 20 = 30$) or text (”1020”). To prevent a logic disaster, it stops the program. You must explicitly convert one type to match the other.

Terminal Output

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

There Is No Dumb Question

David: So a variable's type depends entirely on what I put in it? If I overwrite a number with text, does the type change?
Feynman: You've got it! This is **Dynamic Typing**. Python is flexible; it updates the "category" of the variable every time you overwrite the box. But remember: once you change that 10 to "Apple," the number 10 is recycled by the **Garbage Collector** and is gone forever!

Problem 3: The Data-Behavior Paradox

Trace the variables. What is the final output of the two print statements?

logic.py

```
a = "7"
b = 7
print(a * 3)
print(b * 3)
```

Solution

Output 1: `777` (String replication).
Output 2: `21` (Integer multiplication).
Lesson: The *type* of data determines the *behavior* of the operator. Always know what is inside your variable before applying an operator!

1.7.2 When Types Clash

Python is a **Strongly Typed** language. This means it will not perform "magic" conversions behind your back. If you try to mix incompatible types—like adding a number to a string—Python throws a `TypeError`, signaling that the operation is logically undefined.

Terminal Output

```
>> 'Alice' + 42
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError:  can only concatenate str (not "int") to str
```

Problem 1: The Logical Wall

Explain why Python allows `"Alice" * 3` but crashes on `"Alice" + 3`.

Solution

The Explanation: In Python, the `*` operator with an integer is defined as **Replication** (repeating a sequence). However, the `+` operator is defined as **Concatenation** (joining two sequences). Since an integer is a single value, not a sequence of characters, Python has no rule for "gluing" a number to a word. To fix this, you must explicitly convert the number using `str(3)`.

1.8 String Methods: Built-in Skills

A **method** is a specialized function that "lives" inside a piece of data. Unlike a standalone function like `print()`, a method is called directly on a variable using the **dot (.) operator**. This tells Python: "Perform this specific action on this specific object."

1.8.1 Changing Case

Methods like `title()`, `upper()`, and `lower()` are essential for **data sanitization**. They ensure that user input (which is often messy) is converted into a standard format for storage or display.

name.py

```python
name = "david malen"
print(name.title())     # Capitalizes the first letter of each word
print(name.upper())     # Converts all characters to uppercase
print(name.capitalize()) # Capitalizes only the first character
```

Terminal Output

```
David Malen
DAVID MALEN
```

```
David malen
```

1.8.2 Whitespace and Escape Characters

In programming, **whitespace** refers to characters that occupy space on the screen but are invisible when printed. These are not just "empty" spaces; they are specific instructions to the computer.

To represent these instructions inside a string, we use an **Escape Character**: a backslash (\) followed by a specific letter. The backslash tells Python: "Don't treat the next character as text; treat it as a command."

Escape Seq.	Name	Action
`\n`	Newline	Moves the cursor to the start of the next line.
`\t`	Tab	Inserts a horizontal indentation (usually 4 spaces).
`\\`	Backslash	Prints a literal backslash.
`\'`	Single Quote	Prints a quote without ending the string literal.

whitespace.py

```python
# \n creates a new line; \t creates an indentation
print("Languages:\n\tPython\n\tC\n\tJavaScript")

favorite = " python "
print(favorite.rstrip()) # Removes space from the right side
print(favorite.lstrip()) # Removes space from the left side
print(favorite.strip()) # Removes space from both sides
```

Terminal Output

```
Languages:
    Python
    C
    JavaScript
 python
python
python
```

1.8.3 The Apostrophe Trap

Python determines the start and end of a string based on the first pair of quotes it finds. If you use single quotes for a string that contains an apostrophe, Python will think the string ends prematurely.

apostrophe.py

```python
# This code will crash because of the apostrophe in "Python's"
message = 'One of Python's strengths is its diverse community.'
print(message)
```

Terminal Output

```
File "apostrophe.py", line 1
    message = 'One of Python's strengths...'
                             ^
SyntaxError: invalid syntax
```

There Is No Dumb Question

David: Why did the code crash there? It looks fine to me.
Feynman: Because Python is a simple machine, David! It sees the quote at the start of `'One...'` and the second quote at the end of `...Python'`. It thinks the string is over. Everything after that—`s strengths...`—looks like gibberish to the computer.
David: So I should just use double quotes `" "` whenever I have an apostrophe?
Feynman: Exactly. Or use a backslash `\'` to tell Python: "This quote is just a character, don't use it to end the string!"

1.9 Immutability

This is the most critical concept to master regarding strings: **Strings are Immutable.** This means that once a string is created in memory, it can never be changed.

Problem 2: The "Shadow" Variable Challenge

Analyze the code below. What will be the exact output of the two print statements?

hard$_c$*hallenge.py*

```python
original_name = "  python_pro  "
clean_name = original_name.strip()
clean_name.upper()

print(original_name)
print(clean_name)
```

Solution

Output 1: `" python_pro "` (With the original spaces).
Output 2: `"python_pro"` (Lowercase, but with no spaces).

The Logic:
1. On line 2, `strip()` created a **new** string and saved it into `clean_name`.
2. On line 3, `upper()` was called on `clean_name`. It created a new uppercase string in memory, but **it was never saved to a variable**.
3. Therefore, `clean_name` remains exactly as it was when it was first created on line 2.

There Is No Dumb Question

David: So if I want `clean_name` to actually be uppercase, I have to overwrite it?
Feynman: Precisely! You would need to write `clean_name = clean_name.upper()`.

1.10 Integers and Floating-Point Numbers

In Python, numbers are processed through **Expressions**, which are combinations of values and operators. An expression is the most fundamental form of instruction; it always "evaluates" (reduces) to a single value.

1.10.1 Mathematical Operators

Python provides a robust set of operators. While most follow standard arithmetic, three specific operators:**Exponent**, **Modulus**, and **Integer Division**,are the real workhorses of algorithmic logic.

Operator	Operation	Example	Result
`**`	Exponent	`2 ** 3`	8
`%`	Modulus (Remainder)	`22 % 8`	6
`//`	Integer Division	`22 // 8`	2
`/`	Float Division	`22 / 8`	2.75
`*`	Multiplication	`3 * 5`	15
`-`	Subtraction	`5 - 2`	3
`+`	Addition	`2 + 4`	6

1.10.2 Numerical Identity: int vs. float

Python distinguishes between whole numbers (**Integers**) and numbers with decimal points (**Floating-Point Numbers**).

There Is No Dumb Question

David: Why does `20 / 5` give me `4.0` instead of just `4`?
Feynman: Because in Python, the single slash (`/`) is a "Float Division" operator. Its job is to ensure precision. Even if there is no remainder, Python returns a float to stay consistent. If you want a whole number, you must use the floor division (`//`) operator.
David: So `20 // 5` is `4`, but `20 / 5` is `4.0`?
Feynman: Exactly. One represents an exact quantity (`int`), and the other represents a measured precision (`float`).

1.10.3 The Order of Operations

Python follows the "PEMDAS" rule (Parentheses, Exponents, Multiplication/Division, Addition/Subtraction). If you write a complex expression, Python evaluates it layer by layer until a single value remains.

complex$_m$ath.py

```python
# Python evaluates the innermost parentheses first
result = (5 - 1) * ((7 + 1) / (3 - 1))
print(result)
```

Terminal Output

```
16.0
```

Problem 1: The Remainder Logic

A program needs to check if a number is **even** or **odd**. Which operator is the best tool for this, and why?

Solution

The **Modulus (%)** operator is the correct tool. Any even number divided by 2 has a remainder of 0 (`x % 2 == 0`). Any odd number has a remainder of 1. This is a fundamental pattern in computer science for creating alternating logic.

Problem 2: Predict the Evaluation

Without a computer, solve the following expression:
`5 + 2 ** 3 * (10 % 3)`

Solution

Step 1 (Parentheses): `10 % 3` is `1`. (10 divided by 3 is 3 with 1 left over).
Step 2 (Exponent): `2 ** 3` is `8`.
Step 3 (Multiplication): `8 * 1` is `8`.
Step 4 (Addition): `5 + 8` is `13`.
Final Result: `13`

1.10.4 Syntax and Mathematical Dead-Ends

Python is strict. A `SyntaxError` occurs if you leave an expression "hanging."

errors.py

```python
print((5 + 2) *) # Error: Python expects a second value for the *
```

Terminal Output

```
SyntaxError: invalid syntax
```

There Is No Dumb Question

David: I've heard about a "Precision Quirk" where `0.1 + 0.2` doesn't equal `0.3` exactly. Is that true?
Feynman: Yes! Most computers represent floats in binary (base-2). Just as you can't write 1/3 exactly in decimal (0.333...), a computer can't represent 0.1 exactly in binary.
David: Is that dangerous?
Feynman: In physics or banking, yes! For tiny calculations, it's fine, but if you're building a bank, you never use floats for money—you count in total cents as **integers** to avoid losing pennies to "rounding ghosts."

Problem 3: The "Fair Share" Algorithm

You have 100 dollars to divide among 3 people.

bank.py

```python
total = 100
print(total // 3)
print(total % 3)
```

Explain what these two lines represent in a real-world scenario.

Solution

Line 1 (`total // 3`): Each person gets $33. This is the "Fair Share" in whole dollars.
Line 2 (`total % 3`): This equals `1`. This is the $1 left over that cannot be split equally. In professional code, this represents the "remainder" that might be sent to a service fee or saved for the next transaction.

1.11 The Limits of the Machine: Overflow and Infinity

In mathematics, numbers are conceptual and infinite. In a computer, numbers are "Physical Objects" made of electricity and silicon. Because a computer has a finite number of "slots" (bits) to store a value, it eventually hits a wall where it can no longer represent reality accurately.

1.11.1 Floating-Point Overflow

For `float` types, there is a maximum value roughly equal to 1.8×10^{308}. If a calculation results in a number larger than this, Python does not crash; instead, it returns a special value: `inf` (Infinity).

overflow.py

```python
# Trying to store a number larger than the physical limit
huge_number = 2e308
print(huge_number)

# Operations on infinity
print(huge_number * 2)
print(huge_number + 1)
```

Terminal Output

```
inf
inf
inf
```

There Is No Dumb Question

David: Wait, so `inf` is just the computer saying, "This is too big for my brain to hold?"
Feynman: Exactly, David. It's not "true" mathematical infinity; it's a **Hardware Limit**. Once a number crosses that 308-decimal-place threshold, the bits "overflow" the storage container. At that point, the computer stops tracking the exact value and just treats it as a bottomless pit.
David: Is there a "too small" limit too?

Feynman: Yes! That is called **Underflow**. If a number is so small (e.g., 10^{-324}) that it can't be represented, it simply collapses into `0.0`. The computer "loses" the tiny value forever.

1.11.2 The Precision Gap

As floating-point numbers get larger, the distance between the numbers the computer can represent also gets larger. This leads to a strange paradox: adding a small number to a very large number might result in **zero change**.

Problem 1: The Precision Paradox

Predict the output of the following comparison:

precision.py

```
massive_float = 1.0e19
print(massive_float + 1 == massive_float)
```

Solution

Output: `True`
The Explanation: Because floats only have about 15 to 17 digits of precision, once you reach 10^{16}, adding `1` happens "off the edge" of what the computer can track.

There Is No Dumb Question

David: If the computer can't track small changes in big numbers, how do banks manage billions of dollars without losing cents?
Feynman: They don't use floats, David! To a physicist, 0.00000000001 might not matter, but to a banker, it's a crime. For absolute precision, you use **Integers** to count in cents (so $1.00 is `100`). Because Python's integers are "arbitrary precision," they will grow to use as much RAM as they need—they never "lose" a digit to a rounding ghost.

Problem 2: The "inf" Trap

What is the result of `1.0 / 0.0` in Python?

Solution

Contrary to what you might expect, Python will **not** return `inf`. It will raise a **ZeroDivisionError**. While `inf` is used for numbers that are too large to store, division by zero is a logical impossibility that crashes the program immediately to prevent a total system failure.

1.12 Your First Program

While the interactive shell is excellent for testing Python instructions one at a time, creating real applications requires the **File Editor**. The file editor allows you to type many instructions, save them to a file, and run them as a cohesive program from top to bottom.

1.13 Creating Your First Program

At this point, you have been entering instructions into the interactive shell one line at a time. To build actual software, you must save your instructions in a **Script**. Open a new file in your editor, enter the following code exactly as shown, and save it as `hello.py`.

hello.py

```python
# This program says hello and asks for my name.
print('Hello, world!')

print('What is your name?')   # Wait for user to type their name
myName = input()

print('It is good to meet you, ' + myName)
print('The length of your name is:')
print(len(myName))

print('What is your age?')    # Wait for user to type their age
myAge = input()
print('You will be ' + str(int(myAge) + 1) + ' in a year.')
```

Once saved, run the program. The terminal will execute your instructions sequentially from top to bottom.

Terminal Output

```
Hello, world!
What is your name?
David
It is good to meet you, David
The length of your name is:
5
What is your age?
21
```

```
You will be 22 in a year.
```

1.14 Dissecting the Code: The Core Functions

1.14.1 The print() and input() Functions

These are your primary "Communication Ports."

- **print()**: Evaluates the expression inside the parentheses and displays the result as text on the screen.

- **input()**:function is how your program receives data from the outside world. However, there is a fundamental rule you must memorize: **input() always returns a String**.

  Even if the user types a number, Python captures the *keystrokes* as textual data (`str`), not mathematical values. This leads to the "Visual Paradox" where the data looks like a number but behaves like text.

input_trap.py

```python
# Step 1: Capture input
user_value = input("Enter a number: ")

# Step 2: Proof of identity
print("Value Entered: " + user_value)
print("Internal Type: ", type(user_value))

# Step 3: The Operator Trap
# Because it is a string, '*' replicates the text!
print("Visual Paradox: ", user_value * 3)
```

Terminal Output

```
Enter a number:  10
Value Entered:  10
Internal Type:  <class 'str'>
Visual Paradox:  101010
```

There Is No Dumb Question

David: Wait, I typed `10` and the output was `101010`? That's huge!
Feynman: That is the "String Trap" in action, David. Because `input()` saved your entry as a **String**, Python used the *Replication* operator instead of the *Multiplication* operator. It didn't see the quantity ten; it saw the characters '1' and '0' and repeated that pattern three times.
David: So if I actually wanted `30`, I would have to "cast" the input first?
Feynman: Exactly. You must wrap the input in a conversion function: `value = int(input())`. Only then will the computer treat those keystrokes as a mathematical

quantity.

Problem: The Broken Calculator

A student writes the following code to double a number:

broken.py

```
num = input("Enter a number: ")
print(num + num)
```

If the user enters `5`, what is the **exact** output, and why isn't it `10`?

Solution

Output: `55`
The Logic: Since `input()` returns a string, the `+` operator performs **Concatenation** (joining text) rather than addition. Python glues the character `'5'` to another character `'5'`. To fix this, the first line must be: `num = int(input("Enter a number: "))`.

Important

The Crash Guard: If a user types `"Twelve"` into an `int(input())` prompt, the program will crash with a `ValueError`. The conversion only works if the string contains actual numeric characters.

1.14.2 The len() Function

The `len()` function (short for length) takes a string and evaluates to an integer representing the number of characters in that string.

There Is No Dumb Question

David: Does `len()` count the quotes or just the letters?
Feynman: Just the data, David! The quotes are just the "container" that tells Python where the string starts. However, it *does* count spaces, tabs, and punctuation. If you have `"Hi!"`, the length is 3. To a computer, a space is just as much a "character" as the letter 'Z'.

1.14.3 The str(), int(), and float() Functions

Because Python is a **Strongly Typed** language, you cannot perform math on text. If you want to use a string as a number, you must "cast" it into a new type.

Function	Example Transformation	Resulting Type
`str(29)`	`29` → `"29"`	String
`int("42")`	`"42"` → `42`	Integer
`float(10)`	`10` → `10.0`	Floating-Point

Important

The Type Crash: Trying to run `int("Hello")` will cause a `ValueError`. You can only convert a string to an integer if the string *looks* like a whole number.

1.15 Deep Dive: The Evaluation Stack

The final line of our program is the most complex:
`print('You will be ' + str(int(myAge) + 1) + ' in a year.')`
To understand this, you must visualize how Python "collapses" expressions from the inside out, like a set of nesting dolls.

Step	Expression State	Logical Action
Start	`str(int(myAge) + 1)`	Python looks at the innermost parentheses.
1	`str(int("21") + 1)`	Value `"21"` is retrieved from `myAge`.
2	`str(21 + 1)`	`int()` extracts the numeric value from the text.
3	`str(22)`	Mathematical addition is performed.
Final	`"22"`	`str()` converts the sum back to text for printing.

There Is No Dumb Question

David: Why can't I just do `myAge + 1`? It seems like extra work to convert it twice.
Feynman: Because `myAge` is currently a string! You can't add an "idea" to a "quantity." If you tried `"21" + 1`, Python would stop and say: "I don't know how to glue text to a number." You have to pull the number out of the "text box," add to it, and then put it back in a "text box" so it can be part of the final sentence.

Problem 1: The Character Count Mystery

Predict the output of `len(str(55 * 2))`. Explain your logic.

Solution

The Result: `3`
The Logic: First, `55 * 2` evaluates to the integer `110`. Then, `str(110)` converts that integer into the string `"110"`. Finally, `len("110")` counts the characters in that string. There are three characters, so the result is `3`.

Problem 2: The Conversion Trap

What happens if you run the following code?
`print(int(10.9))`

Solution

The Result: `10`
The Logic: When the `int()` function is given a floating-point number, it does not round; it **truncates**. It simply chops off everything after the decimal point and throws it away.

Problem 3: Finding the TypeError

A user writes: `print("My weight is " + 75.5)`. How do you fix this so it doesn't crash?

Solution

The Fix: `print("My weight is " + str(75.5))`
Explanation: You cannot concatenate (join) a string and a float. Wrapping the number in `str()` converts it into textual data, allowing the `+` operator to function as a joiner.

There Is No Dumb Question

David: Is there a faster way to print numbers without converting them?
Feynman: If you use commas, like `print("Weight:", 75.5)`, Python will handle the conversion for you automatically! But when you use the `+` operator, you are taking full responsibility for the data types.

Predict the Output

1. Predict the output:

Python

```python
x = 5
y = 2
print(x * y)
```

Terminal Output

2. Predict the output:

Python

```python
x = 10
x = x + 3
print(x)
```

Terminal Output

3. Predict the output:

Python

```python
name = "david"
message = "Hello " + name
print(message)
```

Terminal Output

4. Predict the output:

Python

```python
word = "ha"
print(word * 2)
```

Terminal Output

5. Predict the output:

```python
print(3 / 2)
```

Terminal Output

Debugging

Fix the error in each code snippet and explain why it is wrong.

1.

```python
10 / 0
```

2.

```python
1 +
```

3.

```python
"hello " + 5
```

4.

```python
2 * * 3
```

5.

```python
name == "david"
print(name)
```

Theory

1. Explain the difference between `int`, `float`, and `str` with one example each.

2. Why does this cause an error?

```python
"hello" + 3
```

3. Why does this return a float?

```python
6 / 2
```

4. Explain variable reassignment:

```python
x = 5
x = 10
```

(hint: see chapter 1)

5. Why is this True?

```python
print(1 == True)
```

Program Writing (Digits + Number Manipulation Basics)

6. Working with Digits (Integer Logic)
In this section, you will practice extracting and modifying digits of a number using only arithmetic operations.

We define:

$$n = 48756378291, \quad k = 3$$

Here, **k = 3 means the 3rd digit from the right (units side)**.

6.1 Extract the k-th digit
Write a program that extracts the k-th digit of n.

Hint: Think about:

- removing digits using integer division `//`
- isolating digits using modulus `%`

Expected idea: You should isolate a single digit from the number.

6.2 Replace the k-th digit with 0
Using the same values:

$$n = 48756378291, \quad k = 3$$

Write a program that replaces the k-th digit from the right with `0`.

Example:

```terminal
Original: 48756378291
Result:   48756378091
```

Hint: Break the number into:

- left part (digits before target position)
- digit to replace
- right part (digits after target position)

Then rebuild the number.

6.3 Replace the k-th digit with a new value
Now define:

$$n = 48756378291, \quad k = 3, \quad e = 7$$

Write a program that replaces the k-th digit with e.

Example:

```
terminal
Original: 48756378291
Result:   48756378791
```

Final Hint (Important):
Every digit manipulation problem can be solved using:

- integer division (`//`) to remove digits
- modulus (`%`) to extract digits
- powers of 10 to shift positions

String Manipulation

Write a program called `playback.py` that:

- asks the user for a sentence
- outputs the same sentence but replaces every space with `|` (pipe separator)

Why this matters:
In real systems, spaces are often replaced with separators when:

- logging events

- processing data streams
- formatting structured text output

Example 1:

Terminal Output

```
Input: This is david
Output: This | is | david
```

Example 2:

Terminal Output

```
Input: Let's implement a function
Output: Let's | implement | a | function
```

Example 3:

Terminal Output

```
Input: Python is fun to learn
Output: Python | is | fun | to | learn
```

Requirements:

- Use `input()` to get user input
- Replace every space character
- Print the modified string

Hint:
Think about string methods that:

- replace one substring with another
- operate on the entire sentence at once

Documentation:

`https://docs.python.org/3/library/stdtypes.html#string-methods`

Program Writing (Energy Conversion)

Einstein (Mass to Energy Conversion)
Even if you have not studied physics recently, the relationship between mass and energy is given by:

$$E = mc^2$$

where:

- E is energy in Joules
- m is mass in kilograms
- c is the speed of light (300,000,000 m/s)

In a file called `einstein.py`, write a program that:

- prompts the user for mass as an integer
- computes the equivalent energy using the formula $E = mc^2$
- prints the result as an integer

Important Note:
You must use:

$$c = 300000000$$

and compute:

$$E = m \times c \times c$$

Example Runs:
Run 1

Terminal Output

```
Input: 1
Output: 90000000000000000
```

Run 2

Terminal Output

```
Input: 14
Output: 1260000000000000000
```

Run 3

Terminal Output

```
Input: 50
Output: 4500000000000000000
```

Requirements:

- Use `input()` to get mass
- Convert input to integer
- Use integer arithmetic only
- Print the final result

Hint:

- First store the speed of light in a variable
- Then apply the formula step by step
- Be careful with large numbers

Chapter 2

Functions

Assume

Mr. Henderson: "David, welcome aboard. Let's start simple. I need a program to calculate the final price of our top three sellers after adding a 15% sales tax. Think you can handle that?"
David: "Absolutely, Mr. Henderson. I'll have it on your desk in ten minutes."
Using what you learned in Chapter 1, you write a script to handle an apple, bread, and milk:

market_v1.py

```python
# Processing an Apple
apple_price = 0.50
apple_total = round(apple_price * 1.15, 2)
print("Apple Total: $" + str(apple_total))

# Processing Bread
bread_price = 2.50
bread_total = round(bread_price * 1.15, 2)
print("Bread Total: $" + str(bread_total))

# Processing Milk
milk_price = 4.00
milk_total = round(milk_price * 1.15, 2)
print("Milk Total: $" + str(milk_total))
```

The program works perfectly. You feel like a pro—until Mr. Henderson walks back into the office an hour later with a look of concern.
Mr. Henderson: "Great work on those three, David. But I've got some news. A delivery truck just dropped off 500 new products that all need to be added to the system. Oh, and one more thing—the government just raised the sales tax to 18% effective immediately."
David (Internal Monologue): *Wait... 500 items? To add those, I'd have to copy and paste those lines 500 times. My code will be a massive, unreadable wall of text. And to*

update the tax, I have to manually find every single '1.15' and change it to '1.18'?
—

Suddenly, you realize you are trapped. If you miss just one line, the store loses money. If you make a tiny typo during all that copy-pasting, the whole system breaks. You've hit a wall where your current way of coding simply cannot keep up with the real world.
Feynman: "Don't panic, David! This is exactly why we use **Functions**. You're currently building a separate machine for every single apple;This chapter will show you how to build a single factory that handles everything."

2.1 Why bother with functions?

Before we dive into the technical details, it is important to understand the role functions play in software development. Essentially, functions are the primary tool used to organize and structure a program. If you have experience with other programming languages, you likely recognize them as a way to group code. In Python, functions serve two vital roles:

2.1.1 Code Reusability

Functions provide the most efficient way to "package" logic that needs to be executed multiple times or in different parts of a program. Up to now, the code we have written executes immediately from top to bottom. Functions, however, allow us to group specific chunks of code and give them parameters. Once defined, these blocks can be called and reused as many times as necessary without rewriting the logic.

This is critical because it creates a single point of maintenance. If you use a function like `calculate_tax` in ten different parts of your app and the tax rate changes, you only need to update the logic inside that one function. Without functions, you would have to find and change every individual instance of that calculation, which leads to bugs and human error.

2.1.2 Task Decomposition

Functions act as a tool for breaking down a large, complex system into smaller pieces that each have a specific responsibility. This process is known as "decomposition"—taking a giant problem and shattering it into solvable sub-tasks.

Consider the process of manufacturing a car. You wouldn't attempt to build the entire vehicle in a single, massive step. Instead, the process is divided into specialized stages: one team builds the engine, another assembles the chassis, a third handles the electronics, and a fourth focuses on the interior.

In programming, functions allow you to do the same. If you were coding a "drive car" program, you would create separate functions like `start_engine()`, `apply_brakes()`, and `shift_gear()`. It is significantly easier to build, test, and fix these small functions in isolation than it is to attempt to manage the entire "driving" logic in one giant block of code. Ultimately, functions are about defining the procedure: the specific steps of how to accomplish a goal.

Solution

By using a function, we define the "formula" once and reuse it everywhere. If the tax changes or we add 1,000 more items, we only ever have to change one line of code.

market_final.py

```
def calculate_total(price):
    tax_rate = 1.18  # Changed in only ONE place!
    return round(price * tax_rate, 2)

# Now, handling any number of items is easy
print(calculate_total(0.50))
print(calculate_total(2.50))
print(calculate_total(4.00))
```

There Is No Dumb Question

Feynman: David, looking at that code we just wrote for Mr. Henderson... are there any specific parts that look like "Greek" to you? Don't hold back; the simplest questions often lead to the deepest insights.
David: Oh, yeah, absolutely! I've been staring at this, and I feel like I'm looking at a secret code. For starters:

- **What is `def`?** Why do we have to say that? Why can't we just start writing the math?
- **Why is the code indented?** Why is the logic pushed to the right like that? Does the spacing actually change how the computer thinks?
- **Why is there a colon (`:`)?** It seems so small. Does Python really care about a tiny dot and a tail?
- **What is `return`?** I see it at the end of every block. Is that how the machine "spits out" the final answer to the rest of the program?

Feynman: Excellent observations. Each of those is a structural pillar of the Python language. You're noticing the "grammar" of functions. Well, read the upcoming section, David, to get all of this answered!

Now that we know *why* we need functions, it's time to pull back the curtain and look at the "Anatomy" of how they are built.

2.2 Function Basics

Although we haven't gotten very formal about it, you have already been using functions throughout the earlier chapters.

Think back to how we got information from the user: we called the built-in input() function. To find out how long a string was, we used the **`input()`** function. To show something on the screen, we used **`print()`**. These are all functions that someone else wrote for us. Now, it's time to learn how to write our own.

To understand how to build one, let's look at a simple example. We are going to create a function that greets a programmer by name.

anatomy.py

```
def greet(name): # (A) Definition, (B) Parameter
    print("hello programmer", name) # (C) Indented Block
```

A: The Definition (The `def` Keyword)

The **`def`** keyword tells Python to create a "function object." Python doesn't run the code inside yet; it just remembers the instructions under the name you gave it. It is like writing a recipe down—the meal isn't cooked until someone "calls" the recipe.

examples.py

```
#examples
def make():
    ...
def hello():
    ...
```

B: Parameters: The Input Slots

In the code above, **`name`** is a **parameter**. Think of it as an "empty slot." When you call `greet("david")`, the argument `"david"` is dropped into that slot. This allows one function to work with different data every time.

examples.py

```
#examples
def hello(name): # the parameter or the solt is called name.
    print(name)
hello("teacher feynman")
hello("student david")
```

Terminal Output

```
teacher feynman
student david
```

C: The Colon and Indentation

The **colon (:)** signals that a code block is starting. In Python, we use "indentation" (usually 4 spaces) to show which lines belong to the function.

Understanding the "Block" Structure

As you can see in the diagram blocks.png, Python sees code as a series of nested rectangles.

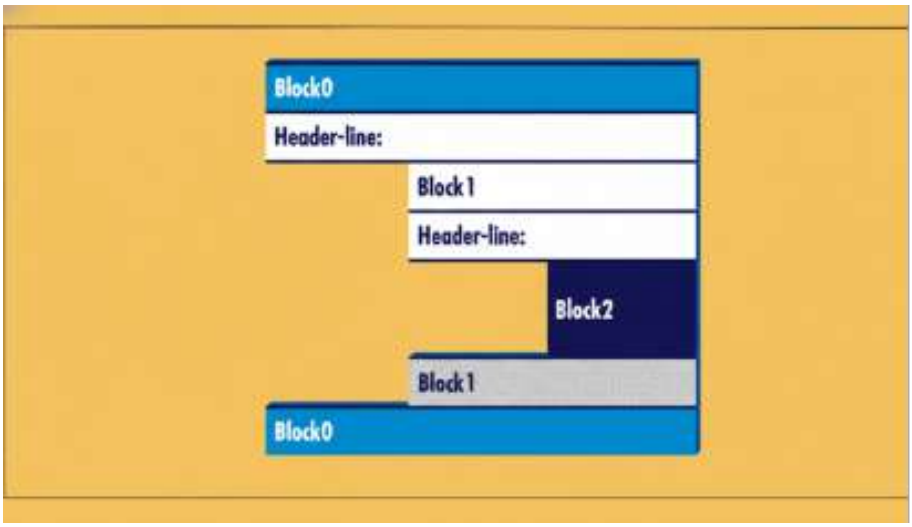


Figure 2.1: Indentation in python

Block 0: is your main program (the outermost level).
Block 1: is the code inside your function.
Block 2: would be code even deeper, like an "if statement" inside that function.

Notice how Block 1 starts after a "Header-line" (our def line). As long as your code stays aligned vertically, Python knows it belongs to that header. While you can use any number of spaces, the most common style is to use 4 spaces for every level of indentation.

If this sounds complicated, just remember: consistently indented code always satisfies Python's rules. If your code is "lined up," Python is happy.

FIX THIS!

Question 1 Indentation: Even a small mistake in your "grammar" will cause your program to crash. Look at the code below. Can you spot why Python will throw an `IndentationError`? Why?

broken_code.py

```
def calculate_tax(price):
tax_rate = 0.15
total = price + (price * tax_rate)
print(total)
```

The Problem: Because the lines under `def` are not pushed to the right, Python thinks the function is empty!

Question 2: The Vanishing Variable

Why does the final line of this program cause an error?

scope_test.py

```
def my_function():
    secret_code = "12345"
    print("Inside:", secret_code)

my_function()
print("Outside:", secret_code) # ERROR!
```

Hint: Think about what happens to a function's "tools" once it finishes running.

Question 3: The Ghost Code

Can you predict the output of these two blocks of codes ?

ghost.py

```
def say_hello_0():
    print("Hello!")
#         VS
def say_hello_1():
    return "Hello!"
print(say_hello_0())
print(say_hello_1())
```

Hint: The `return` statement is like an "Exit" sign, and the `print` function has a side effect only(will be covered in section 2.3).

There Is No Dumb Question

David: Feynman, I keep hearing people use the words "parameter" and "argument" like they're the same thing. But in the code, they seem to happen at different times. Is there actually a difference, or is it just tech-talk?
Feynman: It's a classic point of confusion, David, but the distinction is actually quite beautiful once you see it. Think of it like a **job description** versus the **person** who actually shows up to do the work.

- **Parameters** are the placeholders. They are part of the function's "contract"—they tell the world, "This function needs these specific pieces of information to work." You find these defined in the function header (the `def` line).
- **Arguments** are the actual values you "pass" into the function when you call it. You can provide different arguments every single time you use the function.

David: So, if I have a function `calculate_tax(price)`, then `price` is the parameter?
Feynman: Exactly! `price` is the parameter—it's just a variable waiting for a value. But when you later type `calculate_tax(10.50)`, that `10.50` is the argument.
Feynman: Parameters are the "slots", and arguments are the "data" you plug into those slots. Does that clear the air?
David: Perfectly. One is the requirement, the other is the delivery.

2.2 Passing Arguments

Since a single function can be defined with multiple parameters, a function call must provide a corresponding set of arguments. In Python, you have the flexibility to choose how that data is delivered. There are three primary strategies for passing arguments, each serving a specific structural purpose:

2.2.1 Positional Arguments

The most fundamental way to call a function is by using positional arguments. In this method, Python matches each argument in the function call to a parameter in the definition based strictly on the **order** in which they appear.

To see this in practice, consider a function designed to format an employee's full name and print it:

person.py

```python
def describe_person(first_name, last_name):
    #Combines names into a single formatted string.
    print("User Identity: " + first_name + " " + last_name)

# Matching by position: "Richard" fills first_name, "Feynman" fills last_name
describe_person("Richard", "Feynman")

# Reversing the order changes the output entirely
describe_person("Feynman", "Richard")
```

Terminal Output

```
User Identity:  Richard Feynman
User Identity:  Feynman Richard
```

The function header indicates that `describe_person` requires two pieces of information: a `first_name` and a `last_name`. When we execute the call, Python maps the first value, `"Richard"`, to the first parameter, and the second value, `"Feynman"`, to the second.

This method is efficient but requires the developer to maintain a precise understanding of the function's structure. If the order is accidentally swapped—as seen in the second call—the program will not crash, but it will process the data incorrectly, potentially leading to logical errors in larger systems.

Multiple Function Calls

You can call a function as many times as needed. Describing a second, different person requires just one more call:

repeat$_c$alls.py

```python
describe_person("Richard", "Feynman")
describe_person( "Henderson" , "Leonardo")
```

Terminal Output

```
Richard Feynman
Henderson Leonardo
```

Calling a function multiple times is a very efficient way to work. The code that describes a person is written once in the function. Then, anytime you want to describe a new person, you simply call the function with the new person's information.

Order Matters in Positional Arguments

If you mix up the order, you get unexpected results:

order_error.py

```
# Mixing up the order
describe_person("Henderson" , "Leonardo")
```

Terminal Output

```
Henderson Leonardo
```

Now the output becomes “You are Henderson Leonardo”, which is incorrect because the last name was passed into the first name slot.

2.2.2 Keyword Arguments

A keyword argument is a **name-value pair** that you pass to a function. Instead of relying on the sequence of the data, you directly link the name of the parameter to the value you want it to hold. This approach tells Python exactly where each piece of information belongs, regardless of where it appears in the list.

Keyword arguments eliminate the risk of "order errors" and significantly improve the readability of your script for others.

keywords.py

```
def describe_person(first_name, last_name):

    #Using keyword arguments allows us to be explicit.
    #The order no longer matters because each value is labeled.
    print("User Identity: " + first_name + " " + last_name)

# Even though the last name comes first in the call,
# Python correctly assigns the values based on the labels.
describe_person(last_name="Malen", first_name="David")
```

Terminal Output

```
User Identity:  David Malen
```

By explicitly labeling the data, you remove the guesswork. In the example above, Python sees `last_name="Malen"` and immediately looks for the parameter named `last_name` in the function definition, ignoring the fact that it was the first item provided in the call.

Important Note

When using keyword arguments, you must use the **exact** names defined in the function header. If the function expects `first_name` and you provide `fname`, Python will raise a `TypeError` because it cannot find a matching "slot" for that label.

2.2.3 Default Values

When designing a function, you can assign a "pre-set" value to one or more parameters. This is known as a "Default Value". If the user provides a specific argument when calling the function, Python will use that new value. However, if they omit that argument, Python will automatically fall back to the default you defined.

This technique is incredibly useful for data that remains constant in most cases, as it simplifies your function calls and reduces the chance of errors.

For example, if Mr. Henderson is processing records for a local project in Ethiopia, we can set a default nationality to save time:

defaults.py

```python
def describe_person(first_name, last_name, nationality="Ethiopian"):
    """
    If no nationality is provided, it defaults to 'Ethiopian'.
    Otherwise, the provided argument takes priority.
    """
    print("User Identity: " + first_name + " " + last_name)
    print("Origin: " + nationality + "\n")

# Call 1: Using the default value
describe_person("Leonardo", "Henderson")

# Call 2: Overriding the default value
describe_person("David", "Malen", "American")
```

Terminal Output

```
User Identity:  Leonardo Henderson Origin:  Ethiopian
User Identity:  David Malen Origin:  American
```

Important Note

Python requires that any parameters with default values be placed **after** all parameters that do not have defaults. This ensures that Python can still correctly map positional arguments to the "required" slots before deciding whether to use the optional default ones.

Problem 8-3: Supermarket Registration

Mr.Henderson needs a script to register new stock for his grocery aisles. Write a function called `register_stock()` that accepts an `item_name` and its `price`. The function should print a confirmation message.

- Call the function once using **positional arguments** for "Organic Milk" at 4.50.
- Call the function a second time using **keyword arguments** for "Whole Grain Bread"

at 3.25.

Solution

The Code Solution

registration.py

```python
def register_stock(item_name, price):
    # This line combines the strings and converts the number to text
    print("ENTRY RECORDED: " + item_name + " | Price: \$" + str(price))

# Option 1: Positional call (Python matches data by order)
register_stock("Organic Milk", 4.50)

# Option 2: Keyword call (Data is labeled so order doesn't matter)
register_stock(price=3.25, item_name="Whole Grain Bread")
```

Step-by-Step Breakdown

1. **Setup:** We define the function with two parameters to act as slots for our data.
2. **Conversion:** We use `str(price)` inside the print statement to prevent a Type Error.
3. **Execution:** The first call relies on position (order), while the second uses labels (keywords) to tell Python exactly which value belongs to which parameter.

Terminal Output

```
ENTRY RECORDED: Organic Milk | Price:  $4.5
ENTRY RECORDED: Whole Grain Bread | Price:  $3.25
```

There Is No Dumb Question

David: Why do we use `str(price)` instead of just adding the variable?
Feynman: You can't add a "number" to a "word" directly in Python, David. The `str()` function turns the number into text so you can glue the whole sentence together.

—

Problem 8-4: The Default Supplier

Most of the store's produce comes from a "Local Farm." Update your `register_stock()` function to include a third parameter, `supplier`, with a default value of `"Local Farm"`.

- Register "Red Apples" at 0.80 using the default supplier.
- Register "Imported Cheese" at 15.00 from a specific supplier: `"Swiss Deli"`.

Solution

The Code Solution

stock$_{d}efaults.py$

```python
def register_stock(item_name, price, supplier="Local Farm"):
    # This function uses a default value if no supplier is provided
    print("Item: " + item_name + " (\$ " + str(price) + ")")
    print("Source: " + supplier )

# Case 1: Using the default (only 2 arguments provided)
register_stock("Red Apples", 0.80)

# Case 2: Overriding the default (3 arguments provided)
register_stock("Imported Cheese", 15.00, "Swiss Deli")
```

Step-by-Step Breakdown

1. **Default Parameter:** We add `supplier="Local Farm"` at the end of the list.
2. **The Call Logic:** In the first call, Python sees only two pieces of data and automatically fills the third slot with our default value.
3. **The Override:** In the second call, we provide a third argument, which forces Python to discard the default and use "Swiss Deli" instead.

Terminal Output

```
Item:  Red Apples ($ 0.8)
Source:  Local Farm

Item:  Imported Cheese ($ 15.0)
Source:  Swiss Deli
```

There Is No Dumb Question

David: What happens if I put the `supplier` parameter first in the list?
Feynman: Python will stop you with a Syntax Error. You must list the "required" information first before the "optional" default ones.

—

Problem 8-5: Checkout Calculation

Create a function called `calculate_sale()` that takes a `price` and a `quantity`. Since most customers buy exactly one unit, set the default `quantity` to 1.

- Calculate the total for a single "Steak" at 12.50.
- Calculate the total for 12 "Soda Cans" at 1.05 each using keyword arguments in reverse order (quantity first).

Solution

The Code Solution

checkout.py

```python
def calculate_sale(price, quantity=1):
    # Calculate total and store it in a local variable
    total_cost = price * quantity
    print("TOTAL DUE: \$" + str(total_cost))

# Transaction 1: Only the price is sent (quantity defaults to 1)
calculate_sale(12.50)

# Transaction 2: Keywords allow us to flip the order in the call
calculate_sale(quantity=12, price=1.05)
```

Step-by-Step Breakdown

1. **Math Logic:** The function multiplies price and quantity to find the total.
2. **Default Handling:** By setting `quantity=1`, the function works even if we forget to provide the amount.
3. **Labeling Data:** The keyword call (`quantity=12`) allows us to be flexible with how we send the data into the function.

Terminal Output

```
TOTAL DUE: $12.5
TOTAL DUE: $12.6
```

There Is No Dumb Question

David: Does the computer get confused if I put the quantity before the price in the last call?
Feynman: Not at all! Because you used the "keyword" labels, Python knows exactly where each value belongs regardless of which one you typed first.

2.3 Return Values

Up until now, our functions have been like one-way streets. We send data in, the function prints something, and then the data vanishes. But in the real world, you often need a function to do some work and then **hand the result back** to you so you can use it later. This is what the `return` statement is for.

When you call a built-in function like `len('Hello')`, the call evaluates to `5`. That integer is the **return value**. When creating your own functions, you use the `return` keyword to specify exactly what data should be sent back to the line of code that called it.

Important Note: Print vs. Return

There is a massive difference between showing a result and handing it back to the program:

- **Print:** Displays text on the screen for a human to see. The program "forgets" this value immediately.
- **Return:** Hands a value back to the code. This allows the program to store the result in a variable, pass it to another function, or save it to a database.

Figure 2.2: The left side shows a print side-effect, while the right side shows a value being returned to the caller.

Comparing the Two Versions

To see the difference in action, consider these two versions of the same task:

version$_{1p}$rint.py

```python
# Version 1: Only prints (no return)
def print_formatted_name(first, last):
    full_name = first + " " + last
    print(full_name.title())

# This shows the name, but you cannot save it to a variable!
print_formatted_name("jimi", "hendrix")
```

version$_{2r}$eturn.py

```python
# Version 2: Returns a value
def get_formatted_name(first, last):
    full_name = first + " " + last
    return full_name.title()

# Now the name is stored in 'musician' and can be used later.
musician = get_formatted_name("jimi", "hendrix")
print(musician)
```

Using `return` makes your functions much more powerful and reusable. Functions that only print are useful for quick output, but functions that return values act as building blocks in larger programs.

—

Problem: Basic Salary Calculator

Mr. Henderson needs to calculate the weekly pay for a part-time staff member. Write a function called `get_weekly_pay()` that takes `hourly_rate` and `hours_worked`.

- The function should return the total pay.
- Store the result in a variable called `total_pay`.
- Print a sentence showing the final calculated amount.

Solution

The Code Solution

salary.py

```python
def get_weekly_pay(hourly_rate, hours_worked):
    # Calculate the product of rate and time
    payment = hourly_rate * hours_worked
    # Send the final number back to the main program
    return payment

# We catch the returned number in the total_pay variable
total_pay = get_weekly_pay(15.00, 20)

# We use the returned value to print a summary
print("Staff Weekly Pay: \$" + str(total_pay))
```

Step-by-Step Breakdown

1. **Calculation:** We define the math inside the function and store it in a temporary variable `payment`.
2. **The Hand-off:** The `return` keyword exits the function and carries the number back to the caller.
3. **Assignment:** The variable `total_pay` acts like a bucket that catches the number falling out of the function.
4. **External Use:** Because we returned the value, we can now use it in a print statement outside the function.

Terminal Output

```
Staff Weekly Pay:  $300.0
```

There Is No Dumb Question

David: Why can't I just print the payment inside the function?
Feynman: You could, David, but then the rest of your program wouldn't know what that number was! By using `return`, you give the rest of the code the ability to use that number for other things, like adding it to a monthly total or tax calculation.

—

Problem : Item Price Formatter

Create a function called `format_price()` that takes a raw `price` (number) and returns a string with a dollar sign and two decimal places (e.g., "$5.00").

- The function should return the formatted string.
- Use the function for a price of 9.99 and store it in `label`.
- Print the variable `label`.

Solution

The Code Solution

formatter.py

```python
def format_price(price):
    # Turn the number into a string and add the dollar sign
    display_string = "\$" + str(price)
    # Return the new string to the caller
    return display_string

# Storing the returned string into the label variable
label = format_price(9.99)

print("Shelf Label: " + label)
```

Step-by-Step Breakdown

1. **String Creation:** We use `str()` to convert the float price into text and prepend a dollar sign.
2. **Return Value:** We send the resulting `display_string` back out.
3. **Variable Storage:** The variable `label` captures the string returned by the function.
4. **Output:** We print the final label, demonstrating that the function successfully processed the data and handed it back.

Terminal Output

```
Shelf Label:  $9.99
```

There Is No Dumb Question

David: Can a function return more than just numbers or strings?
Feynman: Absolutely! It can return almost anything in Python. For now, focus on how the `return` statement acts like a delivery truck, bringing the results of the function's internal work out to the main streets of your program.

There Is No Dumb Question

David: Teacher, do we always know exactly how many arguments will be passed to a function when we are writing it?
Feynman: No, we don't actually. Think about a function that calculates a sum. Sometimes the user might want to add two numbers, and other times they might want to add a hundred. If you hard-code it for only two, the function becomes useless the moment a third number arrives.
David: Oh! Then how do we handle that? Is it even possible to write one function that handles any amount of data?
Feynman: Most things are possible in the world of logic, David.
David: How?
Feynman: Well, Python has a special "packing" operator that lets a function say, "I'll take whatever you throw at me." To see how that works, let's move on to the next section!

2.4 Passing an Arbitrary Number of Arguments

In many real-world situations, you cannot predict how many inputs a function will receive. Python solves this problem by allowing functions to accept an **arbitrary number of arguments** using a special parameter syntax.

This is done by placing an asterisk `*` before a parameter name. This tells Python to collect all extra arguments into a single structure.

Key Idea

- The parameter `*name` gathers multiple values into one variable.
- These values are automatically stored as a **tuple**.

Example: Printing Shipment Items

inventory$_{l}ist.py$

```python
def print_shipment(*items):
    # Display all received items
    print(items)

print_shipment("Apples")
print_shipment("Apples", "Milk", "Bread")
```

Terminal Output

```
('Apples',)
('Apples', 'Milk', 'Bread')
```

Explanation

The parameter `*items` acts as a collector for every argument provided during the function call. Even when you only pass a single value, Python still bundles it into a **tuple** structure. This is why you'll see a trailing comma in the output for a single item, like `("Apples",)`, which is simply Python's way of identifying the object as a tuple rather than a standard string in parentheses.

2.4.1 Mixing Positional and Arbitrary Arguments

A function can easily handle both fixed positional arguments and an arbitrary number of extras, but Python enforces a strict structural rule: the arbitrary parameter (`*args`) must always be placed **last** in the function definition.

Why This Rule Exists

This order is essential for how Python parses your code. It first matches the specific positional and keyword arguments to their named slots; once those are filled, it scoops up every remaining piece of data and packs it into the final `*` parameter. If the "collector" came first, it would swallow every argument you provided, leaving nothing left for the required parameters.

Essential Rules for *args

1. **The Catch-All:** The asterisk (`*`) tells Python to gather any number of arguments into a single parameter name.
2. **The Tuple Container:** All collected arguments are stored in a **tuple**, which is an ordered, unchangeable collection.
3. **The Trailing Comma:** When only one argument is passed, Python displays it with a comma (e.g., `('Apples',)`) to confirm it is a tuple.
4. **The Last Stand:** Always place `*args` at the end of your parameter list so Python can correctly assign your required positional values first.

—

Problem 8-10: Food Order Logger

A restaurant wants to build a simple system to record customer orders. Each order includes the customer's name followed by any number of food items.
Since different customers may order different numbers of items, the function must be flexible enough to handle this variation.

Your Task

- Create a function `log_order()`.
- The function must accept:
  - One required parameter: `customer_name`
  - A variable number of food items using `*items`
- Display:
  - The customer's name (clearly labeled)
  - The ordered items as a tuple
- Test your function with multiple examples.

Hint: Python automatically groups extra arguments into a tuple.

Solution

Implementation

order_logger.py

```python
def log_order(customer_name, *items):
    print("\n=== Order Summary ===")
    print("Customer:", customer_name)
    print("Items:", items)

# Test cases
log_order("Alice", "Burger", "Fries", "Soda")
log_order("Bob", "Pizza")
log_order("Charlie", "Pasta", "Salad")
```

Program Output

Terminal Output

```
=== Order Summary === Customer: Alice Items: ('Burger', 'Fries', 'Soda')
=== Order Summary === Customer: Bob Items: ('Pizza',)
=== Order Summary === Customer: Charlie Items: ('Pasta', 'Salad')
```

Step-by-Step Breakdown

1. **First Argument Assignment:** The value `"Alice"` is assigned to `customer_name`.
2. **Collecting Extra Arguments:** The parameter `*items` gathers all remaining arguments.
3. **Tuple Creation:** Python automatically stores the collected values as a tuple:

   `('Burger', 'Fries', 'Soda')`

4. **Single Item Case:** Even a single value becomes a tuple:

   `('Pizza',)`

5. **Why This Matters:** This approach allows the function to handle any number of inputs without modifying the function definition.

Terminal Output

```
Aisle Category: Dairy
Items Logged: ("Milk", "Yogurt", "Cheese", "Butter")
```

There Is No Dumb Question

David: Have we used a function like this before without realizing it?
Feynman: Yes! The built-in `print()` function itself accepts any number of arguments. You can pass one value, multiple values, or even none at all. Internally, it uses this exact concept of arbitrary arguments.

2.5 Local and Global Scope

Scopes

In my time of teaching introduction to programming classes, one of the most troubling topics for students is the distinction between global and local variables. To master Python, you must understand where your variables "live" and when they "die."

Variables and parameters created inside a function belong to that function's **local scope**. In contrast, variables defined outside of any function belong to the **global scope**. A variable that lives inside a function is known as a **local variable**, while one defined at the program level is called a **global variable**. A variable cannot exist in both scopes at the same time—it must belong to exactly one.

You can think of a scope as a **separate space that holds variables**. Once that space is removed, all the values stored inside it disappear.

- **The Global Scope:** This is the single, top-level scope that is created when the program starts running. It remains active throughout the program's execution and is removed only when the program finishes. If this were not the case, variables would keep their old values every time the program runs again.
- **The Local Scope:** A new local scope is created each time a function is executed. When the function finishes, that scope is discarded. As a result, when the function is called again, it starts fresh, with no memory of previous values. (This behavior will be explored further in **Section 2.6: The Call Stack**.)

There Is No Dumb Question

David: Teacher, why would the creators of Python make things so complex by inventing "scopes" for no reason? Why not just make everything a global variable?
Feynman: Oh David, local scopes have more uses than you might think. The reason Python has different scopes is so that when variables are modified inside a function, that function interacts with the rest of the program *only* through its parameters and the return value. This narrows down the number of lines of code you have to worry about.
David: But why do we want this kind of "limited" interaction? Wouldn't it be easier if every function could see everything?
Feynman: Imagine your program is thousands of lines long and has a bug because a variable was set to a bad value. If everything is global, that bad value could have been set from *anywhere*. It would be like looking for a needle in a haystack!
Feynman: But if the bug is caused by a local variable, you know that only the code in that **one specific function** could have set it incorrectly. You've turned a massive haystack into a tiny box.
David: Oh! Now I completely get it. It's about containing the chaos.
Feynman: that's right, David.

The Four Rules of Scope

1. **Global cannot use Local:** Code in the global scope, outside of all functions, cannot use any local variables.
   This is one example that shows how the walls of a function prevent the main program from seeing what happens inside. When you try to print a variable that was created as a local worker, Python will act as if it has never heard of it.

audience.py

```python
def audience():
    # local variable
    comment = "the readers of this book are really smart"

audience()
print(comment)  # ??? Can it be accessed outside the function.
```

Terminal Output

```
Traceback (most recent call last):
  File "audience.py", line 6, in <module>
    print(comment)
NameError: name 'comment' is not defined
```

Errors are Okay!

Don't be alarmed by the red text in your terminal! A **NameError** is Python's way of saying, "I looked everywhere in the global scope for a variable named `comment`, and I couldn't find it."
The error occurs because the variable `comment` exists **only** within the local scope of the `audience()` function. Once the function finishes running at line 4, that variable is destroyed. By the time the computer gets to line 6, the variable is already gone.

2. **Local can access Global:** Code in a local scope can access global variables (reading them).
This example demonstrates how a function can "reach out" and see variables defined in the global scope. As long as you are only reading the value, Python allows this interaction freely.

inventory_read.py

```python
def show_stock():
    # This function reads a global variable called 'milk_stock'
    # and prints its current value
    print(milk_stock)

milk_stock = 42  # Global variable representing total milk cartons in the supermarket

# Calling the function to display current stock
show_stock()

# Printing the global variable directly
print(milk_stock)
```

Terminal Output

```
42
42
```

Question

- Why is the value `42` printed twice?
- What is the difference between reading a global variable inside a function and printing it directly outside the function?

There Is No Dumb Question

David: Alright, I see that we can read global variables inside a function easily. But what if we actually want to *change* its value? Like if I wanted to update the score or a total count from inside the function?
Feynman: Really good question, David! That's exactly where things get tricky. If you try to change a global variable by simply assigning a value to it, Python will think you're just creating a brand new local variable with the same name.
David: So I'd be talking to a "local" version and the "global" one would stay the same?
Feynman: Exactly. It's a safety feature to stop you from accidentally breaking the rest of the program. To actually change a global variable, you need a special "permission slip" called the `global` statement.
David: can you give me an example?
Feynman: Ofc: If you need to modify a global variable from within a function, use the global statement. If you have a line such as global milk_stock at the top of a function, it tells Python: "In this function, milk_stock refer to the global variable, so we do not create a local variable with this name.Therefore, we will be modifying the global variable directly."

inventory_update.py

```python
def update_stock():
    # We declare that we want to modify the global variable 'milk_stock'
    global milk_stock

    # Updating the global inventory value
    milk_stock = "updated stock"

# Initial inventory value in the supermarket
milk_stock = "initial stock"

# Function call that changes the global inventory value
update_stock()

# Printing the updated global variable
print(milk_stock)
```

Terminal Output

```
updated stock
```

So the function "update stock" was able to access the variable milk stock and override it !

3. **Local scopes cannot use variables in other Local scopes**

   Just as the global scope cannot see inside a function, one function cannot see inside another.

Each time a function is called, it creates a new, private "room."

local$_{t}o_{l}ocal.py$

```python
def first_function():
    # A local variable in first_function
    x = 10
    second_function()

def second_function():
    # This will fail! second_function cannot see 'x'
    print(x)

first_function()
```

Terminal Output

```
Traceback (most recent call last):
  File "local_to_local.py", line 10, in <module>
    first_function()
  File "local_to_local.py", line 4, in first_function
    second_function()
  File "local_to_local.py", line 8, in second_function
    print(x)
NameError: name 'x' is not defined
```

Errors are Okay!

In this example, even though `second_function()` was called *from* `first_function()`, it still doesn't have access to the variables inside it.
Think of it this way: just because you invited a guest into your house (called a function) doesn't mean they can see what is hidden inside your private safe (local variables). When the computer is inside `second_function`, it only looks in its own local scope and the global scope. It never looks "sideways" into other functions.

4. **No Local-to-Local Access:** A function cannot access variables that belong to another function's local scope.
   You can think of each function as a separate storage room in a supermarket. Even if one function calls another, the items stored inside the first function remain completely hidden from the second. Python strictly isolates each function's data.

inventory_isolation.py

```python
def receive_stock():
    # Local inventory item inside the first function
    apples = 99

    # Calling another function
    check_stock()

def check_stock():
    # Attempting to access a variable from another function's scope
    print(apples)

# Start the process
receive_stock()
```

Terminal Output

```
Traceback (most recent call last):
  File "inventory_isolation.py", line 10, in <module>
    receive_stock()
  File "inventory_isolation.py", line 5, in receive_stock
    check_stock()
  File "inventory_isolation.py", line 9, in check_stock
    print(apples)
NameError: name 'apples' is not defined
```

Understanding the Error

Do not be misled by the fact that `check_stock()` is called inside `receive_stock()`. This does not give it access to the first function's variables.
When Python executes `check_stock()`, it only searches in:

- Its own local scope
- The global scope

It does **not** look into the local scope of any other function. Since `apples` exists only inside `receive_stock()`, it is invisible to `check_stock()`.

There Is No Dumb Question

David: Wait, so even if `first_function` is the "parent" that called the "child" `second_function`, they still can't share their variables?
Feynman: Exactly, David! If functions could reach into each other's local scopes, changing a variable in one function might accidentally break other functions that happen to be running at the same time.
David: So each function is truly its own little world?
Feynman: Precisely. This makes your code modular. You can write `second_function` without ever needing to know what names the person who wrote `first_function` used.

5. **The Same Variable Name Can Exist in Different Scopes**

This example demonstrates that separate scopes act like independent storage areas. Because of this separation, you can reuse the same variable name in different parts of a program without causing conflicts. Each scope keeps its own version of the variable.

same$_n$ames.py

```
def dairy_section():
    item = 'milk (dairy)'
    print(item)     # prints 'milk (dairy)'

def bakery_section():
    item = 'bread (bakery)'
    print(item)     # prints 'bread (bakery)'
    dairy_section()
    print(item)     # prints 'bread (bakery)' , why does it stay
        the same?

item = 'store default'
bakery_section()
print(item)         # prints 'store default', why is this
    unchanged?
```

Terminal Output

```
bread (bakery)
milk (dairy)
bread (bakery)
store default
```

There Is No Dumb Question

David: If different parts of the program use the same variable name, won't they interfere with each other?
Feynman: Not at all. Think of it like different storage rooms in a supermarket. Each room can have a box labeled "item," but what's inside depends on the room you're in.
David: I see. We can access global variables easily... but what if we want to modify one inside a function?
Feynman: Good thinking. To change a global variable from within a function, you must explicitly declare it using the `global` keyword. Otherwise, Python treats it as a new local variable.

6. **Same Names, Different Scopes:** You can use the same name for different variables if they are in different scopes. That is, there can be a local variable named `spam` and a global variable also named `spam`. Python treats them as two completely different entities.

scopes_example.py

```python
def local_function():
    spam = "I am local" # Local variable
    print(spam)

spam = "I am global" # Global variable
local_function()
print(spam)
```

Terminal Output

```
I am local
I am global
```

2.6 The Call Stack

Understanding the **Call Stack** is at the absolute foundation of a computer science technique called *recursion*. even though recurrence will not be covered in this specific part of the topic, it is the one thing that often ignites self-doubt and makes students want to quit. To master it later, you need to stay tight and be prepared now by mastering the Call Stack.

In order to understand the Call Stack, let's start with this analogy(depicted in the figure below). Imagine you are at your desk. You start your day by **Checking Email**. But as you are typing, your boss walks in and asks you to **Attend a Meeting**. You don't just delete your half-written email; you "pause" that action, move to the next, and put the meeting task on top of it.

While in the meeting, you are asked to **Impress Investors** with a presentation. You stop the meeting discussion to pull up your slides. While presenting, a colleague suddenly needs you

to **Help Jake** with a software bug. As shown in the image below, your "to-do" pile is now a stack of four tasks.

When you finish helping Jake, you start **unrolling** back. You can't go back to your email until the meeting is over, and you can't finish the meeting until you've helped Jake. You move backwards through the pile until you finally reach the bottom.

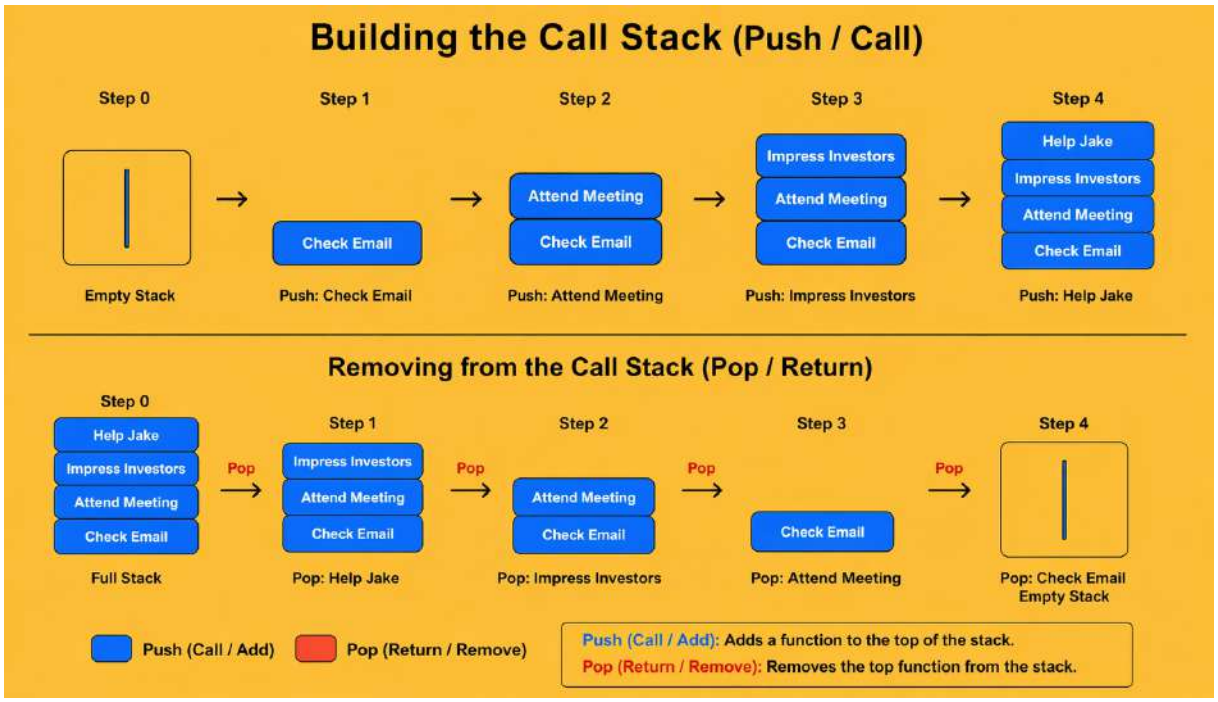


Figure 1: Pushing and popping from a stack.

Understanding Task Order

Imagine you are working through a list of tasks. You begin with one task, but before finishing it, new tasks keep getting added on top.

Scenario:

- You start with: `"Check Email"`
- Then a new task appears: `"Reply to Manager"`
- Then another: `"Help Jake"`

Each new task interrupts the previous one and is placed on top of the stack.

Question: Before you can return to `"Check Email"`, which task must you complete first?

Solution

Step-by-Step Breakdown

1. The most recent task added is `"Help Jake"`.
2. Since it is on top, it must be completed first.
3. After finishing it, you move to `"Reply to Manager"`.
4. Only after completing that can you return to `"Check Email"`.

Key Concept

This behavior follows the principle of **Last-In, First-Out (LIFO)**:

- The last task added is the first one completed.

Comparison

- **LIFO (Stack):** Last task added → First task completed
- **FIFO (Queue):** First task added → First task completed (like a checkout line)

There Is No Dumb Question

David: So the computer is just keeping a giant "mental pile" of where it is in the middle of a hundred different functions?
Feynman: Exactly! When Python calls a function, it "pushes" a new frame onto the stack. When that function returns, Python "pops" it off and looks at the frame directly underneath to see where it left off.
David: And if the pile gets too high?
Feynman: Then you get the famous *Stack Overflow*. You've run out of room on the desk!

2.6.1 The Call Stack in Action

To understand how the call stack works, let's examine the execution of a Python program step by step. This structure is what allows Python to manage the "pile" of tasks we discussed in the previous section.

The Example Program

call_stack_complex.py

```python
def a():
    print("a() starts")
    b()
    d()
    print("a() returns")

def b():
    print("b() starts")
    c()
    print("b() returns")

def c():
    print("c() starts")
    print("c() returns")

def d():
    print("d() starts")
    print("d() returns")

a()
```

Terminal Output

```
a() starts
b() starts
c() starts
c() returns
b() returns
d() starts
d() returns
a() returns
```

Execution of the Program

Let's carefully follow what happens when the program runs. Each time a function is called, execution "pauses" in the current function and moves to the new one.

1. **Starting Point:** The program begins by calling `a()`. Execution enters `a()` and outputs `a() starts`.

2. **a() Calls b():** Inside `a()`, the function `b()` is called. Execution pauses in `a()`, control moves to `b()`, and it outputs `b() starts`.

3. **b() Calls c():** Inside `b()`, the function `c()` is called. Execution pauses in `b()`, control moves to `c()`, and it outputs `c() starts`.

4. **c() Finishes:** Function `c()` does not call any other functions. It outputs `c() returns` and returns to the point in `b()` where it was called.
5. **b() Finishes:** After `c()` returns, `b()` outputs `b() returns` and returns to `a()`.
6. **a() Calls d():** Now `a()` continues and calls `d()`. Execution pauses in `a()`, moves to `d()`, and outputs `d() starts`.
7. **d() Finishes:** Output: `d() returns`. Execution returns to `a()`.
8. **a() Finishes:** Output: `a() returns`. The program ends.

How the Call Stack Works

The call stack is a special structure used by Python to keep track of these function calls.

Important

The Key Idea:

- When a function is called → it is added (**pushed**) onto the stack.
- When a function returns → it is removed (**popped**) from the stack.

Implicit Return: The Secret Life of Functions

Even though this program does not use the `return` statement explicitly, every function still returns.
Key Rule: If a function does not have a `return` statement, Python automatically adds `return None` at the very end. For example, function `c()` behaves like this internally:

c_internal.py

```python
def c():
    print('c() starts')
    print('c() returns')
    return None # Added automatically by Python!
```

What Happens When a Function Ends

When a function reaches a `return` statement OR reaches the end of its code:

1. It returns a value (explicitly or implicitly `None`).
2. Its frame is removed (popped) from the call stack and destroyed.
3. Execution continues in the calling function.

Step-by-Step Stack Changes

We represent the stack from **bottom** → **top**. As you follow these steps, refer to the diagram in **Figure 3.5**.

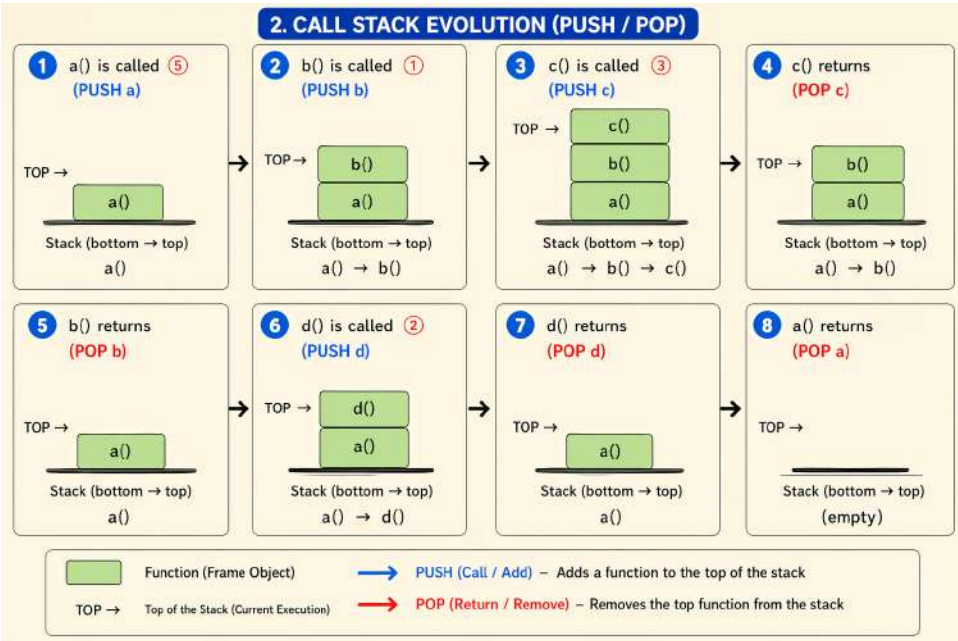


Figure 2: The Call Stack frames for the example program.

1. **a() is called:** Stack: [a]
2. **b() is called from a():** Stack: [a → b]
3. **c() is called from b():** Stack: [a → b → c]
4. **c() returns:** Stack: [a → b]
5. **b() returns:** Stack: [a]
6. **d() is called from a():** Stack: [a → d]
7. **d() returns:** Stack: [a]
8. **a() returns:** Stack: [(empty)]

Important Observations

- **LIFO (Last In, First Out):** The last function called is always the first one to return.
- **Current Execution:** The function at the very top of the stack is the one currently running.
- **Return Behavior:** When a function finishes, execution returns to the exact line where it was called.

There Is No Dumb Question

David: Wait, so `a()` is called first, but it's the very last thing to finish?
Feynman: Exactly! Think of it like a set of nesting dolls. You have to open the outer one (`a`) to get to the inner ones (`b` and `c`). You can't close the outer doll until all the inner ones are back in their place.
David: So `a()` is just waiting for everyone else to finish their job?
Feynman: Precisely. Function calls go deeper, but returns come back step by step. Without this stack, Python wouldn't know which function to go back to after a call. It would be lost in the code!

Summary of Relationships

The structure of function calls in this program can be visualized as a tree:

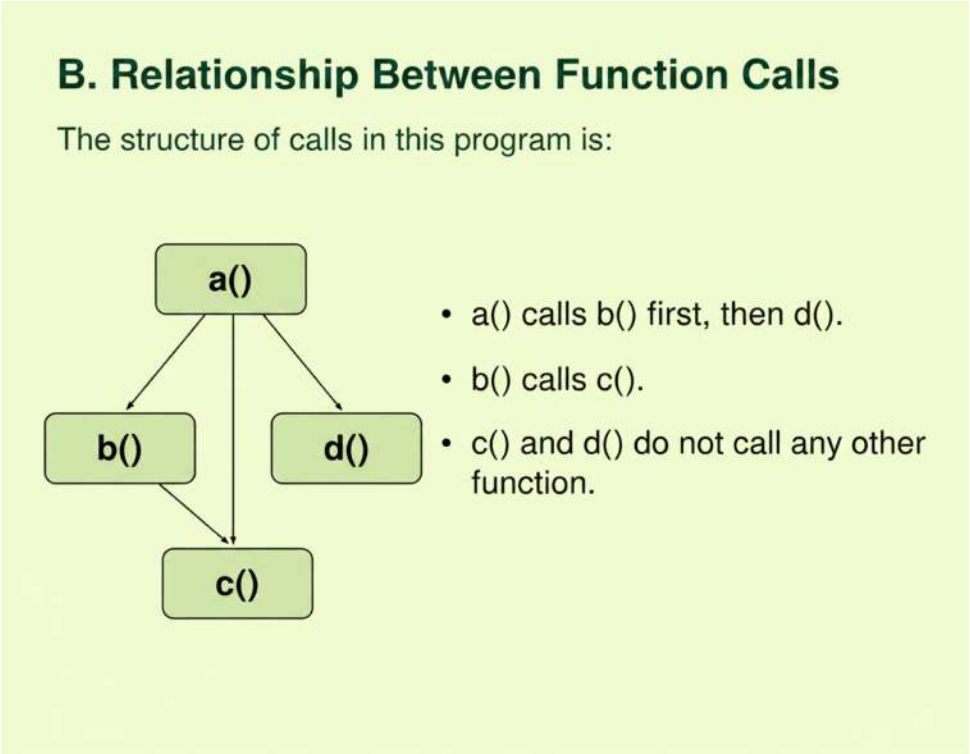


Figure 3: call relationship.

This shows that `a()` controls the overall flow, `b()` leads to `c()`, and `d()` is only called after `b()` has completely left the stack.

Fix the error in each code snippet and explain why it happens conceptually

1.Why does this cause a `SyntaxError`? Rewrite it correctly.

Python

```python
def calculate(a, b=10, c):
    return a + b + c
```

2.Why is this function call invalid?

Python

```python
def power(base, exp):
    return base ** exp

print(power(exp=3, 2))
```

3.Why does `x` remain unchanged? How can you modify the global variable?

Python

```python
x = "Global"

def change():
    x = "Local"

change()
print(x)
```

4. Why is `"HTTPS"` not assigned to `protocol`? Provide a correct call.

Python

```python
def stream(\*data, protocol):
    print(protocol, data)

stream("Packet1", "Packet2", "HTTPS")
```

5. Call Stack Growth (Function Calls)

```python
def first():
    second()

def second():
    third()

def third():
    fourth()

def fourth():
    fifth()

def fifth():
    first()

first()
```

This program crashes with a `RecursionError`.
Explain:

- How function calls are placed on the call stack
- Why the stack keeps growing even though no function calls itself directly

Then fix the code so the program terminates properly.

Functions: Problem Solving (Medium Level)

1.Write a function that takes a customer name and any number of food items using `*args`. The function should print a summary showing the customer and how many items they ordered.

```python
# Example function call:
order("David", "apple", "banana", "mango")
```

Expected Output:

```terminal
David ordered 3 items: apple, banana, mango
```

2.Write a function that takes any number of test scores and returns:

- the average score

```python
# Example function call:
analyze_scores(85, 90, 78)
```

Expected Output:

```terminal
Average: 84.33
```

3.Write a function that takes a full name as input and returns a username by:

- converting to lowercase
- removing spaces
- adding the length of the name at the end

```python
# Example function call:
create_username("David Malen")
```

Expected Output:

```terminal
davidmalen10
```

4.Write a function that takes multiple item prices and calculates:

- total price
- final price after a discount of 10%

```python
# Example function call:
calculate_bill(40, 60, 30)
```

Expected Output:

```terminal
Total: 130
Final after discount: 117.0
```

5.Write three functions:

- start()
- process()
- finish()

Each function should call the next one and print its own name so that the execution order is visible.

```python
# Start the program by calling:
start()
```

Expected Output:

```terminal
start
process
finish
```

6.Write a function that:

- takes a bill amount
- splits it equally among a number of people
- returns how much each person should pay

```python
# Example function call:
split_bill(120, 4)
```

Expected Output:

```terminal
Each person pays: 30.0
```

Program Writing (Functions: Real Applications)

1. Text Slow-Down Converter
Some people speak very quickly, and it can be helpful to visually slow their speech down by inserting pauses.
Write a program in `playback.py` that:

- prompts the user for a sentence
- replaces every space `" "` with `"..."` (three dots)
- prints the modified sentence

Rules:

- Do not remove characters
- Only replace spaces
- Output must preserve all original words

Example Runs:
Run 1

```terminal
Input: This is CS50
This...is...CS50
```

Run 2

```terminal
Input: This is our week on functions
This...is...our...week...on...functions
```

Run 3

```terminal
Input: Let's implement a function called hello
Let's...implement...a...function...called...hello
```

Goal:
Your program should:

- take user input using `input()`
- process the string using a string method
- output the transformed sentence

Hint: Think about a string method that replaces one character or pattern with another.

2. Tip Calculator

Complete the program below by implementing the missing functions.

```python
def main():
    dollars = dollars_to_float(input("How much was the meal? "))
    percent = percent_to_float(input("What percentage would you like to tip? "))
    tip = dollars * percent
    print(f"Leave \${tip:.2f}")

def dollars_to_float(d):
    # TODO

def percent_to_float(p):
    # TODO

main()
```

Requirements:

- `dollars_to_float(d)`:
  - Input: string like `"$50.00"`
  - Remove the leading $ symbol
  - Convert the remaining value to a float
  - Return: `50.0`
- `percent_to_float(p)`:
  - Input: string like `"15%"`
  - Remove the trailing % symbol
  - Convert the number to a float
  - Divide by 100 to get decimal form
  - Return: `0.15`

Example Run:

```terminal
How much was the meal? \$50.00
What percentage would you like to tip? 15\%
Leave \$7.50
```

3. String Cleaning Function
Write a function that:

- takes a string containing numbers and symbols
- removes all non-numeric characters (except `.`)
- returns the cleaned float

Example Runs:

- **Run 1**

```terminal
Enter value: \$50.00
50.00
```

- **Run 2**

```terminal
Enter value: price: 120.5 USD
120.5
```

- **Run 3**

```terminal
Enter value: 15% discount = 15.0
15.0
```

- **Run 4**

```terminal
Enter value: value@@@99.99!!!
99.99
```

Goal:
Your function should:

- extract only digits and the decimal point `.`
- ignore all other characters
- convert the result to a float using `float()`

4. Extract k-th Digit Function
Write a function:

```python
def extract_kth_digit(n, k):
    # TODO
```

The function should:

- take an integer `n`
- take a position `k` (counting from the right, starting at 1)
- return the k-th digit of `n`

Example:

```terminal
extract_kth_digit(48756378291, 3)
Output: 8
```

Hint: Use:

- `//` to remove digits

- `%` to isolate digits

5. Replace k-th Digit with 0
Write a function:

```python
def replace_kth_with_zero(n, k):
    # TODO
```

The function should:

- replace the k-th digit from the right with 0
- return the new number

Example:

```terminal
replace_kth_with_zero(48756378291, 3)
Output: 48756378091
```

6. Replace k-th Digit with a Given Value
Write a function:

```python
def replace_kth_digit(n, k, e):
    # TODO
```

The function should:

- replace the k-th digit from the right with `e`
- return the new number

Example:

```terminal
replace_kth_digit(48756378291, 3, 7)
Output: 48756378791
```

Final Hint (Core Idea of Chapter 2):
Instead of solving one fixed number, your goal is to:

- turn digit logic into reusable functions
- generalize the pattern for any input
- think in terms of parameters, not constants

Chapter 3

Flow Control and Data Structures

ASSUME

As you might recall, your journey began in the narrow aisles of Henderson's Supermarket. Back then, you were just a summer intern writing simple scripts to calculate tax. Since then, you have mastered the art of functions, arguments, parameters, and the intricacies of the call stack. You've moved beyond basic scripts into the world of clean, reusable logic. One afternoon, your hard work and technical growth are rewarded when the store manager calls you into the back office.

Mr. Henderson: "David, I've been looking over the code you wrote for our checkout system. The way you handled those complex tax calculations using functions was pure genius. It's clean, efficient, and, quite frankly, better than what some of our senior contractors provide."

David: "Thank you, Mr. Henderson. I just figured it was better to keep the logic modular so we could update the tax rates in one place."

Mr. Henderson: "Exactly! And that's why I'm making you a regular, full-time employee starting today. But with a promotion comes more power—and more responsibility. We're expanding. I don't just need a function for tax anymore; I need you to organize the entire store's inventory. We have thousands of items coming in, each with its own price and stock level. Can you handle it?"

David: "Boss, I am reading one of the best programming books ever, so I think I can handle it. Let's get to work."

3.1 The New Challenge

Now that you are a regular employee, you are working more hours and, more importantly, being paid for more hours. But as we know, with more power comes more responsibility. Your new role as Inventory Lead isn't just about writing a single function; it involves managing the digital heartbeat of the entire store.

Here are your new responsibilities:

- **Data Organization:** Storing thousands of unique product names, prices, and stock levels without creating a "variable explosion."
- **Inventory Auditing:** Automatically identifying which items are running low so the store never runs out of essentials.
- **Dynamic Lookups:** Building a system where a cashier can type in an item name and instantly retrieve its price.
- **Categorization:** Separating items into departments (e.g., Produce, Dairy, Bakery) to help Mr. Henderson understand store performance.

You quickly realize that you have tons to learn. This chapter will give you every concept needed to sustain your job and keep the supermarket running smoothly. We will cover:

- **Conditionals:** Making complex decisions based on stock and price levels.
- **Lists:** Grouping items together so they don't get lost in a sea of variables.
- **Dictionaries:** Linking item names directly to their specific data for instant retrieval.
- **Loops:** Automating your work so Python does the heavy lifting for you.

Stay tuned and take your notes!

3.2 Conditionals

Even as you step into your new role as a regular employee, you will quickly find that the programs you've written so far are remarkably "dumb." They follow a straight line from top to bottom without ever stopping to think. For example, even a task as simple as classifying a student or a customer by name would be hard for us because we have not covered a foundational topic called **Conditionals**.

Look at this simple snippet of code:

logic_check.py

```python
age = 20

if age > 18:
    print("pass")
else:
    print("wait some more years kiddo")
```

Even if you don't know anything else about Python syntax yet, you instinctively understand what is happening here. If the number stored in `age` is bigger than 18, the person passes. If not, the program prints a message telling them to wait.

Important

The Key Idea:

- Conditionals are the "brain" of your program.
- They allow our code to evaluate a situation and say: "If this is true, do that."

At its core, that is all a conditional is: a crossroads. It allows our program to stop, evaluate a condition, and choose which path to take. Without this, you cannot handle the reality of a supermarket—you can't handle discounts.

3.2.1 Boolean Expressions

As you might have guessed, before we can master the art of making decisions with conditionals, we first need to understand the "conditions" themselves—or, to use a more formal term: **Booleans**. A Boolean expression is simply a fancy name for a conditional test. In Python, a Boolean value is always one of two things: **True** or **False**.

Think back to the code we just saw. The part where we wrote `age > 18` is the Boolean expression. If we were to pull that out and run it on its own, it would look like this:

boolean_test.py

```python
age = 20
print(age > 18)
```

Terminal Output

```
True
```

We can also create Boolean variables directly to track the state of our supermarket. For instance, we might want to know if a customer is currently in the store:

tracking.py

```python
present = True
out_of_stock = False
```

We can come up with several Boolean expressions using the data types we already have in our toolbox. Whether we are checking the brand of a car or the age of a customer, the logic remains the same: we are asking Python a "Yes or No" question.

Comparing Strings

When working with strings, we often use the equality operator (`==`) to check if two pieces of text are identical, or the inequality operator (`!=`) to check if they are different. It is important to remember that Python is case-sensitive.

string_comparisons.py

```python
car = 'audi'

# Checking equality (This will be False)
print(car == 'bmw')

# Checking equality (This will be True)
print(car == 'audi')

# Testing Case Sensitivity (This will be False)
print(car == 'Audi')

requested_topping = 'mushrooms'
# Using the 'not equal' operator
if requested_topping != 'anchovies':
    print("Hold the anchovies!")
```

Terminal Output

```
False True False Hold the anchovies!
```

Comparing Numbers

When it comes to integers, we have a wider range of mathematical tests. These allow us to check if a value is within a specific range—perfect for checking stock levels or customer eligibility.

numerical_comparisons.py

```python
age = 18
# Is the age exactly 18?
print(age == 18)

age = 19
# Is the age under 21?
print(age < 21)

# Is the age less than or equal to 21?
print(age <= 21)

# Is the age greater than 21?
print(age > 21)

# Is the age greater than or equal to 21?
print(age >= 21)
```

Terminal Output

```
True True True False False
```

Summary of Comparison Operators

In summary, these are the primary Boolean expressions you are expected to know. Each of these operators acts as a question that results in a simple True or False.

Operator	Name	Description	Example (x=10)	Result
`==`	Equal to	Returns `True` if both sides are the same.	`x == 10`	`TRUE`
`!=`	Not equal to	Returns `True` if the sides are different.	`x != 10`	`FALSE`
`>`	Greater than	`True` if the left side is larger.	`x > 5`	`TRUE`
`<`	Less than	`True` if the left side is smaller.	`x < 5`	`FALSE`
`>=`	Greater than or equal	`True` if left is larger or exactly equal.	`x >= 10`	`TRUE`
`<=`	Less than or equal	`True` if left is smaller or exactly equal.	`x <= 9`	`FALSE`

Table 3.1: Python Comparison Operators Summary

Important

The Case Matters: Unlike many other programming languages, Python's Boolean values **must** start with a capital letter: `True` and `False`.

Formally, **Boolean** is a distinct data type with exactly two values. Because these are reserved "keywords," they have a protected meaning in Python. If we try to use them incorrectly, we will run into errors.

Terminal Output

```
>> true Traceback (most recent call last):  File "<stdin>", line 1, in <module>
NameError:  name 'true' is not defined
```

Terminal Output

```
>> True = 2 + 2 SyntaxError:  cannot assign to True
```

ERRORS ARE OKAY

Don't panic when you see these red messages!

- **NameError:** This happened because we used a lowercase `true`. Python thinks this is a regular variable name that we haven't defined yet.
- **SyntaxError:** This happened because we tried to change the value of `True`. Python is essentially saying: "You can't redefine the truth!"

Every programmer makes these mistakes. The terminal isn't yelling at you; it's just giving you the feedback needed to fix your logic.

QUESTION

Take a close look at the two code snippets below. One of them will work perfectly for our supermarket register, while the other will cause Python to throw an error and stop the program.
Can you identify the difference in behavior between the `=` operator and the `==` operator?

check_price.py

```
# Scenario A
price = 10
if price == 10:
    print("This item is ten dollars.")

# Scenario B
price = 10
if price = 20:
    print("This item is twenty dollars.")
```

The Challenge: Which scenario causes a `SyntaxError`.

3.2.2 Logical Operators: Combining Tests

There is one more set of tools we need to master before we delve into the formal syntax of conditionals. We can easily infer their necessity by trying to build a program that requires more than one condition to be met at the same time. These are called **Logical Operators**.

The 'and' Operator

Imagine Mr. Henderson asks you to find "High-Value/Low-Stock" items. These are rare items that we need to protect. To identify them, the price must be high **and** the quantity must be low.

rare_items.py

```
price = 15
stock_count = 2

# Both sides must be True for the whole expression to be True
if price > 10 and stock_count < 3:
    print("Rare item detected!") # This runs because 15 > 10 AND 2
        < 3
```

The 'or' Operator

Sometimes, we only need *at least one* of our conditions to be met. For example, if an item is either expired **or** the packaging is damaged, it must be removed from the shelf.

disposal_check.py

```
is_expired = False
is_damaged = True

# If either one (or both) is True, the whole thing is True
if is_expired or is_damaged:
    print("Dispose of item.") # This runs because is_damaged is
        True
```

The 'not' Operator

Finally, we have the `not` operator. Think of this as a "logic flipper." It is used when we want to check if a condition is **absent**. It takes whatever Boolean value follows it and reverses it: `True` becomes `False`, and `False` becomes `True`.

In our store, we might want to welcome customers only if the store is **not** closed.

status_check.py

```
is_closed = False

# 'not False' becomes True
if not is_closed:
    print("Welcome! We are open.")
```

Important

The Key Idea:

- **and:** Strict. Both sides must be `True`.
- **or:** Flexible. Only one side needs to be `True`.
- **not:** Inverse. Flips `True` to `False` and vice-versa.

Operator	Name	Description	Logic Example
`and`	Conjunction	Returns `True` only if both statements are true.	`(5 > 3) and (10 < 20)` → `True`
`or`	Disjunction	Returns `True` if at least one statement is true.	`(5 > 10) or (10 < 20)` → `True`
`not`	Negation	Flips the result (`True` becomes `False` and vice-versa).	`not (5 > 3)` → `False`

Table 3.2: Python Logical Operators Summary

3.3 If Statements

Up until now, we have discussed all the building blocks needed to approach our first major milestone: **If Statements**. Because you already understand how to write conditional tests, the actual syntax of an `if` statement will feel very natural. Several different kinds of `if` statements exist, and your choice of which to use depends entirely on the number of conditions you need to test.

But now, let's dig deeper into the topic.

3.3.1 Simple if Statements

The simplest kind of `if` statement has one test and one action:

syntax_example.py

```
if conditional_test:
    do something
```

In the first line, you can place almost any valid condition you want. If that expression evaluates to `True`, Python executes the indented code below it. In plain English, an `if` statement is read as: *"If this condition is true, execute the code in the clause."*

In Python, an `if` statement consists of the following:

- The `if` **keyword**.
- A **condition** (an expression that evaluates to `True` or `False`).
- A **colon** (`:`).
- Starting on the next line, an **indented block of code** (called the *if clause*).

Important

The Syntax Pattern: All flow control statements in Python (like `if`, `else`, and `elif`) end with a colon (`:`). This is followed by a block of code that is indented to show it belongs to that specific test.

To get an idea of how the Python interpreter reads through your code, look at how it handles a simple name check:

greeting.py

```
name = "david"

if name == "david":
    print("hi david")
```

To follow along with the execution, refer to the flowchart below (`flow1.png`). This diagram uses standard shapes to map out the computer's logic:

- **Ovals:** These represent the **Start** and **End** points of the logic flow.
- **Diamond:** This is the **Decision** point. It asks the question (the condition). It always has two paths coming out of it: one for `True` and one for `False`.
- **Arrows:** These indicate the direction of the **Flow**. They show exactly which line of code the interpreter moves to next.
- **Box:** This is an **Action** (like a `print` statement). Notice that in the diagram, the box is only reached if the arrow coming out of the diamond is `True`.

3.3.2 Refining the Logic

To see how these concepts fit together, let's try to solve a basic problem: comparing two numbers, `x` and `y`, and printing their relationship.

THE COMPARISON CHALLENGE

Write a program that takes two variables, `x` and `y`, and prints whether `x` is less than, greater than, or equal to `y`.

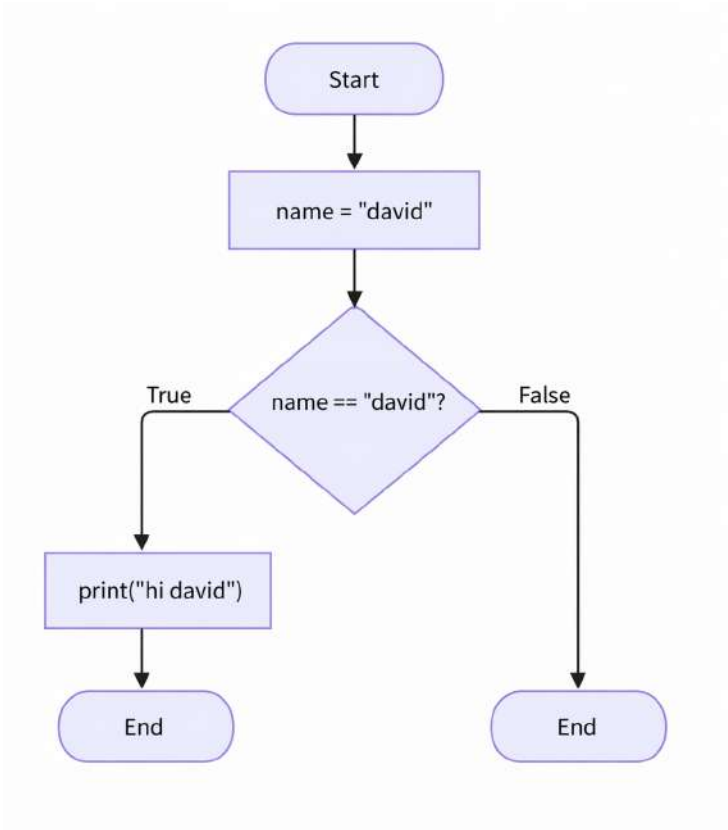


Figure 3.1: The execution path of a basic conditional test.

Solution

The Independent Ifs Initially, we might solve this by asking three separate questions. In this version, each statement is an independent crossroads. Even if the first test is `True`, Python will still stop and check the other two.

comparison_v1.py

```
x = 5
y = 10

if x < y:
    print("x is less than y")

if x > y:
    print("x is greater than y")

if x == y:
    print("x is equal to y")
```

There Is No Dumb Question

David: But teacher, as I can see in the program above, we asked three separate questions: Is it less? Is it greater? Is it equal? Doesn't the computer get tired of asking so many questions if it already found the answer in the first one?
Feynman: That's a sharp observation! In the code above, even if `x < y` is `True`, Python still stops to check if it's greater and then checks if it's equal. It's like checking every door in a hallway even after you've already found your room. It works, but it's "heavy" for the processor.

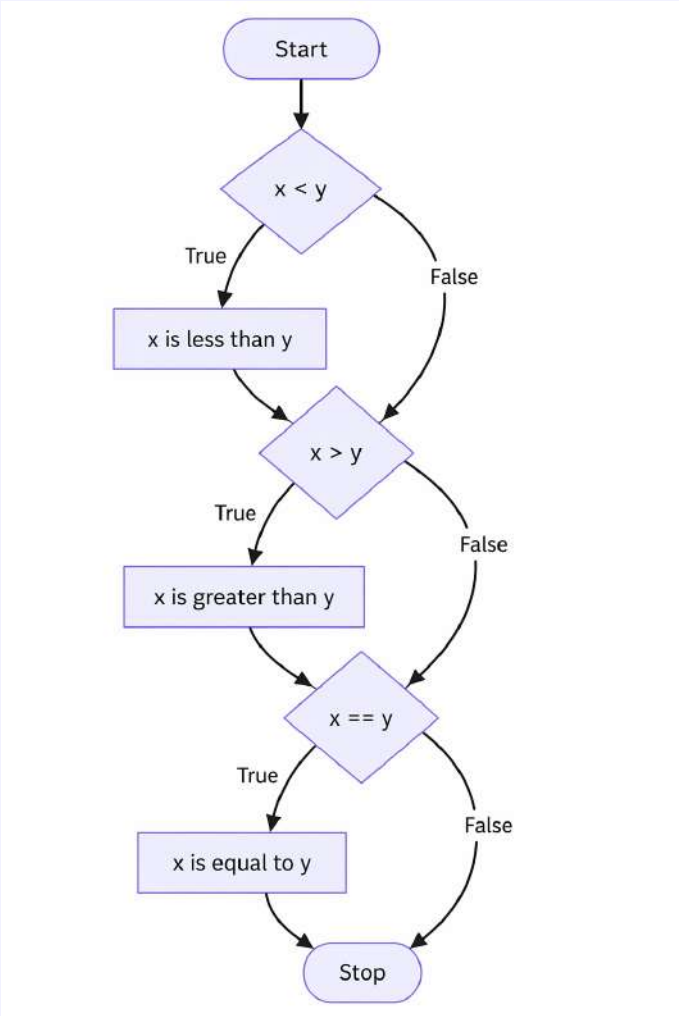


Figure 3.2: Independent 'if' statements checking every condition.

Feynman: To improve this, we use a new keyword called **`elif`**, which is short for "else if." It tells Python: "Only ask this question if the previous ones were `False`." This creates a *chain* of logic. As soon as Python finds one `True` statement, it skips the entire rest of the block and moves to the end.

comparison_v2.py

```
if x < y:
    print("x is less than y")
elif x > y:
    print("x is greater than y")
elif x == y:
    print("x is equal to y")
```

Feynman: Can you guess the flow diagram for this now?
David: I think it looks like a slide! If the first diamond is `True`, we execute the code and "slide" out the bottom of the block. We only hit the next diamond if we "fail" the one before it.

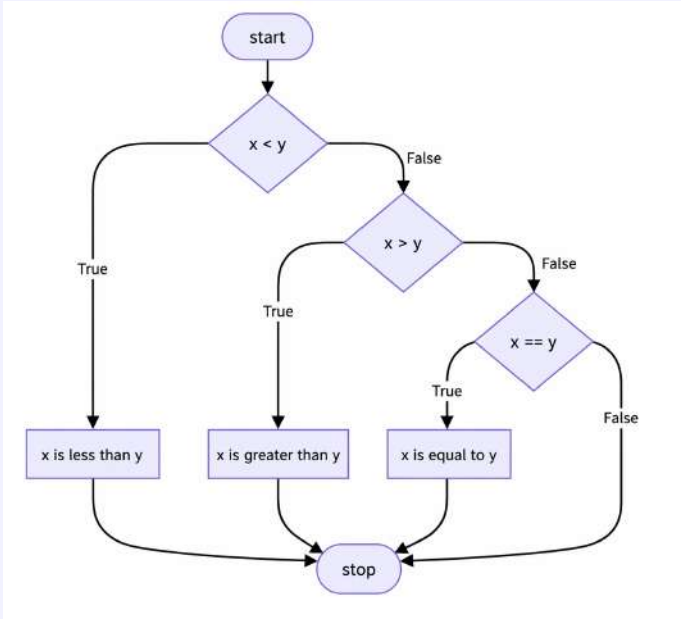


Figure 3.3: The 'elif' chain: Python skips the rest once a match is found.

Feynman: Spot on. But don't you think we could still improve this? If I tell you a number isn't "bigger" and it isn't "smaller" than another number, do you still need to ask if they are equal?
David: Wait... if two numbers are not less than each other, and they are not greater than each other... then they *must* be equal! We don't even need to ask that final question.
Feynman: Exactly! That is where the **`else`** keyword comes in. It is the "catch-all" or the "safety net." It doesn't take a condition because it only runs when *everything else* has failed.

comparison_final.py

```
if x < y:
    print("x is less than y")
elif x > y:
    print("x is greater than y")
else:
    # No condition needed! If we are here, they must be equal.
    print("x is equal to y")
```

David: So the final flow is this?

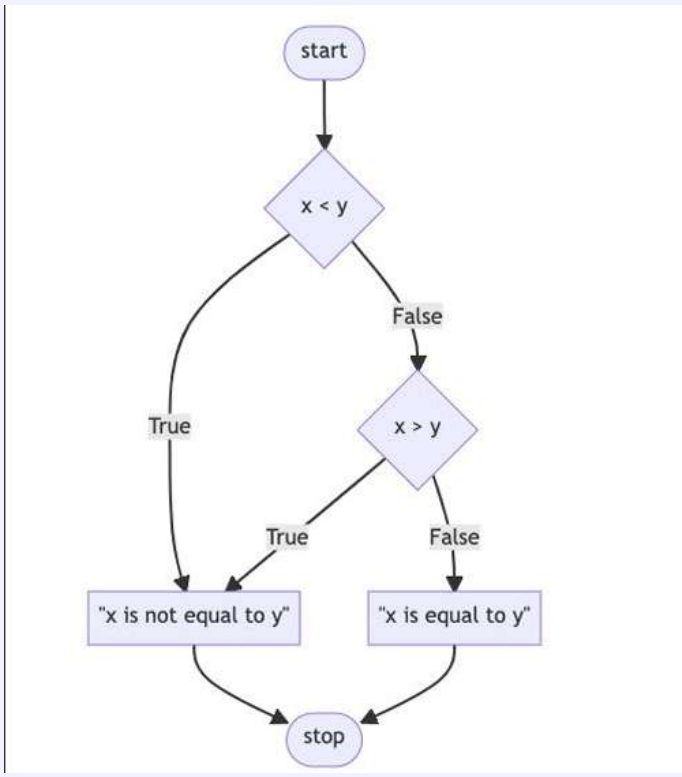


Figure 3.4: The optimized 'if-elif-else' structure.

Feynman: Exactly!

Important

The Efficiency of the Chain
As seen in the dialogue between Feynman and David, the primary difference between using multiple `if` statements and the `if-elif-else` structure is how the Python interpreter "thinks":

- **Independent `ifs`:** Python treats every `if` as a mandatory question. Even if it finds the answer in the first block, it is legally obligated to stop and check every single diamond that follows. This is like a security guard checking every ID in a line even after finding the person they were looking for.
- **The `elif` Chain:** This creates a single, connected logical unit. As soon as Python evaluates a condition as `True`, it executes that block and immediately "jumps" to the very end of the entire chain. It ignores all remaining `elif` and `else` blocks.
- **The `else` Safety Net:** The `else` block is the only part of the structure that requires no condition. It represents "logic by exhaustion"—it only runs if every single test above it failed.

Tip: Always use `if-elif-else` when your conditions are **mutually exclusive** (meaning only one of them can possibly be true at a time). It makes your code faster and prevents "logical double-dipping" where two different blocks might accidentally print something for the same input.

3.4 Practical Examples

Now that we have the theory of `if`, `elif`, and `else`, let's put your skills to the test with real-world scenarios ranging from academic grading to high-security systems.

Problem 1: The Grading System

Write a program that takes a student's score (0100) and assigns a letter grade based on the following scale:

- 90 or above: **A**
- 8089: **B**
- 70-79: **C**
- 6069: **D**
- Below 60: **F**

Hint: Use an `if-elif-else` chain. Since the computer checks from top to bottom, you don't need to check if the score is less than 90 once you move to the 'B' grade.

The Grading Chain

grader.py

```python
score = 85

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
else:
    print("Grade: F")
```

Problem 2: The Nerd List

To be placed on the "Nerd List," a student must meet these specific rules:

1. **Academic Excellence:** GPA of 3.8 or higher.
2. **Leadership:** Must be a Club President **OR** have led at least 2 major projects.
3. **Conduct:** Must **NOT** have any "Disciplinary Actions" on their record.

Hint: Use parentheses to group the `or` logic so Python treats the leadership requirements as one unit.

Logical Combination

nerd_list.py

```python
gpa = 3.85
is_club_president = False
projects_led = 3
has_disciplinary_record = False

if gpa >= 3.8 and (is_club_president or projects_led >= 2) and
    not has_disciplinary_record:
    print("Congratulations! You are on the Nerd List.")
else:
    print("Status: Criteria not met for Nerd List selection.")
```

Problem 3: The Security Vault

Imagine a high-security lab. Access is only granted if:

1. The user has a valid ID card.
2. Their fingerprint is recognized.
3. It is currently during authorized shift hours (before 6:00 PM / 18:00).

The Challenge: Provide specific feedback for **why** someone was rejected at each step. This requires the logic discussed in the next section.

The Security Hierarchy

*Note: The following solution utilizes the **Nested Conditionals** structure that will be explained in Section 3.5.*

vault_access.py

```
has_id = True
fingerprint_match = True
current_hour = 14

if has_id:
    print("ID Verified. Scanning fingerprint...")
    if fingerprint_match:
        print("Fingerprint matched. Checking shift time...")
        if current_hour < 18:
            print("Access Granted. Welcome to the Lab.")
        else:
            print("Access Denied: Shift has ended.")
    else:
        print("Access Denied: Unknown Fingerprint.")
else:
    print("Access Denied: No ID detected.")
```

3.5 Nested Conditionals

A nested conditional is a hierarchy. It creates a **parent-child** relationship between conditions, allowing for complex, multi-stage decision-making.

- **The Outer Layer (The Gatekeeper):** This handles the broadest requirement. If this fails, the computer ignores everything inside it, saving processing power.
- **The Inner Layer (The Specialist):** This handles "fine-grained" details. It only runs if the environment has been "vetted" by the outer layer.

Important

At its deepest level, a conditional is a point in time where the **Linear Flow** of code is interrupted. Every conditional boils down to a single Boolean expression. The computer doesn't understand "maybe"; it only understands 1 (`True`) or 0 (`False`).

There Is No Dumb Question

David: Imagine you are entering a high-security lab. Is this how nesting works?

1. **Outer If:** Do you have an ID card? If No: Go home. (The computer stops checking). If Yes: Proceed.

2. **Inner If (Nested):** Is your fingerprint recognized? If No: Alarm sounds. If Yes: Proceed.
3. **Third Nested If:** Is the current time within your authorized shift?

Feynman: Precisely! Think of it like a funnel. You only reach the narrow bottom if you pass through the wide top first. It keeps your code organized and prevents the computer from doing unnecessary work.

3.6 Data Structures

Problem

A single variable can store only one value at a time. That works for small tasks, but it becomes awkward when you need to manage many related values, such as the prices of items in a supermarket. If you try to create one variable for every product, you quickly run into `variable explosion`.

Important

This is the dead end: too many separate variables become difficult to organize, update, and read.

Solution: List

Python solves this problem with a **list**: a data structure that stores multiple values in one ordered container.

3.6.1 The List Data Type

A **list** is an ordered sequence of values. Unlike a string, which stores characters, a list can store integers, strings, booleans, `None`, or even other lists.

- **Syntax:** A list starts with `[` and ends with `]`.
- **Elements:** The values inside a list are called `items`, and they are separated by commas.

list_examples.py

```python
# Homogeneous list: same type of values
inventory = ['apples', 'bananas', 'tofu', 'milk']

# Heterogeneous list: mixed types
entry = ['Hello', 3.1415, True, None, 42]

# Empty list
shopping_cart = []
```

Important

A list is `mutable`, which means its contents can be changed after creation.

3.6.2 Accessing Data: Indexing

An **index** is a single position in a list. Python uses **zero-based indexing**, so the first item is at index `0`, not `1`.

Important

This is where many students make an **off-by-one** error. If a list has 4 items, the last valid index is `3`, because counting starts from `0`.

Index	0	1	2	3
Value	Bread	Butter	Cheese	Cream

Table 3.1: A list with four items uses indices from `0` to `3`.

`indexing_example.py`

```python
products = ['Bread', 'Butter', 'Cheese', 'Cream']
print(products[0]) # Valid
print(products[1]) # Valid
print(products[4]) # ???
```

Terminal Output

```
Bread
Butter
IndexError: list index out of range
```

Errors are Okay!

Python is signaling an `IndexError: list index out of range` because of the "Zero-Indexing" rule. Since we started counting at 0, our 4-item list ends at index 3. There simply is no "index 4" to grab.
You might think Python is being strictly "bad" or annoying by crashing your program here, but in reality, Python is being incredibly protective of you. To understand why, we have to look at how other languages, like C, handle this same mistake.

A Round of Applause for Python

My personal favorite language is **C**, but C is far less "caring" than Python. As we will see in coming sections, a list name is essentially just a **pointer**an arrow that points to a specific starting location in your computer's memory.

Imagine your computer's memory as a long row of mailboxes. In Python, if you ask for a mailbox that isn't yours (like index 4 in a 4-item list), Python stops you immediately and says, "Hey! That doesn't belong to you!"

In C, the language assumes you know exactly what you are doing. If you ask for index 4, C doesn't stop you. It simply moves to the very next physical slot in your RAM and hands you whatever "garbage" data happens to be sitting there. It might be a piece of another program, a random number, or even sensitive data. This can lead to silent bugs or massive security crashes that are nearly impossible to track down.

Python does the "heavy lifting" of monitoring these boundaries for us. It allows us to write code as "carelessly" as we want, knowing that it won't let us mess around in restricted memory areas. So, let's have a round of applause for Python's strictnessit's actually love in disguise!

There Is No Dumb Question

David: Wait, I think I get it. If the variable is just a "pointer" pointing at the very first box in memory, then index `0` means we don't add anything to the address? We are already standing on the item?

Feynman: Exactly! You've found the "Zero" in zero-indexing. In languages like C, the index is literally a math equation: $Address = Start + Index$. To stay at the start, you add zero. see the image below.

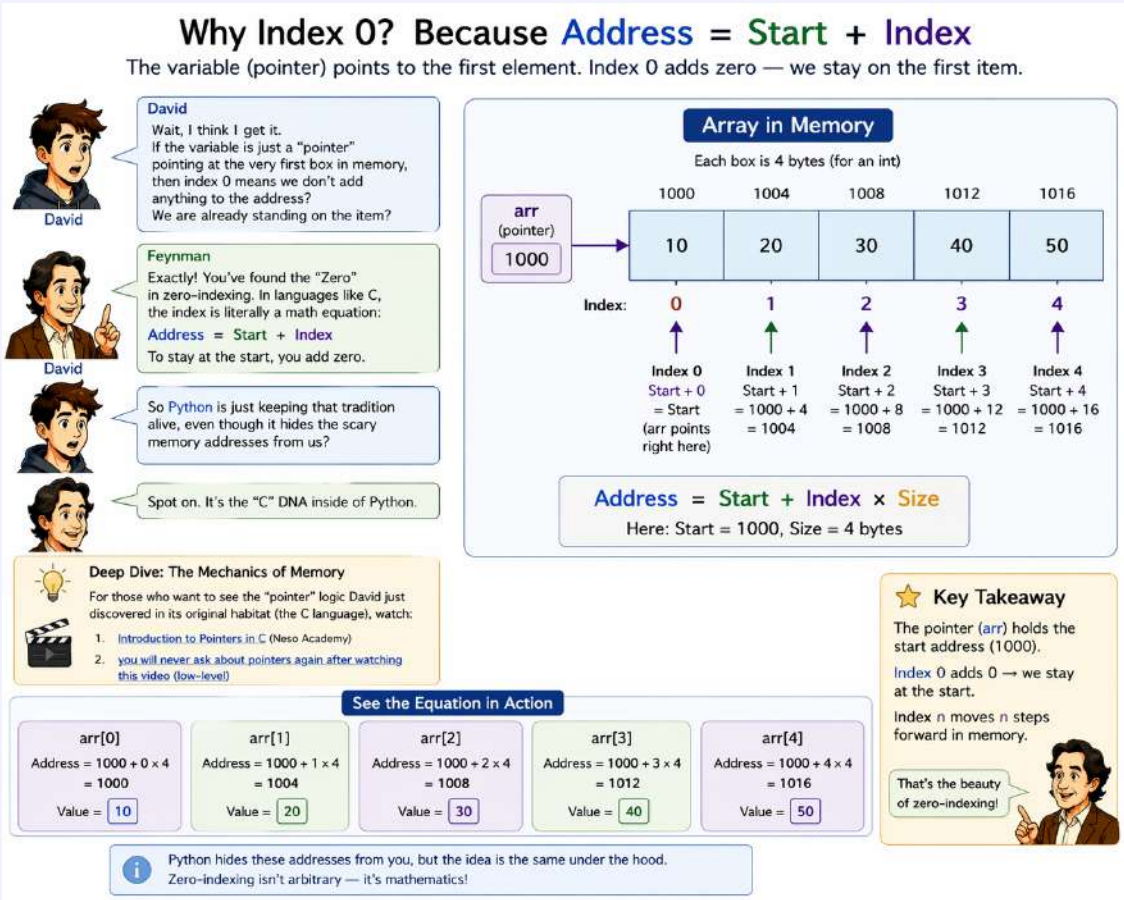


Figure 3.1: Visualized conversation on pointers.

David: So Python is just keeping that tradition alive, even though it hides the scary memory addresses from us?
Feynman: Spot on. It's the "C" DNA inside of Python. If you want to see the raw, "unsafe" version of this—where the computer lets you wander into memory locations you don't own—you should check this out:

Important

Deep Dive: The Mechanics of Memory
For those who want to see the "pointer" logic David just discovered in its original habitat (the C language), watch:

1. Introduction to Pointers in C (Neso Academy)
2. you will never ask about pointers again after watching this video(lowlevel

.

Important

A Note on "The Rabbit Hole"
You are **not required** to master the underlying memory arithmetic of pointers to finish this book. Python's greatest gift is that it handles these complexities for you.
I've included these insights for two reasons:

1. **Future Direction:** It gives you a clear "next step." If you enjoy the logic of how bits and bytes move, you'll know to study *Low-Level Systems* or *C Programming* after this.
2. **The Big Picture:** It never hurts to know how your tools actually work. Understanding the "why" behind the "what" makes you a more resilient problem-solver.

Who knows? One day, one of you might use this foundational knowledge to invent a better algorithm for memory management or a faster way to process data. Until then, just remember: **Index 0 is the start!**

3.6.3 Negative Indexing and Slicing

Negative Indexes

Python also allows counting from the end of a list.

- `-1` refers to the last item.
- `-2` refers to the second-to-last item.
- `-3` refers to the third-to-last item.

negative_indexing.py

```python
spam = ['cat', 'bat', 'rat', 'elephant']
print(spam[-1])
print(spam[-3])
```

Terminal Output

```
elephant
bat
```

Slicing: Capturing a Range

A **slice** is a range of items, and it returns a **new list**. The notation is `[start:end]`.

Important

A slice includes the `start` index but excludes the `end` index. This is another place where off-by-one mistakes are common.

Index	0	1	2	3
Value	cat	bat	rat	elephant

Table 3.2: Difference between an index and a slice range.

slicing_example.py

```
spam = ['cat', 'bat', 'rat', 'elephant']
print(spam[1:3])
print(spam[:2])
print(spam[1:])
```

Terminal Output

```
['bat', 'rat']
['cat', 'bat']
['bat', 'rat', 'elephant']
```

3.6.4 List Manipulation

Changing and Removing Items

Lists are `mutable`, meaning you can change their contents after they are created. You can overwrite a value using its index or remove it entirely using the `del` statement.

mutating_lists.py

```
items = ['Milk', 'Bread', 'Eggs']

# Update: change 'Bread' to 'Brown Bread'
items[1] = 'Brown Bread'

# Delete: remove 'Milk'
del items[0]
```

List Arithmetic

You can combine lists using the `+` operator (concatenation) or repeat them using the `*` operator (replication).

list_arithmetic.py

```
left = [1, 2, 3]
right = ['A', 'B']
print(left + right)

letters = ['X', 'Y']
print(letters * 3)
```

Terminal Output

```
[1, 2, 3, 'A', 'B']
['X', 'Y', 'X', 'Y', 'X', 'Y']
```

Concept Check

Consider the list `fruits = ['apple', 'mango', 'cherry', 'date']`.

1. What is the value of `fruits[1:3]`?
2. How do you change `'date'` to `'elderberry'` using a negative index?
3. If you run `del fruits[0]`, what is the new index of `'mango'`?

There Is No Dumb Question

David: Can I put a list inside another list?
Feynman: Yes. That is a **nested list**. Think of a shelf of boxes: the outer list is the shelf, and each box can contain its own smaller list. If the nested structure is rectangular, it can behave like a **matrix**.
David: So `matrix[0][1]` means what exactly?
Feynman: First choose the row with the first index, then choose the item inside that row with the second index.

nested_lists.py

```
matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

print(matrix[0][1])
print(matrix[2][2])
```

Terminal Output

```
2
9
```

Problem 4: The Category Organizer

Mr. Henderson wants you to organize the inventory into two specific groups:

1. **Fruits:** Banana, Mango, Avocado.
2. **Drinks:** Pepsi, Coke, Sprite.

Create a single variable called `inventory` that holds both categories and print "Mango" and "Sprite" from it.

Accessing the Grid

To access a nested item, use two indexes: `list[row][column]`.

categories.py

```python
# Index 0 is Fruits, Index 1 is Drinks
inventory = [
    ['Banana', 'Mango', 'Avocado'], # Row 0
    ['Pepsi', 'Coke', 'Sprite']     # Row 1
]

# Accessing Mango (Row 0, Column 1)
print(inventory[0][1])

# Accessing Sprite (Row 1, Column 2)
print(inventory[1][2])
```

Terminal Output

```
Mango Sprite
```

Visualizing the 2D List

Think of the 2D list as a table. The first index chooses the **row**, and the second index chooses the **column**.

Index	0	1	2
0 (Fruits)	Banana	Mango	Avocado
1 (Drinks)	Pepsi	Coke	Sprite

Table 3.3: A 2-Row by 3-Column Inventory Grid.

Important

Read `matrix[0][1]` from left to right: first choose the row, then choose the item inside that row.

Problem 5: The 3D Warehouse

You are managing two branches of the store. Each branch has its own list of categories.

- **Branch A** has a category of [Apple, Pear].
- **Branch B** has a category of [Water, Juice].

Organize this into one 3D list and print "Juice."

The 3-Index Depth

warehouse.py

```python
# Structure: [Branch][Category][Item]
warehouse = [
    [ ['Apple', 'Pear'] ],    # Branch 0
    [ ['Water', 'Juice'] ]    # Branch 1
]

# Step 1: Go to Branch 1
# Step 2: Go to Category 0
# Step 3: Go to Item 1
print(warehouse[1][0][1])
```

Terminal Output

```
Juice
```

Important

The Dimension Rule: Every time you "nest" a list inside another, you add a dimension.

- `list[x]` = 1D (A Line)
- `list[x][y]` = 2D (A Table)
- `list[x][y][z]` = 3D (A Cube/Warehouse)

There Is No Dumb Question

David: Does this mean I can have 4D or 5D lists?
Feynman: Mathematically, yes! You can nest them as deep as you want. But be careful—after 3 dimensions, it becomes very hard for a human to visualize where the data is. If you find yourself needing 5 indexes, there is usually a better way to organize your data!

3.6.5 How Lists Work Internally: Reference Behavior

Problem

Sometimes, changing one list unexpectedly changes another. This confuses many beginners because it looks like two separate variables are somehow "linked."

Important

This is not a bug. It happens because lists do not store values directly in variables. Instead, variables store **references** to the list.

Solution

A list variable does not contain the actual data. It contains a **reference** (like a pointer) to a location in memory where the list is stored.
When you assign one list to another variable, both variables point to the **same list**, not a copy.

Important

Both `a` and `b` point to the same list in memory. There is only **one list**, not two.

Code Example

reference_behavior.py

```python
a = [10, 20, 30]
b = a

b[0] = 99

print(a)
print(b)
```

Terminal Output

```
[99, 20, 30]
[99, 20, 30]
```

Important

Changing `b` also changes `a` because both variables refer to the same list.

Creating an Actual Copy

To avoid this shared behavior, you must explicitly create a copy.

copy_example.py

```
a = [10, 20, 30]
b = a[:]

b[0] = 99

print(a)
print(b)
```

Terminal Output

```
[10, 20, 30]
[99, 20, 30]
```

Important

Solution Using `a[:]` creates a new list (a shallow copy). Now `a` and `b` are independent.

There Is No Dumb Question

David: So when I do `b = a`, I'm not copying the list?
Feynman: Exactly. You are copying the **reference**, not the data. Both variables point to the same object.
David: Then why does `a[:]` work differently?
Feynman: Because slicing creates a **new list**. It copies the elements into a new location in memory.
David: So one is sharing, and the other is cloning?
Feynman: Perfect. Assignment shares the same list. Slicing creates a new one.

3.6.6 Reference Behavior and Reassignment

Important

A list variable does not store the list itself. It stores a `reference` to the list object in memory. That is why two variables can point to the same list, and why a change through one name may appear through the other.

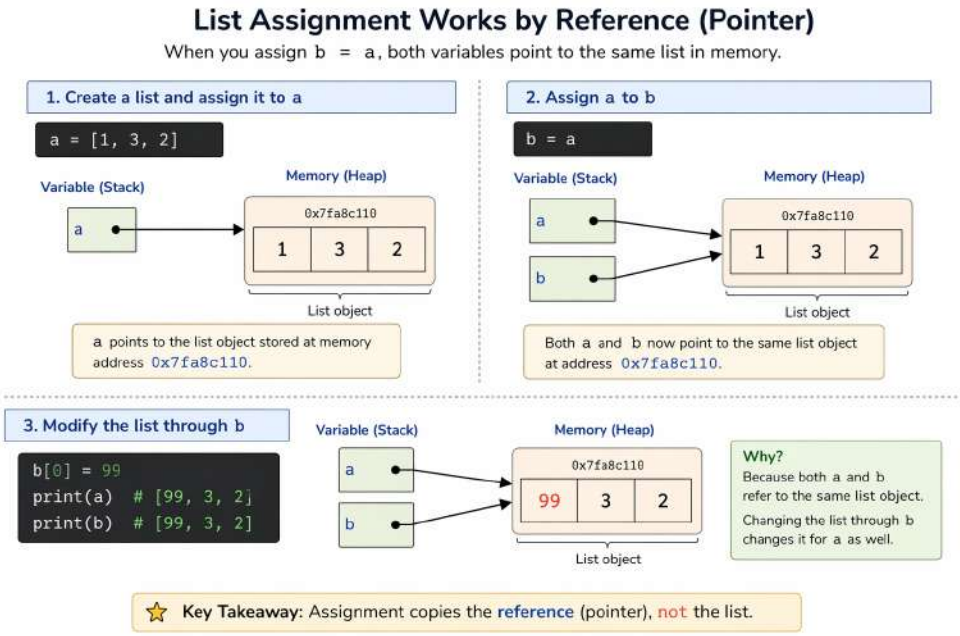


Figure 3.1: Assignment copies the reference, so `a` and `b` point to the same list object.

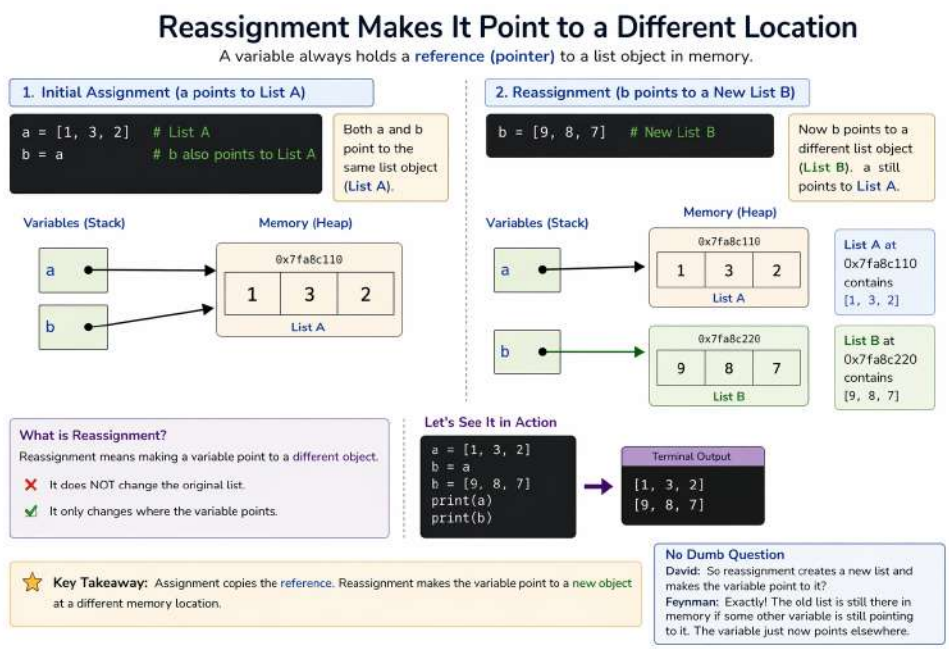


Figure 3.2: Reassignment changes where `b` points; it does not move `a`.

reference$_vs_reassignment.py$

```python
# Step 1: a points to a list object
a = [1, 3, 2]

# Step 2: b gets the same reference as a
b = a

# Step 3: changing through b changes the same list
b[0] = 99

print(a)    # a sees the change because it points to the same object
print(b)    # b sees the same change

# Step 4: reassignment makes b point somewhere else
b = [9, 8, 7]

print(a)    # a still points to the original list
print(b)    # b now points to a different list
```

Terminal Output

```
[99, 3, 2]
[99, 3, 2]
[99, 3, 2]
[9, 8, 7]
```

Important

Modification changes the existing list object. **Reassignment** changes only the variable's target, so the variable points to a different object.

There Is No Dumb Question

David: So when I write `b = a`, I am not making a copy?
Feynman: Correct. You are copying the `reference`, not the list itself. Now `a` and `b` name the same object.
David: Then what happens when I write `b = [9, 8, 7]`?
Feynman: That is `reassignment`. `b` stops pointing to the old list and starts pointing to a new one. `a` stays where it was.
David: So the old list is still there if something else points to it?
Feynman: Exactly. The object remains in memory as long as at least one reference still points to it.

3.7 The Dictionary Data Type

Problem: Lists Are Good, but They Are Not Always the Right Tool

A list is excellent when the order of items matters. But what if you do not want to ask, `"What is at position 0?"` What if you want to ask, `"What is the cat's color?"` or `"What is Alice's birthday?"` A list is a poor fit for that job because it is built around numeric positions, not labels.

P

ython solves this with a **dictionary**: a mutable collection of key-value pairs. In a dictionary, you do not look up values by index; you look them up by **key**.

3.7.1 What a Dictionary Is

A dictionary is a collection of associated values. Each entry is a **key-value pair**. The **key** is the label, and the **value** is the data attached to that label.

- A list uses `0`, `1`, `2`, and so on.
- A dictionary uses keys such as `'name'`, `'color'`, or `42`.

Important

A dictionary is `mutable`, just like a list. But unlike a list, a dictionary is not about positions. It is about connections: each key is connected to one value.

Important

Dictionary keys must be immutable and hashable. In practice, this usually means strings, numbers, or tuples. Lists and other dictionaries cannot be used as keys.

3.7.2 Creating a Dictionary

In code, dictionaries are written with braces, `{}`.

dictionary$_b$*asic.py*

```
myCat = {'size': 'fat', 'color': 'gray', 'disposition': 'loud'}

print(myCat['size'])
print(myCat['color'])
```

Terminal Output

```
fat
gray
```

Important

The key `'size'` is not an index. It is a label. When you write `myCat['size']`, Python returns the value associated with that label.

There Is No Dumb Question

David: Why do we need dictionaries if lists already store many values?
Feynman: Because lists answer a different question. A list says, `"What is at position 2?"` A dictionary says, `"What value belongs to this key?"` That is much better when the data has names, labels, or categories.

3.7.3 Accessing Values

To get a value from a dictionary, write the dictionary name, then the key inside square brackets.

dictionary$_{a}$*ccess.py*

```python
myCat = {'size': 'fat', 'color': 'gray', 'disposition': 'loud'}

print('My cat has ' + myCat['color'] + ' fur.')
```

Terminal Output

```
My cat has gray fur.
```

Important

If you use a key that is not present, Python raises a `KeyError`. This is the dictionary version of a list's `IndexError`.

Terminal Output

```
KeyError: 'color'
```

3.7.4 Dictionary Keys Can Be Numbers Too

Dictionary keys do not have to be strings. They can also be integers, and they do not need to start at `0`.

numeric$_{k}$*eys.py*

```python
spam = {12345: 'Luggage Combination', 42: 'The Answer'}
print(spam[12345])
print(spam[42])
```

Terminal Output

```
Luggage Combination
The Answer
```

Important

A dictionary may use integer keys, but those integers are still keys, not positions. So `spam[42]` does not mean "the 43rd item." It means "the value stored under the key 42."

3.7.5 Dictionaries Versus Lists

Lists are ordered sequences. Dictionaries are mappings. That difference matters.

list$_v s_d ict_c ompare.py$

```
spam = ['cats', 'dogs', 'moose']
bacon = ['dogs', 'moose', 'cats']

print(spam == bacon)

eggs = {'name': 'Zophie', 'species': 'cat', 'age': '8'}
ham = {'species': 'cat', 'age': '8', 'name': 'Zophie'}

print(eggs == ham)
```

Terminal Output

```
False
True
```

Important

For lists, order matters. For dictionaries, the connection between each key and its value matters more than the order in which the pairs were written.

There Is No Dumb Question

David: So a list is about sequence, but a dictionary is about matching?
Feynman: Exactly. A list is like numbered seats in a row. A dictionary is like a phonebook entry: you ask for the name, and you get the number.

3.7.6 Dictionaries Are Not Sliced

Because dictionaries are not position-based, you cannot slice them like lists. There is no meaningful idea of `spam[1:3]` for a dictionary.

Important

Do not think of a dictionary as a row of positions. Think of it as a table of labels and values.

3.7.7 Adding New Key-Value Pairs

Dictionaries are dynamic. You can add new pairs at any time.

add$_{k}ey_{v}alue.py$

```python
alien_0 = {'color': 'green', 'points': 5}

print(alien_0)

alien_0['x_position'] = 0
alien_0['y_position'] = 25

print(alien_0)
```

Terminal Output

```
{'color': 'green', 'points': 5}
{'color': 'green', 'points': 5, 'x_position': 0, 'y_position': 25}
```

Important

The order of the printed key-value pairs may not match the order in which you added them. The important thing is the connection between each key and its value.

3.7.8 Starting with an Empty Dictionary

Sometimes you begin with nothing and build the dictionary step by step.

empty$_{d}ictionary.py$

```python
alien_0 = {}

alien_0['color'] = 'green'
alien_0['points'] = 5

print(alien_0)
```

Terminal Output

```
{'color': 'green', 'points': 5}
```

Important

This pattern is useful when the values are not known in advance, or when your program must collect data gradually.

3.7.9 Modifying and Removing Values

You can replace the value attached to a key.

modify$_{d}$ictionary.py

```
alien_0 = {'color': 'green'}

print('The alien is ' + alien_0['color'] + '.')

alien_0['color'] = 'yellow'

print('The alien is now ' + alien_0['color'] + '.')
```

Terminal Output

```
The alien is green.
The alien is now yellow.
```

You can also remove a key-value pair completely with `del`.

delete$_{d}$ictionary$_{p}$air.py

```
alien_0 = {'color': 'green', 'points': 5}

print(alien_0)

del alien_0['points']

print(alien_0)
```

Terminal Output

```
{'color': 'green', 'points': 5}
{'color': 'green'}
```

Important

Deleting a key removes that pair permanently from the dictionary.

3.7.10 A Dictionary for Real Data

Dictionaries are powerful because they model real-world matching. A person's name can match a birthday, a country can match a capital city, a word can match a meaning, and an alien can match its color and point value.

birthday$_{d}$ictionary.py

```
birthdays = {'Alice': 'Apr 1', 'Bob': 'Dec 12', 'Carol': 'Mar 4'}

print(birthdays['Alice'])
print(birthdays['Bob'])
```

Terminal Output

```
Apr 1
Dec 12
```

Important

This is the real strength of dictionaries: they let you store information by meaning, not just by position.

3.7.11 Ordered in Memory, Not Indexed Like a List

In modern Python, dictionaries remember insertion order when you convert them into a sequence, but they are still not lists, and you still cannot access them with numeric positions.

dict$_{o}$*rder.py*

```python
eggs = {'name': 'Zophie', 'species': 'cat', 'age': '8'}
ham = {'species': 'cat', 'age': '8', 'name': 'Zophie'}

print(list(eggs))
print(list(ham))
```

Terminal Output

```
['name', 'species', 'age']
['species', 'age', 'name']
```

Important

Even though the printed order may reflect insertion order, you should not treat a dictionary like an indexed sequence. `eggs[0]` is not valid dictionary access.

3.7.12 Useful Dictionary Views

Python provides methods for looking at the keys, values, and pairs in a dictionary.

dictionary$_{v}$*iews.py*

```python
myCat = {'size': 'fat', 'color': 'gray', 'disposition': 'loud'}

print(list(myCat.keys()))
print(list(myCat.values()))
print(list(myCat.items()))
```

Terminal Output

```
['fat', 'gray', 'loud'] [('size', 'fat'), ('color', 'gray'), ('disposition',
'loud')]
```

Important

These views help you inspect a dictionary's contents. The `keys()` view shows the labels, the `values()` view shows the data, and the `items()` view shows the pairs together.

3.7.13 Nested Dictionaries

Dictionaries can also hold other dictionaries, which is useful when one object has many layers of information.

nested_dictionary.py

```
student = {
    'name': 'David',
    'details': {
        'age': 15,
        'city': 'Bahir Dar',
        'grade': 10
    }
}

print(student['name'])
print(student['details']['city'])
```

Terminal Output

```
David
Bahir Dar
```

Important

A nested dictionary is a dictionary inside another dictionary. Use one key to reach the inner dictionary, then another key to reach the value inside it.

There Is No Dumb Question

David: So a dictionary can hold a list, another dictionary, or even both?
Feynman: Yes. That is why dictionaries are so useful. They can organize data by meaning, and they can also contain more structure inside them.

3.7.14 Why Dictionaries Matter

Lists are best when order matters. Dictionaries are best when meaning matters. If you need the third item in a row, use a list. If you need the value attached to a label, use a dictionary.

Important

A strong programmer does not choose lists or dictionaries by habit. The choice depends on the shape of the data.

Problem 6: The Labeled Inventory

Mr. Henderson wants to stop using index numbers like `inventory[0][1]`. It's confusing! He wants to access items by their category names: **"Fruits"** and **"Drinks"**.

1. **Fruits:** Banana, Mango, Avocado.
2. **Drinks:** Pepsi, Coke, Sprite.

Organize this into a nested structure where you can type `inventory['Fruits']` to see the fruit list.

The Labeled Grid

By nesting lists *inside* a dictionary, we give our data human-readable names.

labeled_inventory.py

```python
# A Dictionary containing Lists
inventory = {
    'Fruits': ['Banana', 'Mango', 'Avocado'],
    'Drinks': ['Pepsi', 'Coke', 'Sprite']
}

# Accessing Mango:
# Step 1: Get the 'Fruits' list
# Step 2: Get index 1 from that list
print(inventory['Fruits'][1])
```

Terminal Output

```
Mango
```

Visualizing the Dictionary Table

In a dictionary grid, the "Row Index" is replaced by a "Key".

Key (Category)	Index 0	Index 1	Index 2
'Fruits'	Banana	Mango	Avocado
'Drinks'	Pepsi	Coke	Sprite

Table 3.1: A Dictionary-to-List Mapping.

Problem 7: The Master Warehouse

We have two branches. Each branch has its own categorized inventory.

- **Branch_A:** Fruits → [Apple, Pear].
- **Branch_B:** Drinks → [Water, Juice].

Organize this into a 3D structure using only dictionaries and lists.

Deep Key Access

master$_w$arehouse.py

```python
warehouse = {
    'Branch_A': {
        'Fruits': ['Apple', 'Pear']
    },
    'Branch_B': {
        'Drinks': ['Water', 'Juice']
    }
}

# Goal: Print 'Juice'
# Step 1: Branch_B
# Step 2: Drinks
# Step 3: Index 1
print(warehouse['Branch_B']['Drinks'][1])
```

Terminal Output

```
Juice
```

Important

Why use Dictionaries over Lists?
In a 3D list, you have to remember `[1][0][1]` means "Branch B, Drinks, Juice." In a Dictionary, your code **documents itself**: `['Branch_B']['Drinks'][1]`. It is much harder to make a mistake!

There Is No Dumb Question

David: Wait, can the "Key" be anything? Like a number or a float?
Feynman: Almost anything! Keys must be **immutable** (unchangeable). Strings and integers are the most common. However, unlike a list, dictionaries have **no order**. In a list, "Apple" is always first. In a dictionary, "Apple" is just wherever its key is.

3.8 Loops: The Engine of Automation

Assume:The Phone Call

The phone rings. David picks up, looking tired but excited.
Henderson: David! Where are you? Aren't you coming to work? There is a mountain of inventory left waiting for you!
David: Boss! Please, hear me out. Could I have today off? I have just one topic left to read. Once I finish this, I can completely build that inventory system you asked for.
Henderson: (Grumbling) One topic? You're staying home for one topic?
David: It's the most important one! If I get this, I can make the computer do the work for us.
Henderson: Alright, fine. One day. But you better hurry up!
David: Thank you, boss! You won't regret it!

There Is No Dumb Question

Feynman: So, David. Up to this point, we have discussed the basic building blocks of Python and how to structure data. But we haven't seen how to actually **automate** things.
Feynman: Automation is really what Computer Science is about at the end of the day. And in order to automate things... I don't think you have the necessary tools yet, do we?
David: I'm starting to realize that. I can store the data in lists, but if I want to process it, I'm still the one doing the typing.
Feynman: Exactly. Why don't we start with the simplest idea? Print `"I am David"` 10 times.
David: Easy! I'll just write the print statement 10 times.
Feynman: (Smiling) And if Mr. Henderson asks you to do it 10,000 times for every item in the warehouse?
David: ...Yeah. You're right teacher. We do need to discuss having something that allows me to *actually* do something without manual labor.

3.8.1 The `while` Loop: The "As Long As" Machine

while loops are the most fundamental way to repeat actions in Python. If you think of a program as a path, a loop is a **roundabout**. Instead of walking straight past a piece of code once, Python enters the circle and keeps spinning until a specific "gate" opens to let it out.

Even without knowing the formal syntax of Python, you can likely guess the core idea of a loop just by looking at its logic. It is a structure that allows you to say: **"While this condition is True, keep doing this thing."**

In its rawest form, the logic looks like this:

conceptual$_l$oop.py

```
while condition_is_True:
    # Keep performing this indented task
```

In order to construct and use `while` loops properly so they don't break your computer or your brain, we need two essential components:

1. **A Condition:** This is the "Gatekeeper." It is a test (usually a comparison like `<` or `==`) that decides if the loop should keep running.
2. **An Exit Strategy:** This is the "Escape." It is something *inside* the loop that eventually makes the condition **False**. Without this, the gate never opens.

The Counting Machine

Mr. Henderson wants a simple program that counts for him. Specifically, he needs you to print the numbers from **0 up to 9**.

Building the Loop Step-by-Step

To solve this, we don't just start typing. We plan the "Machine" first.
Step 1: Figure out the Condition
We need to print numbers as long as the numbers are less than 10. If we assume our counter is a variable named `i`, our condition is mathematically: `i < 10`.
The Syntax:
In Python, we write the word `while`, then our condition, followed by a **colon** (`:`). Every line of code that is **indented** under this colon belongs to the loop. These lines will execute repeatedly as long as that condition stays True.
Step 2: Initialize
Before the loop can check the condition, the variable `i` must exist. We must give it a starting value. Since we want to count from zero, we set `i = 0`.

the$_u$nbroken$_l$oop.py

```
i = 0              # Start here
while i < 10:      # Check: is 0 < 10? Yes.
    print(i)       # Do the task
```

There Is No Dumb Question

Feynman: Look closely at that code above, David. If we ran that through your computer right now, what would go wrong?

David: (Stares at the screen) Well... `i` starts at 0. Python checks if `0 < 10`. That's True, so it prints 0. Then it goes back to the top and checks again. Since `i` is still 0, it prints 0 again... it never stops!And i think this is the flowchart,right?

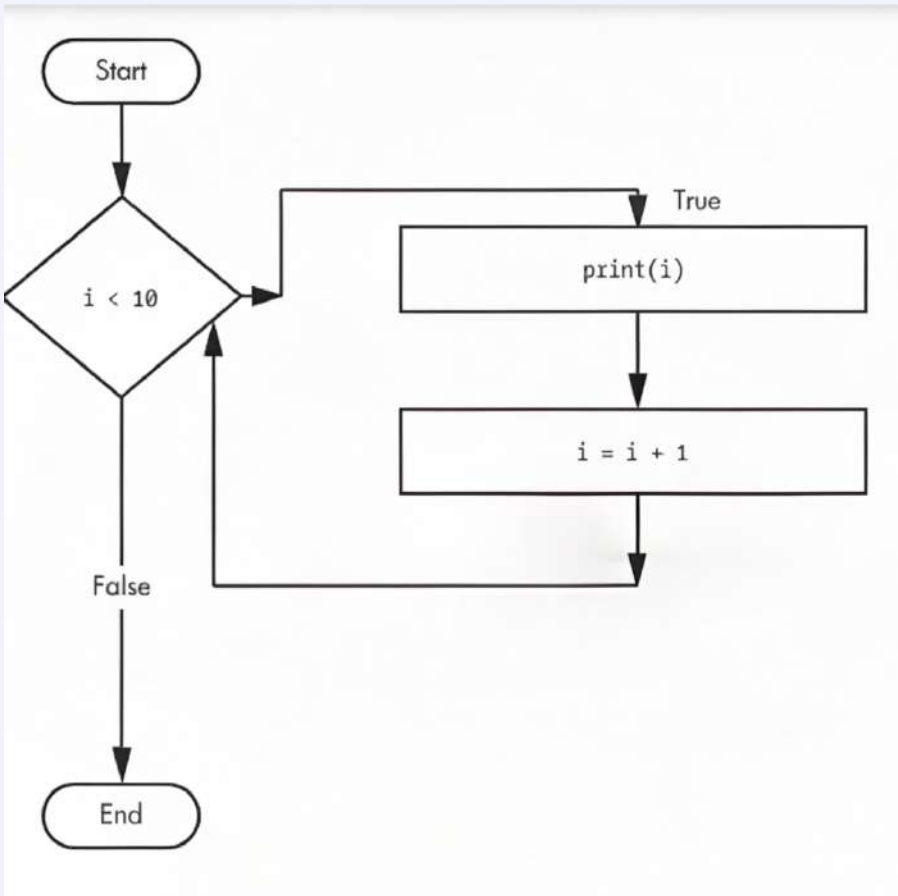


Figure 3.1: The Logic of an Infinite Loop

Feynman: Exactly. You've created the "Infinite Loop." The condition is **always** True. We are missing our second essential component: **The Exit Strategy.**
David: So I need to move `i` toward 10 so that eventually `10 < 10` becomes False?
Feynman: Precisely. We need to "update" the variable: `i = i + 1`.
David: Right! But wait—what prevents the computer from jumping back to the very first line (`i = 0`) and resetting the count to zero every time?
Feynman: That is the beauty of Python's flow! Python reads from top to bottom, but once it enters a `while` block, it "traps" the execution inside that indented section. It will **not** leave that loop or go back to the code above it until the condition finally breaks.

The Complete Automation

Now we combine all the pieces. Notice how the indentation tells Python exactly what is "inside" the loop machine and what is "outside."

final_loop.py

```python
i = 0              # Step 1: Initialize (Outside the loop)
while i < 10:      # Step 2: The Condition (The Gate)
    print(i)       # Step 3: The Task (Inside the loop)
    i = i + 1      # Step 4: The Update (The Exit Strategy)

print("Done!")     # This runs only when the loop is False
```

Terminal Output

```
0
1
2
3
4
5
6
7
8
9
Done!
```

Important

The line `i = i + 1` is an **assignment**, not a mathematical equality. It tells Python to "take the current value of `i`, add 1 to it, and store the result back in `i`." This is called an **increment**.
For cleaner code, you can use the shorthand: `i += 1`.

The Validation Loop: Handling User Input

In a real inventory system, users make mistakes. They might type a negative number for age or leave a field blank. If we only use `if` statements, we can only catch a mistake a limited number of times. To build a robust system, we need a loop that "traps" the user until they provide valid data.

Problem : The Age Validator

Mr. Henderson wants the system to ask for a user's name and age. However, if the user enters a negative age, the program must keep asking until a valid (positive) age is entered.

The "Wrong" Way vs. The Loop Way

The Dead End: Nested Ifs

Your first instinct might be to nest conditionals to catch a second mistake:

nested$_{f}ailure.py$

```
name = input("Enter name: ")
age = int(input("Enter age: "))

if age < 0:
    print("Invalid age!")
    age = int(input("Try again: "))
    if age < 0:
        print("Still invalid!")
        age = int(input("Try one more time: "))
        # We can't do this forever!
```

There Is No Dumb Question

Feynman: David, look at that code. What happens if the user is stubborn and enters a negative number five times in a row?
David: I'd have to keep nesting `if` statements deeper and deeper. The code would start looking like a giant sideways triangle! It is a complete mess.
Feynman: Exactly. You are trying to solve a repeating problem with a static tool. We need a way to "stay" at the question until the answer is right.

The Solution: The Validation Loop

Instead of checking once, we check "while" the data is bad.

age$_{v}alidator.py$

```
name = input("Enter your name: ")
age = int(input("Enter your age: ")) # Step1: asking the user
    for input
while age < 0: # Step 3: The Condition
    age = int(input("Enter your age: ")) # Step 2: Ask again

# Step 3: The Exit (Condition becomes False)
print(f"Access Granted. Hello {name}, you are {age} years old.
    ")
```

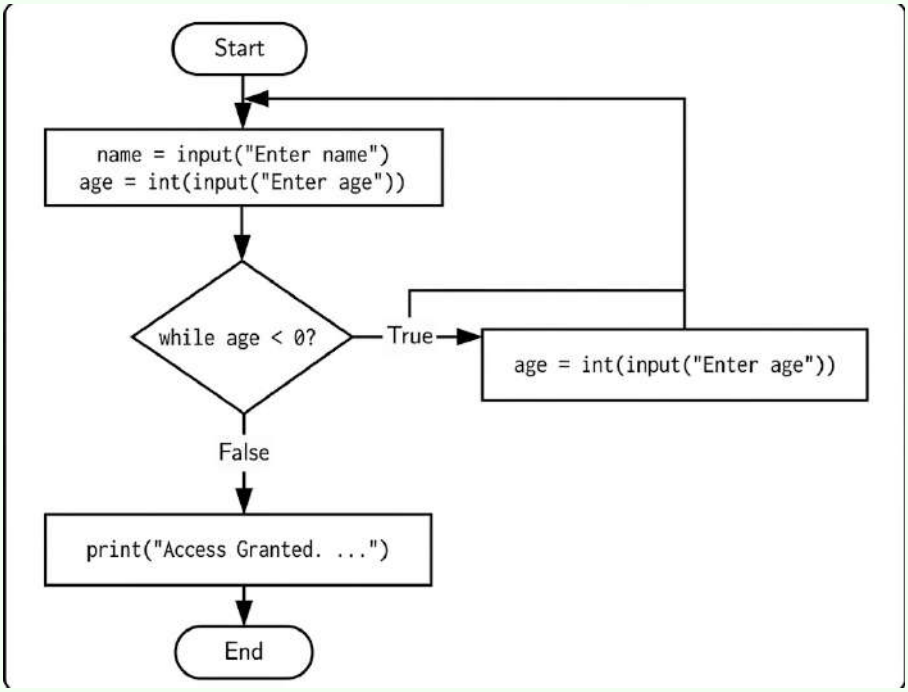


Figure 3.2: The flowchart for age_validation.py

Important

The Interaction Toolkit:

- `input()`: Pauses execution to get text from the user. It always returns a **string**.
- `int()`: Converts a string to an integer so we can perform math or comparisons.
- **f-strings**: Placing an `f` before quotes (e.g., `f"{var}"`) allows you to inject variables directly into text using curly braces.
- **Assignment vs Equality**: Remember `age < 0` is a **comparison** (asking a question).

There Is No Dumb Question

Feynman: Python reads from top to bottom, but once it enters a `while` block, it "traps" the execution inside that indented section. It will **not** leave that loop or go back to the code above it until the condition finally becomes false.
David: So the loop is like a filter. It keeps the "bad" data spinning inside until it turns "good," and then it lets it through to the rest of the program.
Feynman: Perfect analogy.

Visualizing the Loop: Unrolling the Machine

Although we write a loop in just three or four lines of code, the computer doesn't see a "circle." In the computer's memory, the loop "unrolls" into a long, straight sequence of actions. Understanding this is the key to seeing how automation actually works—it is just a shorthand for a long list of instructions.

Problem: The Unrolling Visualization

To understand what happens inside the CPU, let's take a 3-step loop and "unroll" it. We want to see every single individual instruction the computer executes from start to finish.

What the Computer Actually Does

When you write this compact code:

loop$_c$ompact.py

```
i = 1
while i <= 3:
    print(i)
    i += 1
```

The computer "unrolls" it into this linear, step-by-step sequence of events in its memory:

unrolled$_e$xecution.txt

```
1. [Assign]     i = 1
2. [Check]      Is 1 <= 3? (True)
3. [Execute]    print(1)
4. [Update]     i = 2 (1 + 1)
5. [Check]      Is 2 <= 3? (True)
6. [Execute]    print(2)
7. [Update]     i = 3 (2 + 1)
8. [Check]      Is 3 <= 3? (True)
9. [Execute]    print(3)
10. [Update]    i = 4 (3 + 1)
11. [Check]     Is 4 <= 3? (FALSE) -> STOP
```

There Is No Dumb Question

Feynman: David, look at steps 10 and 11 in the unrolled list. What do you notice about the value of `i`?
David: Wait... the loop stopped printing at 3, but the variable `i` actually ended up as 4?

Feynman: Exactly! The computer doesn't know it's "done" until it tries to start the next lap and fails the condition check. `i` must become 4 for the gate to slam shut.
David: So the "Exit Strategy" (`i += 1`) runs one last time even after the final `print` happened.
Feynman: Precisely. If you didn't have that last update, the computer would just stay at Step 2 forever, asking "Is 1 <= 3?" over and over until the end of time.

Important

The "Unrolled" Reality:

- **Linearity:** Loops are just a programmer's shorthand. The hardware still processes everything one line at a time.
- **The Terminal Value:** In a `while i < 10` loop, the variable `i` will actually be **10** when the code finishes. It is the value that finally made the condition **False**.
- **Assignment Recap:** `i += 1` is not a math equation; it is an **assignment**. It is the physical "click" of the gear that moves the machine to the next step of the unrolled sequence.

Loop Control: `break` and `continue`

Sometimes, the standard "While this is True" logic is too rigid. You might need to kill a loop early because you found what you were looking for, or you might need to skip a specific item because it is faulty. In Python, we use **Control Statements** to steer the loop while it is in motion.

Keyword	Mechanical Action	Real-World Analogy
`break`	Kills the loop immediately and moves to the next section of code.	Finding a "Hazardous" item and shutting down the entire conveyor belt.
`continue`	Aborts the current lap and jumps back to the condition check.	Skipping over an item and continuing the searching opreation.

Table 3.1: Comparison of Loop Control Keywords

There Is No Dumb Question

Feynman: David, imagine you are scanning 1,000 milk cartons looking for one that is expired. If you find it at carton number 5, do you keep checking the other 995?
David: No, I'd stop immediately. Why waste time?
Feynman: That is `break`. But what if you are scanning prices and you see a "Display Only" item with no price tag? You don't want to stop the whole scan; you just want to skip that one item and move to the next.
David: So `continue` is like saying "Never mind this specific lap, let's just go to the next one"?
Feynman: Exactly. You are skipping the *instructions*, not the *loop*.

—

Problem 11: The Emergency Stop (`break`)

Mr. Henderson has a list of items. Your loop is counting them. However, if the loop encounters an item called `"BOMB"`, it must stop immediately before the system crashes.

Implementing an Early Exit

emergency$_{e}xit.py$

```python
items = ["Apple", "Banana", "BOMB", "Mango"]
i = 0

while i < len(items):
    item = items[i]
    if item == "BOMB":
        print("DANGER! Stopping scan.")
        break  # The loop terminates immediately

    print(f"Scanning: {item}")
    i += 1

print("System Check Complete.")
```

Terminal Output

```
Scanning: Apple
Scanning: Banana
DANGER! Stopping scan.
System Check Complete.
```

Note: "Mango" was never scanned because the **`break`** *statement killed the loop at index 2.*

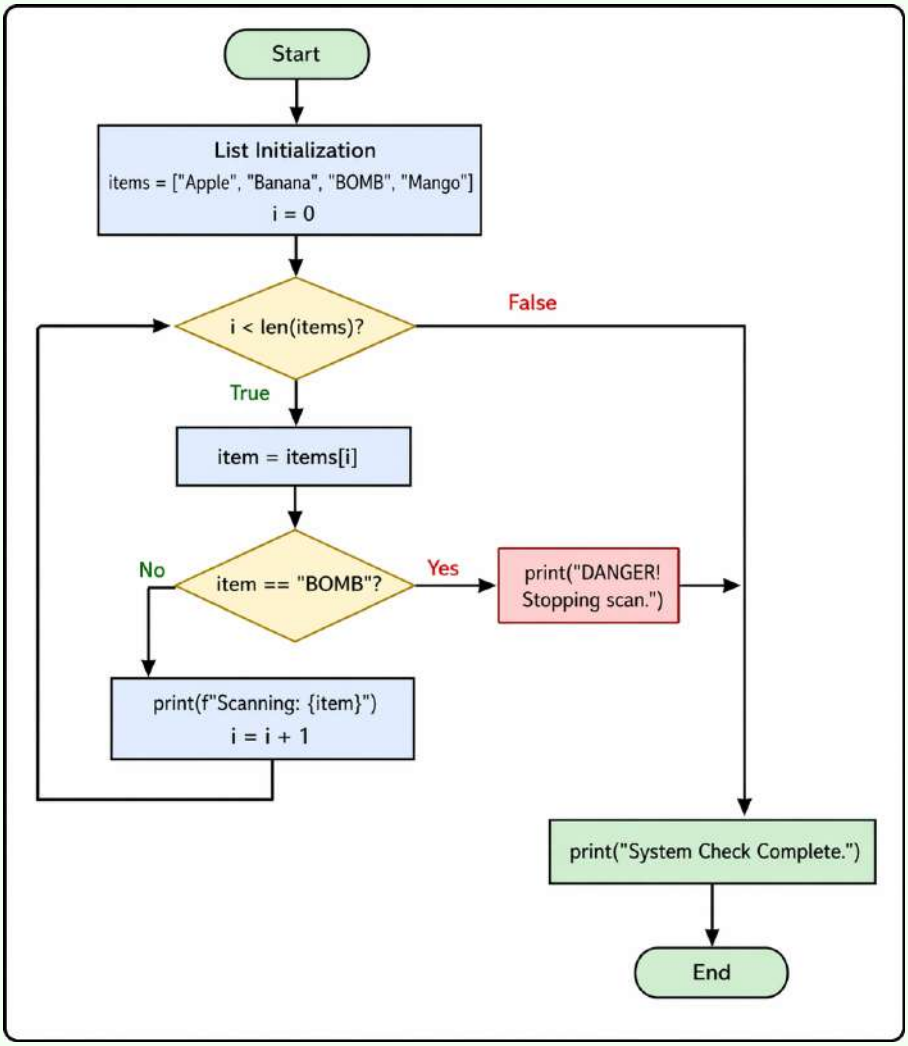


Figure 3.1: Flowchart for emergency_exit.py

—

Problem 12: The Tax Skipper (`continue`)

Calculate the processing for items. If an item is `"Bread"`, it is a basic necessity and should be skipped for tax calculation. Use `continue` to skip the work for Bread.

Skipping Specific Laps

tax$_s$kipper.py

```
1  items = ["Milk", "Bread", "Eggs"]
2  i = 0
3
4  while i < len(items):
5      current = items[i]
6      i += 1 # CRITICAL: Increment BEFORE continuing
7
8      if current == "Bread":
9          print("Skipping Tax for Bread...")
10         continue # Jumps back to the 'while' line immediately
11     print(f"Calculating Tax for {current}...") #because of
           continue for bread we will never get to this print line
           (line 11)
```

Terminal Output

```
Calculating Tax for Milk...
Skipping Tax for Bread...
Calculating Tax for Eggs...
```

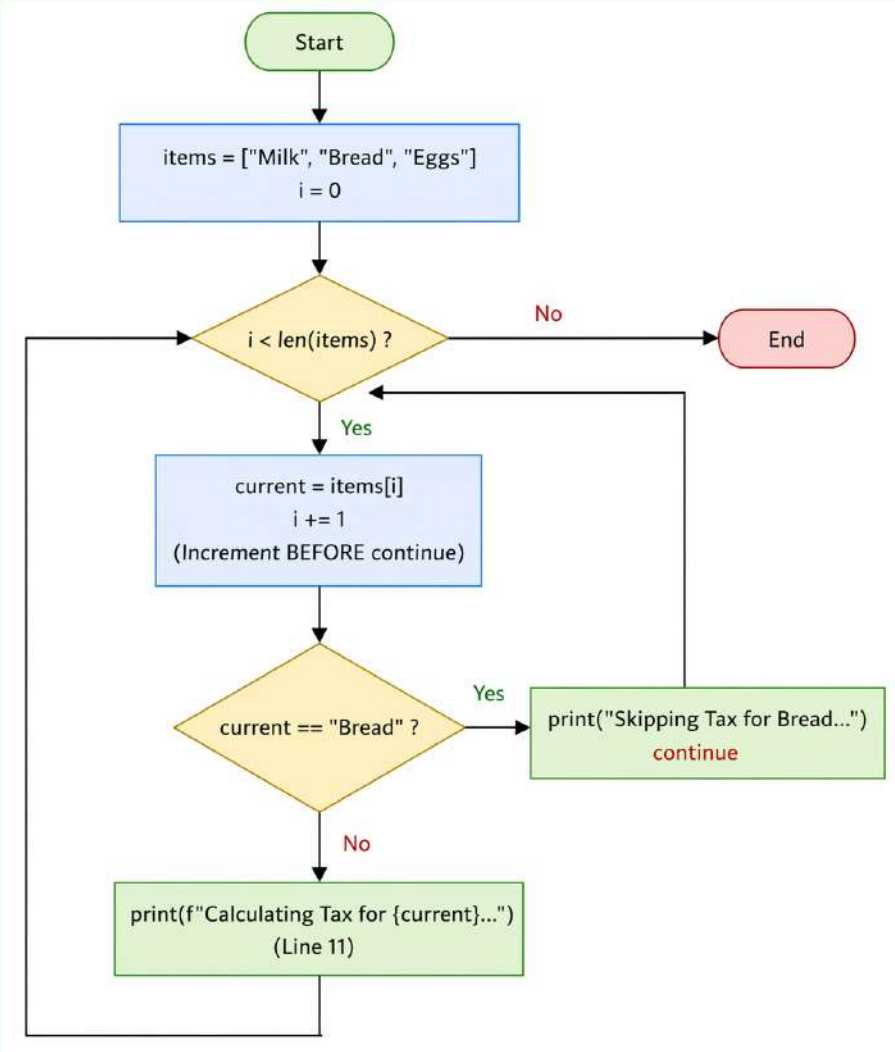


Figure 3.2: Flowchart for tax_skipper.py

—

Important

Common usage of while loop: While loops are the "Workhorses of Uncertainty." While the `for` loop(section 3.8.2) is used when you know exactly how many items you have, the `while` loop is used when you don't know how long the task will take. In our warehouse, there are three primary reasons to keep a `while` loop in your toolkit:

1. **Input Validation**: Forcing a user to provide correct data (e.g., "Enter a positive number") before the program continues.

2. **Event-Based Triggers**: Running until a specific event occurs, like a sensor hitting a limit or a "Stop" command being received.

3.8.2 The `for` Loop: The Elegant Collection Scanner

In Python, there are generally three ways to repeat an action: **While Loops**, **For Loops**, and **Recursion**. While much of the choice comes down to a programmer's personal preference, certain tasks are "best suited" for specific tools.

To loop through a list using a `while` loop, you must manage three distinct manual steps:

1. **Initialize**: Create a counter (e.g., `i = 0`).
2. **Condition**: Set a boundary (e.g., `while i < len(items)`).
3. **Increment**: Manually move to the next item (e.g., `i += 1`).

If you miss just one of these, the system fails. The `for` loop is designed to automate this entire process, making your code cleaner and harder to break.

There Is No Dumb Question

David: So a `while` loop is like driving a manual car where I have to shift every gear, and a `for` loop is like an automatic?
Feynman: Exactly! In a `for` loop, Python handles the "gears" (indexing and incrementing) behind the scenes. You just tell it what to drive over.

The IDE Syntax

In your editor, the structure of a `for` loop is remarkably simple:

for$_{l}oop_{b}asics.py$

```python
# 'items' is the Iterable (the data structure)
# 'item' is the target variable (each piece of data)

for item in items:
    # Do something with the item
    print(item)
```

There Is No Dumb Question

David: I see this new keyword `in`. What is its job?
Feynman: Think of a man reading a book. The book is the **Iterable** (the collection of pages). Each page is an **Item**. The man takes "one page **in** the book" at a time, reads it, and moves on. He doesn't need to check the page numbers to know when to stop; he just stops when he runs out of pages.

Important

The Mechanics of "In":

- **The Iterable**: The object after the `in` keyword must be "iterable"—meaning it is a data structure like a list, string, or dictionary that can be stepped through.
- **The Item**: This is a temporary variable that Python assigns to the current piece of data in the sequence.
- **Mapping**: The keyword `in` acts as the bridge. It automatically maps the variable `item` to the next available spot in the `items` object.

—

Problem 15: The Automated Manifest

Mr. Henderson has a shipment of `["Sensors", "Displays", "Cables"]`. Instead of using a manual counter, use a `for` loop to print each item along with a "Scan Complete" message.

The For Loop Approach

auto$_s$*can.py*

```python
shipment = ["Sensors", "Displays", "Cables"]

# For every 'part' found IN the 'shipment' list:
for part in shipment:
    print(f"Item detected: {part}")
    print("--- Scan Complete ---")
```

Terminal Output

```
Item detected: Sensors
--- Scan Complete ---
Item detected: Displays
--- Scan Complete ---
Item detected: Cables
--- Scan Complete ---
```

Important

Notice that we never asked for the `len()` of the list, and we never wrote `i += 1`. Python reached the end of the "book" and closed it automatically.

There Is No Dumb Question

David: Feynman, look at how much work it takes to print a simple list of inventory items with a `while` loop!

manual$_c$ounter.py

```
items = ["Spam", "Eggs", "Ham"]
i = 0
while i < len(items):
    x = items[i]
    print(x)
    i += 1
```

Feynman: It's very cluttered, David. You are managing the variable `i` and the length of the list yourself. In Python, we let the loop handle the work:

clean$_f$or$_l$oop.py

```
for x in ["Spam", "Eggs", "Ham"]:
    print(x)
```

David: No counter? No `i += 1`? How does it know when to stop?
Feynman: It knows because it looks at the list. It assigns `x` to the first item, runs your code, and automatically moves to the next. When there are no items left, it simply stops.

The `range()` Function: Counting on Demand

Sometimes we don't have a list of items; we just want to repeat an action a specific number of times. This is where `range()` comes in. It generates a sequence of numbers for the loop to follow.

Usage	Numbers Produced	The Rule
`range(5)`	0, 1, 2, 3, 4	Starts at 0, stops **before** 5.
`range(2, 6)`	2, 3, 4, 5	Starts at 2, stops **before** 6.
`range(0, 10, 2)`	0, 2, 4, 6, 8	Starts at 0, jumps by 2.

Table 3.2: How the Range Function Behaves

Problem 13: The Step-Counting Scanner

Mr. Henderson wants to sample his stock. He wants you to print every second item from a 10-item list. Use `for` and `range()` to do this efficiently.

Combining For and Range

step$_s$ampler.py

```python
items = ["A", "B", "C", "D", "E", "F", "G", "H", "I", "J"]

# Start at 0, go to the end (len), jump by 2
for i in range(0, len(items), 2):
    print(f"Sampling Index {i}: {items[i]}")
```

Terminal Output

```
Sampling Index 0: A
Sampling Index 2: C
Sampling Index 4: E
Sampling Index 6: G
Sampling Index 8: I
```

—

Important

Key Takeaways for Loops:

- **Target Variable**: The variable used in the header (like `x` or `i`) is just a name. Python re-assigns it at the start of every lap.
- **The "Up-To" Rule**: `range(10)` gives you 10 numbers, but the last one is **9**. It always stops right before the limit.
- **Tuples and Unpacking**: You can loop through pairs of data easily: `for (a, b) in [(1,2), (3,4)]:`.

3.8.3 Nested Loops

Problem 15: The Coordinate Grid

Given two lists:

- `x_coords = [1, 2]`
- `y_coords = [1, 2, 3]`

Produce the following terminal output by combining them:

Terminal Output

```
Point: (1, 1)
Point: (1, 2)
Point: (1, 3)
Point: (2, 1)
Point: (2, 2)
Point: (2, 3)
```

How do we generate this grid without writing six separate print statements?

First Principles Thinking

Let's solve this by analyzing the patterns from first principles.
Step 1: Analyzing the Terminal Output Look closely at the coordinate pairs (x, y):

- The first number (x) stays exactly the same (1) for the first three lines.
- While x is frozen at 1, the second number (y) counts up: $1 \to 2 \to 3$.
- Only after y finishes counting its complete set does x finally click forward to 2. Then the cycle repeats.

Step 2: Deducing the Structure Because one value is counting completely while another is held steady, a single loop cannot do this. We need an **Outer Loop** to control x and an **Inner Loop** to control y.
The Analogy: The Shelf Search Think of this like a man searching for a book on a set of shelves:

- The outer loop is the **Shelf** he is standing in front of.
- The inner loop is the **Book** his eyes are scanning on that shelf.
- He goes over the first shelf linearly: Book 1, Book 2, Book 3. Only when he runs out of books does he step to the second shelf and start over at Book 1.(See the image below)

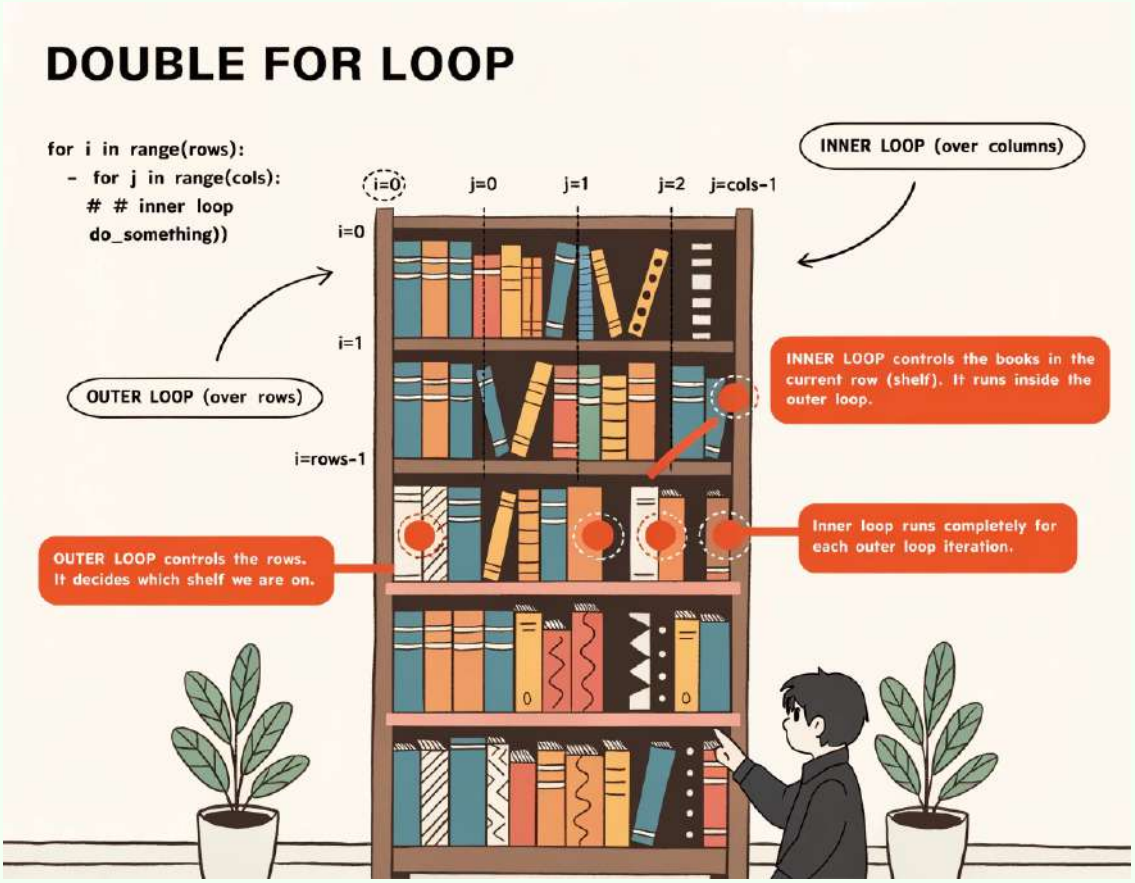


Figure 3.3: nested loop analogy

The Code Solution:

nested$_{c}$*oords.py*

```
x_coords = [1, 2]
y_coords = [1, 2, 3]

# The Outer loop picks an 'x' and holds it steady
for x in x_coords:
    # The Inner loop cycles through every 'y' for that 'x'
    for y in y_coords:
        print(f"Point: ({x}, {y})")
```

—

The Dynamic Intake System

Mr. Henderson has a truck arriving with an unknown number of items. He needs David to:

1. Use a **`while`** loop to let the user type in part names one by one.
2. Stop the intake when the user types **`"done"`**.
3. Use a **`for`** loop to print a final "Inventory Manifest" of everything collected.

Combining the Loops (First Principles)

The Reasoning: This is a two-phase operation.

- **Phase 1 (Input)**: We use a `while` loop because we don't know if the truck has 2 items or 200. We loop until a *state change* (typing "done").
- **Phase 2 (Output)**: Once the list is built, it is a "finished book." We use a `for` loop to scan it because it is now a known collection.

intake_system.py

```python
new_stock = []
entry = ""

# Phase 1: Building the list (Uncertainty)
while entry.lower() != "done":
    entry = input("Enter part name (or 'done' to finish): ")
    if entry.lower() != "done":
        new_stock.append(entry)

print("\n--- FINAL MANIFEST ---")

# Phase 2: Processing the list (Automation)
for item in new_stock:
    print(f"Stored in Bin: {item.upper()}")
```

Logic Trace

Action	Tool Used	First Principles Reasoning
Collecting Data	`while` Loop	We use this for **Uncertainty**. David doesn't know if the truck has 5 items or 50, so the loop runs until a "State Change" (typing 'done').
Processing Data	`for` Loop	We use this for **Automation**. Once the items are in the list, we have a "Fixed Collection." We scan the entire "book" from start to finish.

There Is No Dumb Question

David: Wait, why didn't I just print the item inside the `while` loop as I typed it?
Feynman: You could! But in a real system, you often need to save all the data first to check for errors or sort it before printing. By saving it to a list and then using a `for` loop, you separate the "Data Entry" from the "Data Processing." This is the first step toward building professional software.

—

Problem 17: The High-Risk Audit

A shipment of sensors has arrived, but many are reporting "Faulty" or "Critical" status. David is given a raw list of data:

```
logs = ["Faulty", "OK", "Faulty", "Critical", "OK", "Faulty", "Critical"]
```

The Mission:

1. **Build** a Frequency Map (Dictionary) to count every status found in the logs.
2. **Analyze** that map to find **High-Risk** statuses (any status appearing more than once).
3. **Generate** a formatted "Danger Report" for Mr. Henderson.

Step-by-Step Engineering

Phase 1: Building the Memory (The Scanner) We start by creating an empty dictionary. As the `for` loop scans each log, we check our "Memory." If the word is new, we start the count at 1. If we recognize it, we increment the tally.

Step	Item Scanned	Dictionary State	First Principles Action
1	"Faulty"	`{"Faulty": 1}`	New entry: Set to 1.
2	"OK"	`{"Faulty": 1, "OK": 1}`	New entry: Set to 1.
3	"Faulty"	`{"Faulty": 2, "OK": 1}`	**Recognized**: +1 tally.
4	"Critical"	`{"Faulty": 2, "OK": 1, "Crit": 1}`	New entry: Set to 1.

Table 3.3: Trace table of the "Memory" building process.

Phase 2: The Analysis (The Filter) Once the scan is complete, we no longer need the original list. We loop through the **Dictionary** keys and values. We use an `if` statement to filter the items and only report those that have a high count.
The Completed Code:

audit_pro.py

```python
# Step 1: The Raw Data
logs = ["Faulty", "OK", "Faulty", "Critical", "OK", "Faulty",
    "Critical"]
status_counts = {}

# Step 2: Build the Frequency Map (Phase 1)
for status in logs:
    if status in status_counts: # if it already exits
        status_counts[status] += 1 # then incremnet
    else: # if it new (we have never seen it before)
        status_counts[status] = 1 # initialize its count to 1

print("--- MR. HENDERSON'S DANGER REPORT ---")
#example --> status_counts = {"faulty":2,"ok":3, ... }

# Step 3: Analyze and Filter (Phase 2)
for status, count in status_counts.items():# status become the
    key like Faulty,Ok,... and count becomes the actual count
    like 1,2,..
    # Only report if it appears more than once (High Risk)
    if count > 1:
        print(f"ALERT: '{status}' detected {count} times.
            Inspect immediately.")
```

Terminal Output

```
--- MR. HENDERSON'S DANGER REPORT ---
ALERT: 'Faulty' detected 3 times. Inspect immediately.
ALERT: 'OK' detected 2 times. Inspect immediately.
ALERT: 'Critical' detected 2 times. Inspect immediately.
```

Important

The "Two-Pass" Secret: Notice how we separated the **Counting** from the **Reporting**. In professional software, we call this "Separation of Concerns." It ensures that even if Mr. Henderson's list grows to 1,000,000 items, David's code stays organized and efficient.

Problem 18: Serial Number Integrity

Every piece of hardware in the warehouse must have a unique ID. Mr. Henderson provides a list:

```
serials = [1001, 1002, 1003, 1001, 1005]
```

Write a function `has_duplicate(serials)` that returns `True` if a collision is found and `False` if the shipment is clean.

The "Seen Memory" Algorithm

Phase 1: The Strategy To solve this from First Principles, we don't just "loop"—we **Audit**.

- **The Memory**: A list called `seen` that starts empty.
- **The Condition**: If the current number is *already* in `seen`, we stop and alert Mr. Henderson.

Phase 2: The Logic Trace

Step	Current SN	State of 'Seen'	Decision
1	1001	`[]`	New! Add to memory.
2	1002	`[1001]`	New! Add to memory.
3	1003	`[1001, 1002]`	New! Add to memory.
4	1001	`[1001, 1002, 1003]`	**MATCH! Return True.**

Phase 3: The Optimized Code

integrity_check.py

```python
def has_duplicate(serials):
    # 'seen' acts as our digital notepad (Memory)
    seen = []

    for sn in serials:
        # STEP 1: Check if we have encountered this SN before
        if sn in seen:
            # Found a duplicate! We stop the loop immediately.
            # This is called an 'Early Return'.
            return True

        # STEP 2: If it's new, record it in our notepad
        seen.append(sn)

    # STEP 3: If the loop finishes, no duplicates were found
    return False

# Execution
incoming_truck = [1001, 1002, 1003, 1001, 1005]
if has_duplicate(incoming_truck): # it would be if True: (b/c the function returns True when there is a duplicate)
    print("WARNING: Data Collision Detected!")
```

Important

First Principles Insight: The Power of the Exit

By using `return True` inside the loop, the function exits the second it finds a problem. It doesn't matter if there are a million more items left in the list; the computer stops working the moment the answer is known.

As you remember from **Chapter 2**, the `return` keyword doesn't just "break" a loop; it **completely terminates the function**. No code below that line will run once a value is returned. This is the ultimate tool for **Efficiency**—we call it an "Early Exit."

Project: Henderson's Global Inventory (HGI)

Background

Mr. Henderson is upgrading his warehouse from paper records to a digital inventory system. He needs a reliable **source of truth** that can track items, update stock, record sales, apply discounts, and calculate the total value of the warehouse.
In this project, you will practice **nested dictionaries**, **conditionals**, and **loops**. Each function represents a real warehouse task. The goal is not only to write working code, but also to think like a programmer building a useful system for a business.

In a file named `inventory.py`, implement the core inventory logic for the warehouse. The system uses a global dictionary called `store`. Each key is an `item_id`, and each value is a sub-dictionary with these fields:

- `"name"`: the item name
- `"price"`: the price of one unit
- `"stock"`: how many units are available

You must complete the following five functions:

- **`add_item(item_id, name, price, stock)`**: Add a new item to the warehouse or update an existing one. If the `item_id` already exists, increase its stock by the new amount. If the `item_id` does not exist, create a new entry.

  Example:

  - If item `101` already has stock `5`, then calling `add_item(101, "Pen", 2.5, 3)` should make the stock `8`.
  - If item `202` does not exist, calling `add_item(202, "Notebook", 4.0, 10)` should create a new record.

- **`process_sale(item_id, quantity)`**: Process a sale and update stock after the item is sold. This function must return different values depending on the situation:

  1. Return `None` if the `item_id` does not exist.
  2. Return `False` if the requested `quantity` is greater than the available stock.
  3. Otherwise, subtract the quantity from stock and return the **total cost**, which is `price * quantity`.

  Example:

  - `process_sale(101, 2)` on an item priced at `3.50` should return `7.0`.

  - If the stock is only `1`, then `process_sale(101, 3)` should return `False`.
  - If the item does not exist, the function should return `None`.

- **`run_audit(threshold)`**: Check the warehouse and find items with low stock. Return a **list of strings** for every item whose stock is strictly less than the given `threshold`.

  Format: `"ItemName (ID: 101) - Stock:  2"`

  Example:

  - If `threshold = 5` and an item has stock `2`, it should appear in the report.
  - If an item has stock `5`, it should **not** appear, because the stock must be strictly less than the threshold.

- **`apply_discount(category_keyword, percentage)`**: Search through all items and reduce the price of every item whose name contains the given keyword. The reduction should be based on the percentage value provided.

  Example:

  - If `category_keyword = "book"` and an item is named `"History Book"`, the price should be reduced.
  - If the old price is `100` and the percentage is `0.10`, the new price becomes `90`.
  - Items that do not contain the keyword should remain unchanged.

- **`get_total_value()`**: Calculate the total value of the entire warehouse. This is the sum of `price * stock` for every item in the store.

  Example:

  - If one item is worth `20` and another is worth `35`, the function should return `55`.
  - This represents the total money currently tied up in inventory.

inventory.py

```python
# HGI: Distribution Template
store = {}

def add_item(item_id, name, price, stock):
    """
    Adds a new item or updates stock for an existing item.

    Behavior:
    - If the ID already exists, increase the current stock.
    - If the ID is new, create a new item record.

    Example:
```

```
    - add_item(101, "Pen", 2.5, 3) on an existing item with stock 5
      should update the stock to 8.
    """
    # TODO
    pass

def process_sale(item_id, quantity):
    """
    Processes a sale by checking item existence and available stock
        .

    Returns:
    - None: if the item ID is not found
    - False: if there is not enough stock
    - float: the total cost of the sale (price * quantity)

    Example:
    - process_sale(101, 2) on an item priced at 3.50 should return
        7.0
    - process_sale(101, 5) should return False if stock is too low
    """
    # TODO
    pass

def run_audit(threshold):
    """
    Scans the inventory and returns a report of low-stock items.

    Returns:
    - A list of formatted strings for items whose stock is below
        threshold

    Example:
    - If threshold is 5, an item with stock 2 should be included
    - Format: "ItemName (ID: 101) - Stock: 2"
    """
    # TODO
    pass

def apply_discount(category_keyword, percentage):
    """
    Applies a discount to every item whose name contains the
        keyword.

    Behavior:
    - Search item names for the keyword
    - Reduce matching prices by the given percentage

    Example:
```

```
    - apply_discount("book", 0.10) turns 100.0 into 90.0
    """
    # TODO
    pass

def get_total_value():
    """
    Calculates the total value of all items in the warehouse.

    Returns:
    - The sum of (price * stock) for every item

    Example:
    - If one item is worth 20 and another is worth 35,
      this function returns 55.
    """
    # TODO
    pass

# --- HENDERSON'S INTERFACE: DO NOT MODIFY ---
while True:
    print("\n[1] Add [2] Sale [3] Audit [4] Discount [5] Value [6] Exit")
    cmd = input("HGI Command > ")

    if cmd == "1":
        i, n = int(input("ID: ")), input("Name: ")
        p, q = float(input("Price: ")), int(input("Qty: "))
        add_item(i, n, p, q)
    elif cmd == "2":
        i, q = int(input("ID: ")), int(input("Qty: "))
        res = process_sale(i, q)
        if res is None: print("Error: ID not found.")
        elif res is False: print("Error: Low stock.")
        else: print(f"Sale successful! Total: ${res:.2f}")
    elif cmd == "3":
        t = int(input("Threshold: "))
        report = run_audit(t)
        for line in report: print(line)
    elif cmd == "4":
        k, p = input("Keyword: "), float(input("Discount (e.g. 0.1): "))
        apply_discount(k, p)
    elif cmd == "5":
        print(f"Total Warehouse Value: ${get_total_value():.2f}")
    elif cmd == "6":
        break
```

Conditionals (Predict the Output + Reasoning)

For each question:

- Predict the output
- Explain WHY that output occurs

Do not run the code.

1.

```python
total = 120

if total > 100:
    print("Premium Discount")
elif total > 50:
    print("Standard Discount")
else:
    print("No Discount")
```

Output: _____

Why? _____

2.

```python
milk = 0

if milk > 0:
    print("Milk available")
elif milk == 0:
    print("Out of stock")
else:
    print("Invalid stock")
```

Output: _____

Why? _____

3.

```python
price = 50

if price >= 50:
    print("Expensive")

if price >= 30:
    print("Moderate")
else:
    print("Cheap")
```

Output: _____

Why? _____

4.

```python
bread = 2
milk = 1

if bread > 2 and milk > 0:
    print("Free Bread Offer")
elif bread > 1 or milk > 0:
    print("Partial Offer")
else:
    print("No Offer")
```

Output: _____

Why? _____

5. (Grading System - Independent Conditions)

A supermarket assigns reward levels based on purchases.

Python

```python
purchases = 6

if purchases > 5:
    print("Gold Member")

if purchases > 3:
    print("Silver Member")

if purchases > 0:
    print("Bronze Member")
```

Output: _____
Why? _____

Key Idea:
In this last question:

- All `if` statements run independently
- There is no `elif` or `else`
- Multiple outputs are possible

Loops (Predict the Output)

For each question:

- Predict the output
- Think carefully about how repetition works

Do not run the code. **Key Ideas to Think About:**

- When does the loop stop?
- What changes inside the loop?
- What happens when `break` or `continue` appears?
- Does every iteration always run fully?

1. Inventory Counting (while loop)

```python
stock = 3

while stock > 0:
    print("Item sold")
    stock = stock - 1
```

Output: _____

2. Restocking Process

```python
stock = 0

while stock < 3:
    print("Restocking")
    stock += 1
```

Output: _____

3.

```python
stock = 2

while stock > 0:
    print("Selling item")
```

Output: _____

Why does this happen? _____

4. For Loop (Supermarket Shelf Scan)

```python
items = ["milk", "bread", "eggs"]

for item in items:
    print("Checking", item)
```

Output: _____

5. Price Tags (range loop)

```python
for i in range(1, 4):
    print("Price label", i)
```

Output: _____

6. Skip Behavior (Important Concept)

```python
for i in range(1, 6):
    if i == 3:
        continue
    print("Item", i)
```

Output: _____

7. Break Statement (Early Stop)

```python
for i in range(1, 6):
    if i == 4:
        break
    print("Scanning", i)
```

Output: _____

Lists and Dictionaries (Predict the Output)

For each question:

- Predict the output
- Carefully trace how data changes step by step

Do not run the code.

1. Inventory List (Basic List Access)

Python

```python
items = ["milk", "bread", "eggs", "rice"]

print(items[1])
print(items[3])
```

Output: _____

2. Inventory Update (List Modification)

Python

```python
items = ["milk", "bread", "eggs"]

items.append("butter")
items[1] = "whole bread"

print(items)
```

Output: _____

3. Stock Check with Conditionals

```python
stock = ["milk", "bread", "eggs"]

if "milk" in stock:
    print("Milk available")
else:
    print("Milk missing")
```

Output: _____

4. Dictionary Inventory (Key Lookup)

```python
inventory = {
    "milk": 5,
    "bread": 0,
    "eggs": 12
}

print(inventory["milk"])
print(inventory["bread"])
```

Output: _____

5. Dictionary Update (Stock Change)

```python
inventory = {
    "milk": 5,
    "bread": 0
}

inventory["bread"] = 3
inventory["milk"] -= 2

print(inventory)
```

Output: _____

6. Loop Through List (Inventory Scan)

```python
items = ["milk", "bread", "eggs"]

for item in items:
    print("Checking", item)
```

Output: _____

7. Loop Through Dictionary (Keys Only)

```python
inventory = {
    "milk": 5,
    "bread": 2,
    "eggs": 0
}

for item in inventory:
    print(item)
```

Output: _____

8. Loop Through Dictionary (Values Check)

Python

```python
inventory = {
    "milk": 5,
    "bread": 0,
    "eggs": 2
}

for item in inventory:
    if inventory[item] == 0:
        print(item, "out of stock")
```

Output: _____

9. Mixed Logic (List + Condition + Loop)

Python

```python
items = ["milk", "bread", "eggs"]

for item in items:
    if item == "bread":
        print("Skip bread")
    else:
        print("Process", item)
```

Output: _____

Key Ideas to Understand:

- Lists are ordered → index matters
- Dictionaries are key-value → lookup matters
- Loops iterate through structure elements
- Conditionals filter or modify behavior during iteration

Debugging (Lists, Dictionaries, and Loops)

For each program:

- Identify what is wrong
- Explain why the bug happens
- Predict what would happen if it runs

Do NOT run the code.

1.

```python
items = ["milk", "bread", "eggs"]

for i in items:
    if i == 1:
        print("Found bread")
```

What is wrong? Why?

2.

```python
stock = 3

while stock > 0:
    print("Selling item")
```

What is wrong? Why does it never stop?

3.

```python
inventory = {
    "milk": 5,
    "bread": 0
}

print(inventory["eggs"])
```

What will happen when this runs?

4.

```python
inventory = {
    "milk": 5,
    "bread": 2
}

inventory["milk"] = inventory["milk"] - 1
inventory["bread"] = inventory["bread"] + 2

print(inventory)
```

Is there a logical error? If yes, explain it.
Hint: Think about unintended stock inflation/consumption.

5.

```python
items = ["milk", "bread", "eggs"]

for item in items:
    if item = "bread":
        print("Skip bread")
    else:
        print("Process", item)
```

What is wrong? What would happen if corrected?

6.

```python
items = ["milk", "bread", "eggs", "bread"]

for item in items:
    if item == "bread":
        items.remove(item)

print(items)
```

What is the problem in this logic?
Hint: Think about how lists change during iteration.

7. Stock Counting Bug

```python
stock = {"milk": 2, "bread": 0, "eggs": 5}

for item in stock:
    if stock[item] > 1:
        print(item, "available")
    else:
        print(item, "low stock")
```

Is the logic correct for all cases? Explain any issue.
Hint: What about zero stock vs low stock interpretation?

8.

```python
i = 0
items = ["milk", "bread", "eggs"]

while i < len(items):
    print(items[i])
```

Why does this code not behave correctly?

Key Debugging Skills:

- Check variable updates inside loops
- Watch list modification during iteration
- Understand dictionary key safety
- Distinguish assignment vs comparison
- Track loop termination conditions

Program Writing (Name Analysis System)

You are building a simple "name analysis system" for a school registration system. The system must analyze a user's name and find the most frequently occurring letter.

Task:
Write a program that:

- asks the user to enter their name
- counts how many times each letter appears
- prints the letter that appears the most

Important Rules:

- Ignore spaces in the name
- Treat uppercase and lowercase as the same letter
- If there is a tie, print the first most frequent letter encountered

Example Run 1:

Terminal Output

```
Input: David
Output: d
```

Explanation:

- d appears 2 times
- a, v, i appear 1 time each
- so d is the most frequent

Example Run 2:

Terminal Output

```
Input: Banana
Output: a
```

Explanation:

- a appears 3 times
- n appears 2 times
- b appears 1 time

Hints (Guided Thinking):

- Use a dictionary to store letter counts
- Loop through each character in the name
- Update counts as you go
- Keep track of the maximum frequency while looping

Challenge Thinking:
Ask yourself:

- How do I compare values while looping?
- How do I remember the "best answer so far"?

Program Writing (Greeting Decision System)

You are building a simple "bank greeting evaluation system."
A bank rewards customers based on how they are greeted when they enter.

Rules:
Write a program called `bank.py` that:

- asks the user for a greeting
- ignores leading spaces
- treats uppercase and lowercase the same
- applies the following rules:

- If the greeting starts with `"hello"` → output `$0`

- If the greeting starts with `"h"` (but not "hello") → output `$20`
- Otherwise → output `$100`

Important Note:

- "hello", "Hello", "hello there", "hello, Newman" all count as `$0`
- Only the starting word matters, not the full sentence

Example Runs:
Run 1

Terminal Output

```
Input: Hello
Output: $0
```

Run 2

Terminal Output

```
Input: Hello, Newman
Output: $0
```

Run 3

Terminal Output

```
Input: How you doing?
Output: $20
```

Run 4

Terminal Output

```
Input: What's happening?
Output: $100
```

Hints (Guided Thinking):

- Remove leading spaces using:
  - `strip()`

- Convert input to lowercase using:
  - `lower()`
- Check prefixes using:
  - `startswith("hello")`
  - `startswith("h")`

Decision Thinking Guide:
Ask yourself:

- Does this string begin with "hello" exactly?
- If not, does it at least begin with "h"?
- Otherwise, fall into default case

Key Insight:
You are not comparing full sentences.
You are only analyzing the **starting pattern of the string**.

Program Writing (Simple Arithmetic Interpreter)

You are building a simple calculator system that can understand and evaluate basic arithmetic expressions.
This program acts like a small interpreter.

Task:
Write a program called `interpreter.py` that:

- prompts the user for a mathematical expression
- reads input in the form:

$$x\ y\ z$$

  where:
  - `x` is an integer
  - `y` is one of: `+`, `-`, `*`, `/`

  - z is an integer
- evaluates the expression
- prints the result as a float with one decimal place

Important Note:
You may assume:

- input is always correctly formatted
- division by zero will never occur

Example Runs:
Run 1

Terminal Output

```
Input: 1 + 1
Output: 2.0
```

Run 2

Terminal Output

```
Input: 2 - 3
Output: -1.0
```

Run 3

Terminal Output

```
Input: 2 * 2
Output: 4.0
```

Run 4

Terminal Output

```
Input: 50 / 5
Output: 10.0
```

Hints (Guided Thinking):

- Use `input()` to get the full expression as a string

- Split the string using:
  - `split(" ")`
- Assign values directly:
  - `x, y, z = expression.split(" ")`
- Convert numbers using:
  - `int(x)` and `int(z)`
- Use conditionals to check operator:
  - if `y == "+"`
  - if `y == "-"`
  - etc.

Step-by-Step Thinking Guide:
Ask yourself:

- What are the three parts of the input?
- How do I separate them cleanly?
- What operation should each symbol perform?
- How do I ensure output is always a float?

Key Insight:
You are not just doing math.
You are teaching the computer to **understand structured text input and interpret it as a rule-based system**.

Program Writing (Coke Machine Simulation)

You are simulating a simple vending machine system.
A Coke machine sells a bottle for exactly **50 cents**.
The machine only accepts the following coins:

- 25 cents
- 10 cents
- 5 cents

All other inputs are ignored.

Task:
Write a program called `coke.py` that:

- continuously prompts the user to insert a coin
- keeps track of the total amount inserted
- updates the remaining amount after each valid coin
- ignores invalid coins (does not change the total)
- stops when at least 50 cents has been inserted
- outputs the change owed (if any)

Important Behavior Rules:

- The machine always starts with `Amount Due = 50`
- Only `25`, `10`, and `5` are valid coins
- Any other number is ignored completely
- The program ends only when total inserted ≥ 50

Example Runs:
Run 1

Terminal Output

```
Insert Coin: 25
Amount Due: 25

Insert Coin: 10
Amount Due: 15

Insert Coin: 5
Amount Due: 10
```

Run 2 (Invalid coin ignored)

Terminal Output

```
Insert Coin: 30
Amount Due: 50
```

Run 3 (Exact payment)

Terminal Output

```
Insert Coin: 25
Insert Coin: 25
Change Owed: 0
```

Run 4 (Overpayment)

Terminal Output

```
Insert Coin: 25
Insert Coin: 10
Insert Coin: 25
Change Owed: 10
```

Hints (Guided Thinking):

- Start with a variable:
  - `amount_due = 50`
- Use a loop that continues while:
  - `amount_due > 0`
- Inside the loop:
  - ask user for coin input
  - convert input to integer
  - check if coin is valid (25, 10, or 5)
- If valid:
  - subtract from `amount_due`
- If invalid:

  - do nothing (ignore input)

Step-by-Step Thinking Guide:
Think like the machine:

- "How much money do I still need?"
- "Did the user give me a valid coin?"
- "Should I reduce the amount or ignore it?"
- "Have I reached 0 or less yet?"

Final Output Rule:
When the loop ends:

- print `Change Owed:` remaining value (if any)

Key Insight:
This is not just input/output.
This is a **state machine**:

- the machine's state = amount still owed
- each input updates the state
- loop ends when state condition is satisfied

Chapter 4

Libraries

ASSUME

One afternoon, Mr. Henderson calls you into the back office. You have just finished building the Henderson Global Inventory (HGI) system, where you mastered the mechanics of the `while` loop and even built a method to catch duplicate serial numbers.

Mr. Henderson: "David, the way you organized our warehouse using those loops was pure genius. You've moved us from manual paper records to a digital system. You keep impressing us by outperforming our senior contractors."

David: "Thank you so much, Mr. Henderson. This book I am currently reading is really helping me figure out these problems."

Mr. Henderson: "David, you are really good at your job, and that is why I am making you our new Junior Software Developer. You can now focus on building software that optimizes our efficiency significantly. As we expand, we need to do even more complex things, like building a robust security system and calculating the exact time a shipment will arrive."

David: "Thank you so much, Mr. Henderson. I will start working on this right away."

4.1 New Job, New Role

Congratulations! You have come a long way. Being promoted from a summer intern to a junior software developer in such a short time is impressive, but you also have more responsibility coming your way. These new tasks are not like the problems you are used to. You cannot solve everything with a few functions and loops anymore.

Here are your responsibilities:

- **Arrival tracking:** Automatically record the exact date and time a shipment arrives so

Mr. Henderson knows precisely when stock hits the floor.

- **Security optimization:** Create unique, random security passcodes for the HGI logs to prevent unauthorized access.
- **Market trend analysis:** Compare your prices with current currency values and make pricing decisions based on that information.

This is not an easy set of projects. You still have a lot to learn before you can build these optimizations, but do not worry; by the end of this chapter, you will have the tools you need to get started. We will cover:

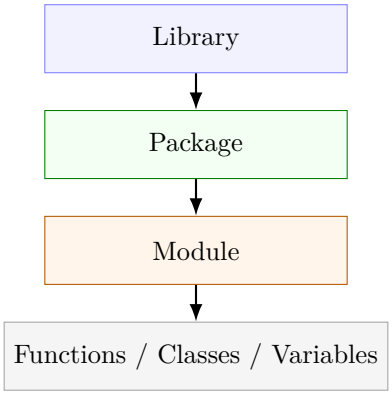
- **Libraries:** Borrowing "superpowers" from other developers so you don't have to reinvent the whole program.

4.2 Libraries

Imagine you are building a house, but instead of having to make every single tool yourself, you are given a fully equipped toolbox created by experts. You don't waste your time forging hammers or designing screwdrivers; you just pick what you need and start building.

Libraries follow the same idea. When I first learned about libraries, one of my teachers jokingly called them "legal plagiarism." That phrase sounds strange, but the idea is simple. Libraries are organized pieces of code that other people have already written and shared for us to use. Instead of rebuilding everything from scratch, we responsibly reuse their work.

There are three major components we need in order to understand libraries.



4.2.1 Modules

- A module is a single Python file that Python can load and reuse. Since libraries are organized collections of code, a module is simply one Python file that contains reusable code. Python loads that file, executes it, and makes its contents available through the file's name. This first definition is easiest to understand with user-defined modules.

There Is No Dumb Question

David: What is the difference between user-defined modules and non-user-defined modules?
Feynman: Great question, David. Actually, modules are classified into three types.

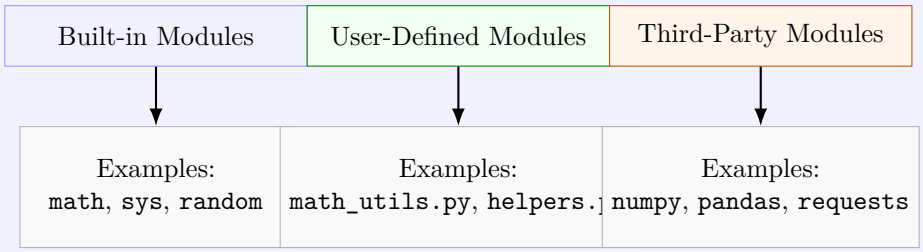
1. Built-in modules,
2. User-defined modules, and
3. Third-party modules.

1. **Built-in modules:** Built-in modules are part of Python's standard library and come preinstalled with the language, so you can use them immediately without installing anything. Examples include `math`, `sys`, and `random`.

2. **User-defined modules:** These modules are files you create yourself, usually `.py` files, which contain your own functions, classes, or variables that you import into other programs.

3. **Third-party modules:** These are written by other developers and are not included in Python by default. You install them using tools like `pip`. They are external code, but once installed, you use them like standard modules.

David: So the full classification is into three categories, not two?
Feynman: Yes, that is correct. Dividing modules only into user-defined and non-user-defined groups is too broad. These three categories give us a clearer picture.



For example, let's look at this user-defined module:

math_utils.py

```
def add(a,b):
    return a + b
```

Here we created one Python file that takes two numbers and adds them together. Now we

are going to call this function from another Python file.

Important

To use user-defined modules, you usually keep them in the same project folder or import them through a valid package path so Python knows where to find them.

Don't worry about the keywords in the code below; we will cover them shortly.

module.py

```
#example
import math_utils

print(math_utils.add(5,3))
```

Terminal Output

```
8
```

As you can see, we used our previous code as a module, imported it into out current python file and call the function from the module, so if we want to use a function from another python file, instead of copy pasting or rewriting the whole function we can just import it and use it here.

As you can see, we used our earlier code as a module, imported it into our current Python file, and called the function from that module. So if we want to use a function from another Python file, we do not need to copy and paste or rewrite the whole function; we can simply import it and use it.

We can also use modules from other sources. Some modules are preinstalled with Python when you install it from `python.org`, while others must be installed separately. For this example, we will use a built-in module called `math`.

math.py

```
#example
import math

print(math.sqrt(16))
```

Terminal Output

```
4
```

The earlier definition is easiest to understand for user-defined modules, but imported modules follow the same larger idea. In general, a module is a single importable unit of Python code that defines a namespace. It can come from:

- a `.py` file, as discussed before;

- a built-in module such as `math`; or
- a third-party module installed from outside Python's standard library.

Important

A namespace is like a labeled toolbox. In an earlier chapter, we used a similar analogy for variables, but a namespace is a more general version of that idea. When we store a variable like `x = 10`, the value is stored in a namespace and the label attached to it is `x`.

- This idea applies not only to variables but also to modules. The functions inside a module live inside that module's namespace. When we import the module, Python gives us access to those names so we can call the specific function we want.

4.2.2 Package

- Think of packages as parents of modules. A package is a directory (folder) that contains multiple modules. Just as parents can have more than one child, a package can contain more than one module. Unlike a single module, a package organizes several related namespaces.

directory

```
my_package/
    math_utils.py
    string_utils.py
```

As you can see, inside the folder we have two different Python files, and each of those files is a module. In practice, you usually access a module through its package, which is why package names appear in many import statements.

Just as we can create modules ourselves, we can also create packages. We do that by creating a folder and placing Python files inside it. As noted earlier, Python still needs a valid location so it can find those files correctly.

For the package above, we can use code like this to work with it properly.

math_utils.py

```
def add(a,b):
    return a + b
```

We define the function just as we did before, but now the module file must live inside the package that we plan to import from. The main difference is the way we write the import statement.

In fact, for modules inside packages, we have two major ways to import them.

importing_packages_1.py

```python
#Method 1
import my_package.math_utils

print(math_utils.add(2,5))
```

Terminal Output

```
7
```

importing_packages_2.py

```python
#Method 2
from my_package import math_utils

print(math_utils.add(2,5))
```

Terminal Output

```
7
```

We can also import specific functions from a module.

importing_functions.py

```python
# importing function
from my_package.math_utils import add

print(add(2,5))
```

Terminal Output

```
7
```

The way we import built-in packages and third-party packages is no different in principle.

One real third-party package we can look at is `numpy`.

- `numpy` is one of the most well-known packages for fast numerical computing, but it is not preinstalled. We will see how to install external packages shortly.

directory

```
numpy/
    __init__.py
    core/
    linalg/
    random/
```

numpy_package.py

```python
# example
import numpy as np

A = np.array([[1, 2], [3, 4]])
det = np.linalg.det(A)
print(det)
```

Terminal Output

```
-2.0
```

As you can see, this form is a little longer. We start with the package name, then the module, and then the function. Here, `numpy` is the package and `linalg` is the module. Later, we will also see shorter ways to write imports.

`__init__.py`

- In the directory listing for the user-defined package, you saw a file named `__init__.py`. Its name looks unusual, but it has an important purpose.
- `__init__.py` is a special Python file. In many projects, it may even be empty, but historically it told Python, "This folder should be treated as a package." You can think of it as the package's greeter: it is the first file Python notices when the package is accessed.
- Starting with Python 3.3, `__init__.py` became optional in many cases because Python supports namespace packages. Still, you will continue to see it often in real projects.

4.2.3 Libraries

- Finally, libraries. A library is the highest level in this discussion. It is a broader collection of packages and modules designed to solve larger problems. Libraries are typically installed as one unit, for example with `pip`, and they may contain many packages.

directory

```
NumPy (library you install)/
  numpy (top-level package you import)
     linalg (module/subpackage)
     random
     fft
     ...
```

In the previous section, we mentioned one of the most famous packages, `numpy`, but there is a larger distribution behind it: NumPy. That can feel confusing at first, but a practical way to think about it is this: the installable project is the larger library distribution, while `numpy` is the top-level package you import in code.

NumPy mainly exposes one top-level package, `numpy`, but many other libraries expose more than one package. One distribution can provide one top-level package or several.

Important

We do not usually "import a library" in the same way we import a module. First, we install a library using a tool such as `pip`. Then, inside our code, we import the specific packages or modules that library provides.

There Is No Dumb Question

David: Mr. Feynman, how do we install these external libraries?
Mr. Feynman: Great question, David. To install external libraries, we use a tool called `pip`. It is the standard tool for installing Python packages. Think of it as a delivery system that brings code written by other developers into your project. You can install packages with `pip` as follows:

- **Step 1** – Check whether `pip` is already installed.

  For Windows:

  cmd

  ```
  pip --version
  ```

  For Linux (Debian / Ubuntu):

  terminal

  ```
  pip3 --version
  ```

For macOS:

```terminal
pip3 --version
```

- **Step 2** – Upgrade `pip`. For Windows:

```cmd
python -m pip install --upgrade pip
```

For Linux (Debian / Ubuntu):

```terminal
python3 -m pip install --upgrade pip
```

For macOS:

```terminal
python3 -m pip install --upgrade pip
```

- **Step 3** – Install a library. Example: install NumPy. For Windows:

```cmd
pip install numpy
```

For Linux (Debian / Ubuntu):

```terminal
pip3 install numpy
```

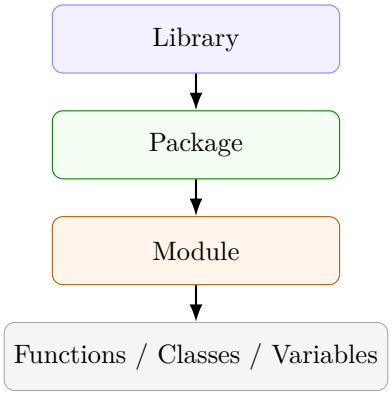
For macOS:

```terminal
pip3 install numpy
```

David: So what exactly is pip?
Mr. Feynman: `pip` is Python's package manager. It runs in the terminal or command

prompt and allows you to install, upgrade, and remove libraries that are not part of Python's built-in standard library.
David: Do I need to know every library?
Mr. Feynman: Trying to know every library is a bad strategy. There are thousands of Python libraries, and many are niche. Even experienced developers know only a small subset well.

Look at the hierarchy below to see their relationship more clearly.



4.3 Keywords

4.3.1 import

- The most important keyword in this chapter is `import`. The name is self-explanatory because we use it to bring code into our program. We can use `import` with a whole module, a package, or a specific component. It tells Python what to make available in the current file.

example1.py

```
#example - importing a function from a module
from math import sqrt
print(sqrt(25))
```

Terminal Output

```
5
```

example2.py

```
#example - importing a module
import math
print(math.sqrt(16))
```

Terminal Output

```
4
```

example3.py

```python
#example - importing a package
import numpy as np
arr = np.array([1,2,3])
print(arr.mean())
```

Terminal Output

```
2
```

Important

The `as` keyword in Python is used to assign an **alias** (a temporary name) to something, not just to packages. It does not create new functionality. It simply renames something for convenience or clarity within a specific context.

4.3.2 from

- Another crucial keyword is `from`. `import` tells Python what to import, while `from` tells Python where to import that component from. The syntax is close to a normal English statement: "from here import this."
- The "here" part is important. The item we import can be a module, package, function, class, or variable, but the source we import from is usually a package or module.

example1.py

```python
# example - importing from a module
from math import sqrt
print(sqrt(49))
```

Terminal Output

```
7
```

example2.py

```python
# example - importing from a package
from numpy import array
n = array([1,2,3])
print(n)
```

Terminal Output

```
[1,2,3]
```

Important

- The keyword `from` cannot stand alone. It depends on `import`. By itself, `from` only names a location; `import` tells Python what to bring in from that location.
- With these keywords, you must also provide the name of the function, package, module, class, or variable you want to import. Without that name, the statement is incomplete.

4.4 Third-Party Modules, Packages, and Libraries

4.4.1 sys

- `sys` is a module inside the Python Standard Library; it is not a separate library by itself. The standard library contains many modules such as `sys`, `os`, `math`, and `json`.
- `sys` acts as a bridge between your Python code and the Python runtime that is executing your program. It gives you access to interpreter behavior and execution-environment data.
- It is bundled with Python itself.
- It is available immediately after installing Python, so no `pip` command is needed.

Let's look at some runtime arguments using `sys`.

sys1.py

```python
# example
import sys
print(sys.argv)
```

`sys.argv`

- `sys.argv` is a list of strings representing how a program was launched.

When you run a Python file in the terminal, instead of typing only `python sys1.py`, you can add extra words after the filename, and `sys.argv` will capture them.

For Windows

cmd

```
python sys1.py nathan mikiyas
```

For Linux (Debian / Ubuntu)

```terminal
python3 sys1.py nathan mikiyas
```

For macOS

```terminal
python3 sys1.py David Feynman
```

When you run your Python program like that, `sys.argv` collects everything to the right of `python` or `python3` and stores those pieces as items in a list. So the output will be:

Terminal Output

```
['sys1.py','David','Feynman']
```

You can accept input from the user not only by calling `input()`, but also by reading command-line arguments when the program starts.

But how do we access the inputs?

Since we can access the elements of a list, we can also assign them to variables.

For instance, let's look at the code below:

```sys2.py
import sys
name = sys.argv[1]
print(f"Hello, {name}")
```

We assigned the second element of the `sys.argv` list to a variable called name, and later on we used that variable to say hello to the user.

```terminal
python sys2.py David
```

Terminal Output

```
Hello, David
```

With the same principle, we can access and use data from the user directly from the command line, which is often faster and more efficient than asking for input interactively.

There Is No Dumb Question

David: But what happens if we do not provide any argument? What if we leave it blank instead of writing the name David?
Mr. Feynman: Good question, David. In that case, we get an error. If we do not pass any argument to `sys.argv`, then the list will be only `['sys2.py']` instead of `['sys2.py', 'Nathan']`. That means we have index `[0]`, but not `[1]`. So when we try to access an argument that does not exist, Python raises an error. For example:

sys2.py

```
import sys
name = sys.argv[1]

print(f"Hello, {name}")
```

cmd

```
python sys2.py
```

Terminal Output

```
Traceback (most recent call last):
File sys2.py, line 2, in <module>
name = sys.argv[1]
IndexError: list index out of range
```

So we have to prevent that from happening. This brings us back to conditionals. We can write a conditional statement to check whether the argument is present. For that, we use the built-in function `len()`, which is short for "length." `len(sys.argv)` tells us how many elements are in the list. If the length is less than `2`, then the argument is missing.

sys3.py

```
import sys

if len(sys.argv) < 2:
    sys.exit("Missing argument")

print(sys.argv[1])
```

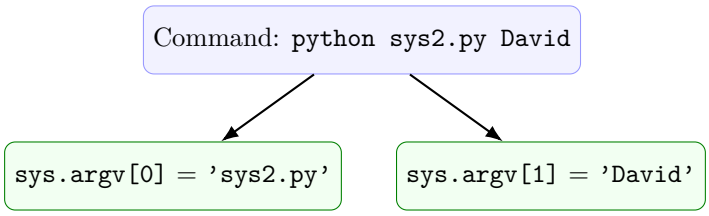
Now we have a conditional that prints an error message when no argument is supplied, so we have handled the problem safely.

terminal

```
python sys3.py
```

Terminal Output

```
Missing argument
```



sys.exit

- This function terminates Python execution immediately. You can display an error message for the user, but after that the program stops right away.

For example:

sys_exit1.py

```
import sys
sys.exit("Error occurred")
```

Terminal Output

```
Error occurred
```

But this is not the only use of `sys.exit()`. We can also pass a numeric argument that represents an exit status, or return code. That value tells the operating system why the program stopped.

The "Traffic Light" System

Think of it like the lights on your warehouse control panel:

- **`sys.exit(0)`** (Green Light): This means "Success." Everything went exactly as planned.

sys_exit_0.py

```
import sys

dict = {"name": "David"}

item = "apple"

```

```python
if item in dict:
    print(f"The name is {dict[item]}")
    # Signal to the computer: "Everything went perfectly!"
    sys.exit(0)
```

- **`sys.exit(1)`** (Red Light): This means "General Error." Something went wrong.

sys_exit_1.py

```python
import sys

dict = {"name": "David"}

item = "apple"

if item not in dict:
    print("ERROR: Name not found in the dictionary.")
    # Signal to the computer: "Stop! Something is wrong or missing!"
    sys.exit(1)
```

- **`sys.exit(2)`** and higher: These are used for specific errors. For example, you might decide that `2` means "Database Connection Failed" and `3` means "User Access Denied."

`sys.path`

- There is one question we have not covered yet: how does Python know where to look when we import modules?
- `sys.path` is a list of directory paths that Python uses to decide where to look for modules when you use `import`. Python searches through the folders listed in `sys.path` until it finds a matching module.

Let's say we have a module called `mymodule`.

sys_path_1.py

```python
import mymodule
```

Python:

- Goes through each folder in `sys.path`
- Checks if mymodule.py exists
- If found, it imports it
- If not found, it raises a `ModuleNotFoundError`.

sys_path_2.py

```python
import sys
print(sys.path)
```

Terminal Output

```
[
'',
'/usr/lib/python3.x',
'/usr/lib/python3.x/lib-dynload',
'/home/user/.local/lib/python3.x/site-packages'
]
```

Each entry is a directory Python will scan for modules.

Problem: Express Lane

Right now, to run the HGI system, David has to start the program and navigate through menus. As the new Junior Software Developer, you want a faster method. Mr. Henderson wants to check the price of "Milk" without clicking through three different screens.

Solution

The `sys` solution: Use `sys.argv`. This allows David to type `python inventory.py Milk` directly into the terminal. The `sys` module grabs the word "Milk" and hands it to the program.

inventory.py

```python
import sys

# --- THE SYS.ARGV "EXPRESS LANE" PROBLEM SOLVER ---

# sys.argv[0] is always the name of the script (inventory.py)
# sys.argv[1] will be the item name David types in the terminal

if len(sys.argv) > 1:
    # If David typed an extra word after 'python inventory.py'
    target_item = sys.argv[1]
    price = get_item_price(target_item)

    if price:
        print(f"EXPRESS LANE: The price of {target_item} is {price}")
        sys.exit(0) # Emergency Stop: Job is done, don't show the menu!
    else:
        print(f"EXPRESS LANE: Item '{target_item}' not found in HGI.")
        sys.exit(1) # Exit with an error code
```

4.4.2 datetime

The `datetime` module exists to represent and manipulate:

- dates (year, month, day)
- times (hour, minute, second, microsecond)
- combinations of both
- time differences (durations)

date

The `date` class represents only a calendar date.

date.py

```python
from datetime import date

d = date(2026, 4, 25)
print(d)
```

Terminal Output

```
2026-04-25
```

Important

The `datetime` module is especially useful for real systems like HGI because businesses often need to store when an event happened, compare dates, or calculate how much time has passed.

Terminal Output

```
2026-01-01
```

You get:

- year
- month
- day

No time information.

time

Represents only the time of day.

date.py

```python
from datetime import time

t = time(14, 30, 0)
print(t)
```

Terminal Output

```
14:30:00
```

- hour
- minute
- second

No date is attached.

datetime

This is the important one. It combines both date and time.

datetime.py

```python
from datetime import datetime

dt = datetime(2026, 1, 1, 14, 30, 0)
print(dt)
```

Terminal Output

```
2026-01-01 14:30:00
```

This is what most real systems use.

Getting the current time

This is one of the most common operations.

current_time.py

```
from datetime import datetime

now = datetime.now()
print(now)
```

This gives the current local date and time. **Common mistake:** Assuming this is universal time. It is not. It uses your computer's clock, so it gives your local time.

Time differences

This is where `datetime` becomes powerful.

duration.py

```
from datetime import datetime, timedelta

now = datetime.now()
later = now + timedelta(days=7)

print(later)
```

This adds 7 days.

You can also subtract:

duration.py

```
diff = later - now
print(diff)
```

Terminal Output

```
7 days, 0:00:00
```

So instead of points in time, `timedelta` represents durations.

Formatting dates

Raw `datetime` output is not user-friendly. You can format it with `strftime`.

- `strftime` (string format time) is a method used on `datetime` objects to convert a `datetime` into a formatted string.

Basic Usage:

```
string_formatting.py
from datetime import datetime
now = datetime.now()
print(now.strftime("%Y-%m-%d"))
```

How it works

You pass a format string made of special codes.
Each code is replaced by part of the date or time.

For example:

```
strftime.py
now.strftime("%Y-%m-%d %H:%M:%S")
```

Date

- `%Y` → 4-digit year (2026)
- `%m` → month (01-12)
- `%d` → day (01-31)

Time

- `%H` → hour in 24-hour format (00-23)
- `%M` → minute (00-59)
- `%S` → second (00-59)
- `%I` → 12-hour clock
- `%p` → AM or PM
- `%a` → abbreviated weekday name

So the output would be:

Terminal Output

```
2026-04-25 14:30:00
```

You can format it in any way you like:

strftime.py

```
# Human readable format
now.strftime("%d/%m/%Y")
```

Terminal Output

```
01/01/2026
```

strftime.py

```
# 12-hour format
now.strftime("%I:%M %p")
```

Terminal Output

```
02:30 PM
```

There Is No Dumb Question

David: But time is constantly changing. Does the string update as time changes? **Mr. Feynman:** Sadly, no. The date and time are converted into a string. That string does not change by itself as time passes. It only reflects the moment when you created it. **David:** What about the time object? Does it modify the current local time? **Mr. Feynman:** No. It does not modify the object or the current time. It creates a new string and leaves `now` unchanged.

Problem: Package Arrival

As a junior software engineer, David has to move as much "static" data (just numbers) as possible into "dynamic" records (history and logs). Instead of only updating the stock number in the dictionary, David needs to keep track of arriving packages in a better way. But how?

Solution

Alternatively, David can create a second dictionary or a list called `delivery_log`. Every time a shipment or package arrives, the computer "stamps" the current time onto the record. This is usually called a *timestamp*.
David can implement this idea using the following code:

inventory.py

```python
from datetime import datetime  # The professional digital clock

# 1. Our Warehouse Records
inventory = {"Milk": 50, "Bread": 30}
delivery_log = []

def track_arrival():
    item = input("What item arrived? ").capitalize()
    qty = int(input(f"How many units of {item}? "))

    # 2. Get the current time and format it
    # %B = Month Name, %d = Day, %Y = Year, %X = Time
    arrival_time = datetime.now()
    clean_time = arrival_time.strftime("%B %d, %Y at %X")

    # 3. Update the Inventory and the Log
    if item in inventory:
        inventory[item] += qty
    else:
        inventory[item] = qty

    # Create the "Paper Trail"
    log_entry = {
        "item": item,
        "quantity": qty,
        "timestamp": clean_time
    }
    delivery_log.append(log_entry)

    print(f"\n[SUCCESS] {qty} units of {item} logged at {clean_time}.")
```

4.3 random

ASSUME

Mr. Henderson stops at David's desk, looking really concerned but composed.

Mr. Henderson: "David, I think we have had a data breach. Global Mart just launched a promotion for a 'Weekend Blowout Sale'. Their prices are exactly 5% lower than ours on every single item. Even the specialized Ethiopic spices we just stocked!"

David: "That is... suspiciously specific, Mr. Henderson. How would they know our exact markup?"

Mr. Henderson: "That is what I am wondering! One of my floor managers said he saw a stranger near your terminal last night when you were at your club meeting. The screen was just... open. Our whole HGI system was right there for anyone to see."

David: "Wait, did they just look at the screen?"

Mr. Henderson: "Worse. I think they plugged in a thumb drive. Since we don't have a `login page`, they probably just ran a 'System Audit' and copied the entire pricing dictionary. They didn't even have to hack anything; we basically gave it to them."

David: "I see the problem. We've been focusing on the 'engine' but we forgot the 'vault'." **Mr. Henderson:** "Exactly. I need you to lock this down, David. If someone isn't you, they shouldn't even see the word 'Milk' on that screen. And make the password something strong. No more '12345' or 'Henderson1'."

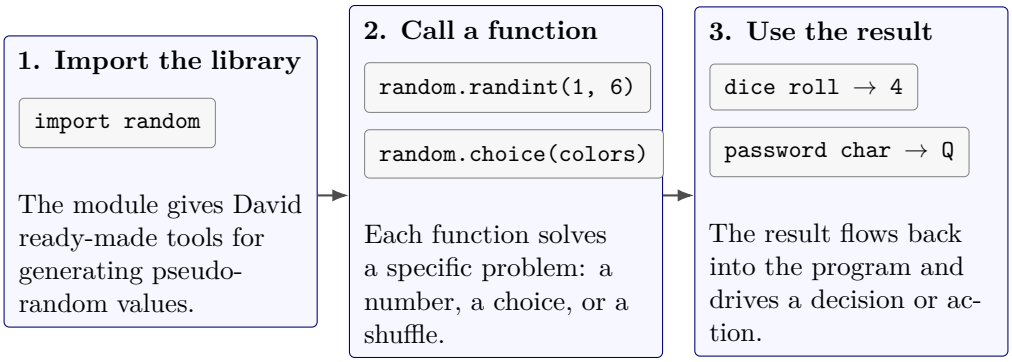
David: "On it, sir."

You have faced a security problem, which is one of the most common software problems, but how do you fix it?

By the end of this chapter, we will see how to solve problems like this.

- `random` is a common module. It generates pseudo-random values. That means the numbers are not truly random; they are produced by a deterministic algorithm and can be reproduced. For most everyday programming tasks, however, they are random enough to be useful. These values are used for:
  - games (guess my number)
  - randomized selection
  - shuffling data
- Like `sys`, the `random` module gives us several important functions for different purposes.

How the `random` Module Fits into a Program



4.3.1 `random.random()`

- The `random` function selects a random float between the range of 0.0 and 1.0
  - 0.0 <= x < 1.0
- Example outputs:
  - 0.12
  - 0.874
  - 0.934

random_1.py

```python
import random
print(random.random())
```

Terminal Output

```
0.54
```

4.3.2 `random.randint(a,b)`

- The `randint` function returns an integer rather than a float within a user specified range.
  - a <= x <= b (inclusive on both ends)

random_2.py

```python
import random

print(random.randint(1, 6))
```

Terminal Output

```
3
```

4.3.3 `random.choice(sequence)`

- The `choice` function returns a random element from a given sequence (list, tuple, string, any sequence type).

random_3.py

```
import random

colors = ["red", "blue", "green"]

print(random.choice(colors))
```

4.3.4 `random.shuffle(list)`

- The `shuffle` function rearranges the elements of the list into a random sequence. Modifies the list in place.

random_4.py

```
import random

cards = [1, 2, 3, 4, 5]
random.shuffle(cards)

print(cards)
```

Terminal Output

```
[4,2,5,3,1]
```

A common source of confusion for programmers using `random.shuffle` is what it does to the list. The function does not return a new list; instead, it rearranges the elements of the original list in place, since lists are mutable.

Wrong Use

wrong_use.py

```
shuffled = random.shuffle(cards)
print(shuffled)
```

Correct Use

correct_use.py

```
random.shuffle(cards)
print(cards)
```

4.3.5 `random.randrange(start, stop, step)`

- `randrange` is similar to `randint`, but it gives you more control because you can specify a start, a stop, and a step.

randrange.py

```
import random

print(random.randrange(0, 10, 2))
```

Terminal Output

```
6
4
2
0
8
```

What makes `randrange` special is that it builds a specific sequence from the instructions provided by the developer, then selects a random value from that sequence.

- In our situation, the instructions specify that the sequence has to begin at zero, increase in increments of two, and end before reaching 10.

Problem: Strong Password

To recover from the security issue, David must strengthen his password and create a login page that uses a robust password. But how can he accomplish this by using the `random` module?

Solution

David creates a separate authentication key generator (password generator) to reduce the risk of weak-password breaches. He then imports that file as a module from `inventory.py`.

auth_generator.py

```python
import random
import string
import sys

def generate_key(length=16):
    """
    Generates a high-strength random password.
    Includes letters, digits, and special characters.
    """
    password = ""
    chars = string.ascii_letters + string.digits + "!@#\$%^&*"

    # We use random.choice to pick characters one by one
    for _ in range(length):
        pick = random.choice(chars)
        password += pick

    return password

"""If David runs this file directly, it prints a new key. If
   it is imported, only the function is used."""
if __name__ == "__main__":
    print("--- NEW WAREHOUSE KEY GENERATOR ---")
    print(f"Generated Key: {generate_key()}")
    sys.exit(0)
```

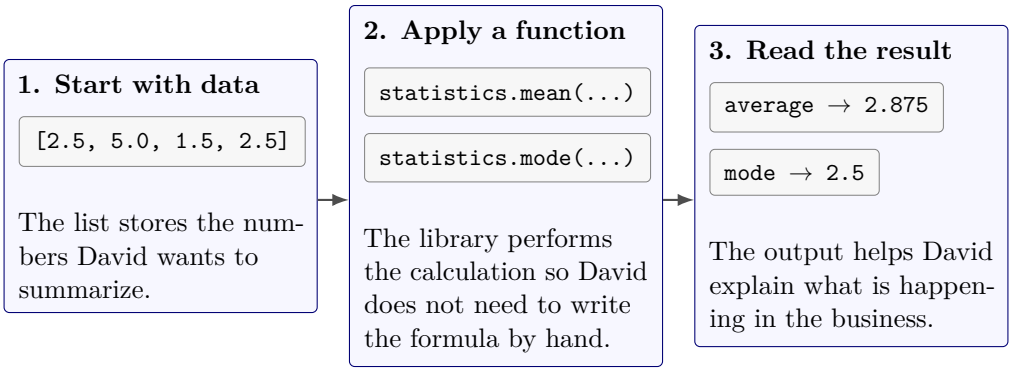
inventory.py

```python
import sys
import auth_generator  # <--- This imports David's security file as a module!

def run_login():
    # In a professional system, this password would be stored securely.
    # For now, we are using a set key for the login.
    correct_password = "Henderson_Secure_2026!"

    print("\n--- HENDERSON GLOBAL INVENTORY: SECURE ACCESS ---")
    attempts = 3

    while attempts > 0:
        entry = input(f"Enter Access Key ({attempts} attempts remaining): ")

        if entry == correct_password:
            print("Access Granted. Welcome back, David.")
            return True
        else:
            attempts -= 1
            print("Invalid Key!")

    # If David fails 3 times, the system shuts down for safety
    print("\n[!] SECURITY ALERT: Unauthorized access detected.")
    sys.exit(1)

# --- SYSTEM START ---
run_login()
```

4.4 statistics

- The statistics module provides basic descriptive statistics for numerical data.
  - It answers questions like:
    - What is the average
    - What value appears the most
    - How spread out is the data
  - It is a clean, built-in module for summarizing data.

How the statistics Module Turns Data into Insight



4.4.1 `statistics.mean()`

- This function calculates the average of a given data.

statistics_1.py

```python
import statistics

data = [1, 2, 3, 4, 5]
print(statistics.mean(data))
```

Terminal Output

```
3
```

There are some errors you may encounter if you pass an empty list or a value that is not a sequence of numbers, such as a string. The data should contain integers or floats.

4.4.2 `statistics.median(data)`

- This function calculates the median of a given data set.

statistics_2.py

```python
import statistics

data = [1, 3, 5]
print(statistics.median(data))
```

Terminal Output

```
3
```

For a data set with an even number of values, the `statistics.median()` function sorts the data in increasing order, picks the two middle values, and calculates their average.

statistics_3.py

```
data = [1, 2, 3, 4]
print(statistics.median(data))
```

Terminal Output

```
2.5
```

4.4.3 `statistics.mode(data)`

- This function calculates the most frequent value of a given data.

statistics_4.py

```
import statistics

data = [1, 2, 2, 3]
print(statistics.mode(data))
```

Terminal Output

```
2
```

There is one catch. If no value appears more than once in a multi-element data set, older Python versions may raise an error, while newer Python versions return the first value encountered. A safer alternative is `statistics.multimode(data)`, which returns a list of all values tied for the mode.

statistics_5.py

```
import statistics

data = [1, 2, 3, 4]
print(statistics.multimode(data))
```

Terminal Output

```
[1,2,3,4]
```

4.4.4 `statistics.stdev(data)`

- The `stdev` function measures the spread of data, or how far values deviate from the mean. In general, standard deviation is defined as follows:

Population Standard Deviation

- Use this when you have the entire population and there is no sampling uncertainty.

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}$$

- $\sigma \rightarrow$ Population standard deviation
- $\mu \rightarrow$ Population mean
- $x_i \rightarrow$ i-th data value
- $\Sigma \rightarrow$ Summation operator
- $\frac{1}{N} \rightarrow$ Averaging factor
- $\sqrt{\cdot} \rightarrow$ Square root
- $i \rightarrow$ Index

Sample Standard Deviation

- Use this when you do not have every data point and are working with a sample from a larger population.

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Meaning of the symbols

- $s \rightarrow$ Sample standard deviation
- $\bar{x} \rightarrow$ Sample mean
- $x_i \rightarrow$ i-th data value
- $\Sigma \rightarrow$ Summation operator
- $n \rightarrow$ sample size
- $\sqrt{\cdot} \rightarrow$ Square root
- $i \rightarrow$ Index

  For further reading: `https://youtu.be/ALn1McUXg-c?si=UP2a3KilMf1XqJPP`

If you are not familiar with the concept of standard deviation, no worries, you can stick to the first definition we covered. Our function measures the average distance of values from the mean.

- stdev = sample standard deviation
- pstdev = population standard deviation

statistics_6.py

```python
import statistics

data = [2, 4, 4, 4, 5, 5, 7, 9]

sample_std = statistics.stdev(data)
population_std = statistics.pstdev(data)

print("Sample std:", sample_std)
print("Population std:", population_std)
```

Terminal Output

```
Sample std:  2.138 Population std:  2.000
```

What does this mean?

To understand what is happening here, you can follow the following steps:

Step 1: Understand the data

- Data = [2,4,4,4,5,5,7,9]
  - Mean = 5

Step 2: Population standard deviation = 2.000

- **Interpretation:**
  - You are assuming this dataset is the entire population
  - The average distance of values from the mean is 2 units.

Step 3: Sample standard deviation = 2.138

- **Interpretation:**
  - You are pretending this data is just a sample of a larger unseen population
  - Because samples underestimate spread, the formula inflates the result a little bit.

Important

`stdev` does not know where the data came from or how much of the original population it represents. It applies Bessel's correction automatically by using $n - 1$ in the denominator. That makes it appropriate for samples, not for complete populations.

One error we might encounter while using these functions concerns the number of data points. There must be at least two data points to calculate the standard deviation of a data set.

statistics_7.py

```
statistics.stdev([5]) # error
```

There Is No Dumb Question

David: What will happen if we put nothing inside the functions in `statistics`?
Feynman: That is a really good question, David. If we put nothing inside these functions, it is like calling a function without the arguments it needs. The function then has nothing to operate on, so Python raises an error.
David: What if we enter strings instead of numbers?
Feynman: You are on fire today, David. These functions calculate values such as the mean, mode, and standard deviation. Strings cannot go through those numeric operations. Only integers and floats can, so Python raises another error.

Problem: Business Understanding

To help Mr. Henderson understand his business better, David can integrate the `statistics` library into the HGI system. This allows him to analyze the sales history and find the average spending and the most common price point. How can David help Mr. Henderson with this problem?

Solution

David will create a list called `sales_history` to store the total of every transaction. Then, he will add a "Reports" section to his menu.

inventory.py

```python
import statistics
import sys

# David's tracking data
inventory = {"apple": 0.5, "milk": 2.5, "bread": 1.5}
sales_history = [2.5, 2.5, 5.0, 1.5, 2.5, 10.0, 1.5] # Sample data to start

def show_stats():
    if not sales_history:
        print("No sales recorded yet.")
        return

    # 1. The Mean (Average Sale)
    avg_spend = statistics.mean(sales_history)

    # 2. The Mode (Most frequent price)
    try:
        popular_price = statistics.mode(sales_history)
    except statistics.StatisticsError:
        popular_price = "Multiple/No clear popular price"

    print("\n--- HENDERSON SALES ANALYTICS ---")
    print(f"Average Customer Spend (Mean): ${avg_spend:.2f}")
    print(f"Most Common Transaction (Mode): ${popular_price}")
    print(f"Total Transactions: {len(sales_history)}")
```

4.5 API + requests + JSON

4.5.1 API

We all use web pages, right? Especially me, as a Linux user; much of my life is in the browser. But what do we do with browsers? Browsers let us communicate with servers through the web. We send a request, and the server sends back a response. You might have seen responses like:

- 200 OK
- 404 Page Not Found

But APIs are different. Instead of returning HTML, which is the language used to build websites and interfaces, APIs usually return data through a URL. When you access an API endpoint, you get JSON rather than a web page. But how do we access APIs in the first place?

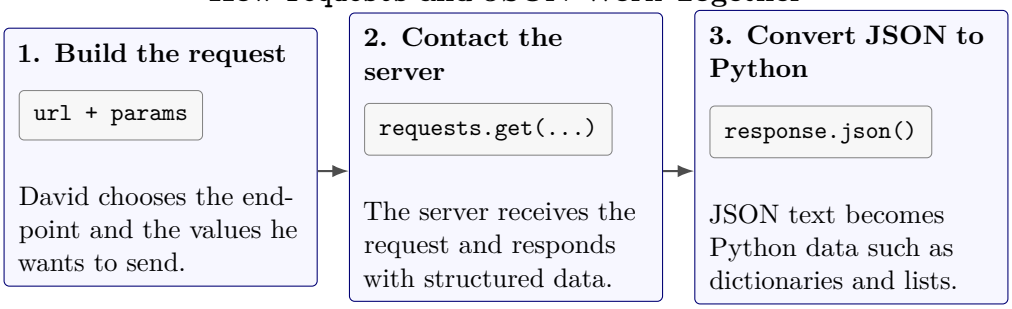
4.5.2 `requests` (getting data from the internet)

Unlike what we have seen until now, `requests` is a package. It contains multiple internal modules such as `requests.api` and `requests.sessions`. It is basically a client library that accesses web pages, sends requests such as GET and POST, and receives responses.

The basic mental model:

- Every interaction follows this structure
  - Build request
  - Send request
  - Receive response
  - Extract data

How `requests` and JSON Work Together



Build request

- You define:
  - where you want to go (URL)
  - what you want to search

book.py

```python
import requests

query = "harry potter"

url = "https://openlibrary.org/search.json"
params = {
    "title": query
}
```

What is happening here?

- Here we are trying to search for a book with the Open Library API, so we assign the endpoint URL to the variable `url`.

- We also need to specify which book we are looking for, so we put the search term in a dictionary called `params` (short for parameters). In this case, the title is "Harry Potter." We have defined what we want and where to look for it, but we have not sent the request yet.

Mental model: I wrote the email, but I have not sent it yet.

Send request

- Now the request can be executed. Since we have both our destination and the thing we want, this is straightforward.

response.py

```
    response = requests.get(url, params=params)
```

What is happening here?

- Python connects to Open Library servers
- sends our query: "find books with this title"
- waits for the server response

Mental model: I sent the email, and now I am waiting for a response.

Receive response

- Now the server responds with structured data.

response.py

```
print(response)
```

Terminal Output

```
Response [200]
```

What is happening here?

- The request succeeded.
- The server returned data.

Extract data

- Now we convert the response into a usable Python structure.

- The API response includes a status code (for example, 200), headers (information the server sends back, such as the server type), and a body. In many cases, the body is JSON, which we will cover next. Since JSON text is not yet a native Python object, we convert it into Python data structures such as dictionaries and lists using the following function.

response.py

```
data = response.json()
```

The dictionary looks like this:

dictionary

```
data = {
    "docs": [
        {
            "title": "Harry Potter and the Sorcerer's Stone",
            "author_name": ["J.K. Rowling"]
        },
        {
            "title": "Harry Potter and the Chamber of Secrets",
            "author_name": ["J.K. Rowling"]
        }
    ]
}
```

Through this mechanism, we can access an API through `requests` and extract data. In a sense, our Python code is acting like a search engine.

4.5.3 JSON

It stands for JavaScript Object Notation. It is a text format for structured data exchange between systems. You can think of it as text with rules. It looks similar to a Python dictionary, but JSON is not a Python object. Remember: it is just text.

j.json

```
{
  "name": "Alice",
  "age": 20,
  "is_student": true
}
```

Why JSON exists:

- Different systems need a common language. Different products use different languages. Browsers use JavaScript, while servers may use Python, Java, Go, and more. JSON acts as a bridge between them. JSON solves this problem because it is:
  - simple
  - language-independent
  - human-readable

However, there is an important difference between JSON and Python types. When Python reads JSON, it converts the values like this:

- object `{}` → `dict`
- array `[]` → `list`
- string → `str`
- number → `int` / `float`
- null → `None`

Project: Optimizing our HGI with the currency value

Mr. Henderson wants to update his prices and import process based on the value of the U.S. dollar. To do that, the HGI system needs access to current exchange-rate data. How can David do this?

Solution

To support this optimization, we need to get a response from an API with data related to the U.S. dollar. Based on that data, David can run a simple analysis and estimate whether it is a good time to import or to sell.

inventory.py

```python
import requests
import statistics

def analyze_market_exchange():
    # A REAL, OPEN API FOR CURRENCY MARKET TRENDS
    url = "https://api.frankfurter.app/2026-01-01..2026-04-26"


    print("Fetching real market data from Frankfurter API...")
    response = requests.get(url)

    if response.status_code == 200:
        data = response.json()

        # The API returns a dictionary of dates, each containing the USD rate.
        rates = data["rates"]
        price_list = [details["USD"] for details in rates.values() if "USD" in details]

        # Use David's statistics skills.
        avg_market_price = statistics.mean(price_list)
        most_common_price = statistics.mode(price_list)
        current_price = price_list[-1]

        print("\n--- MARKET TREND REPORT (USD/EUR) ---")
        print(f"Current Market Price: {current_price}")
        print(f"Period Average (Mean): {avg_market_price}")
        print(f"Most Frequent Price (Mode): {most_common_price}")

        if current_price > avg_market_price:
            print("Trend: The dollar is stronger than average. This may be a good time to import.")
        else:
            print("Trend: The dollar is weaker than average. Import costs may change soon.")
    else:
        print("Could not fetch market data from the API.")

if __name__ == "__main__":
    analyze_market_exchange()
```

Project: Building the HGI Market Intelligence Report

Background

Mr. Henderson is no longer satisfied with checking market conditions by hand. HGI now wants a small digital reporting tool that can collect exchange-rate data, summarize it clearly, and stamp the report with the current date and time. In this project, you will practice **APIs**, **`requests`**, **JSON data**, **`statistics`**, and **`datetime`**. The goal is not only to fetch information from the internet, but also to turn raw market data into a useful management report that helps HGI decide when to import stock.

In a file named `market_report.py`, write a Python program for the HGI reporting terminal. The program should request currency data from an API, extract the values you need, summarize the numbers, and print a clean report for management.
Your program must do the following:

- sends a request to a currency API endpoint,
- converts the server response from JSON into Python data,
- extracts the USD values from the returned dataset,
- calculates the current value, the average value, and the most common value,
- prints a readable market report,
- adds the current date and time to the report using `datetime.now()`.

Your reporting system should feel realistic. In other words, it should not only fetch numbers, but also explain what the numbers mean for HGI management.

Expected Behavior Examples

- If the API returns several days of currency data, the program should collect all valid USD values into one list.
- The final output should clearly show the current price, the period average, and the most frequent value.
- The report should include a timestamp so management knows when the analysis was generated.
- The program should finish by printing a short conclusion such as whether the dollar is stronger or weaker than the average for that period.

Hints

- Use `requests.get(...)` to contact the API.
- Use `response.json()` to convert JSON text into Python data.
- Access nested values by working through dictionaries carefully.
- Use `statistics.mean()` and `statistics.mode()` for the summary.
- Use `datetime.now()` and `strftime()` to label the report with the current time.

This is the kind of improvement that saves real time inside a real company. You are no longer just learning separate libraries in isolation. You are combining them into one real reporting workflow that helps HGI make faster decisions.

End of Unit Questions

Predict the Output

Predict the output of each code snippet. Write the exact output.

1.

predict_1.py

```
from datetime import date
booked = date(2026, 8, 14)
print(booked.month)
```

Terminal Output

2.

predict_2.py

```
from datetime import timedelta
print(timedelta(days=1, hours=2))
```

Terminal Output

3.

predict_3.py

```
import random
random.seed(7)
print(random.randint(1, 3))
```

Terminal Output

4.

predict_4.py

```
import statistics
scores = [4, 8, 8, 10]
print(statistics.mode(scores))
```

Terminal Output

5.

predict_5.py

```
import json
raw = '{"name": "David"}'
data = json.loads(raw)
print(data["name"])
```

Terminal Output

6.

predict_6.py

```
from datetime import datetime
moment = datetime(2026, 1, 1, 9, 5)
print(moment.strftime("%H:%M"))
```

Terminal Output

Debugging

Fix the error in each code snippet and explain why it is wrong.

1.

debug_1.py

```
import statistics
numbers = []
print(statistics.mean(numbers))
```

2.

debug_2.py

```
from datetime import time
now = time.now()
print(now)
```

3.

debug_3.py

```
import random
cards = [1, 2, 3]
shuffled = random.shuffle(cards)
print(shuffled[0])
```

4.

debug_4.py

```
import json
text = '{name: "David"}'
data = json.loads(text)
print(data)
```

5.

debug_5.py

```
import requests
response = requests.get("https://openlibrary.org/search.json")
print(response["docs"])
```

Theory and Concepts

Answer in complete sentences.

1. Explain the difference between a `module`, a `package`, and a `function`. Give one example of each.
2. Explain the difference between `date`, `time`, and `datetime`.
3. Why does `datetime.now()` depend on the computer that runs the program?
4. Explain the difference between `statistics.mean()` and `statistics.mode()`.
5. What is JSON, and why is it useful when programs communicate with APIs?
6. What is the difference between these two imports?

theory_6.py

```
import datetime
from datetime import datetime
```

Library Reasoning

1. Inventory Timestamping

trace_1.py

```
from datetime import datetime
arrival = datetime.now()
label = arrival.strftime("%Y-%m-%d")
```

What type is stored in `arrival`? What type is stored in `label`? Why are they different?

2. Report Summary

trace_2.py

```
import statistics
orders = [12, 15, 12, 9, 30, 12]
```

Which function would you use to find the average order size? Which function would you use to find the most common order size?

3. Random Access Key

trace_3.py

```
import random
import string
chars = string.ascii_letters + string.digits
key = ""
for _ in range(4):
    key += random.choice(chars)
print(key)
```

Why will the output usually change from one run to another?

4. API Workflow Put these steps in the correct order: build the request, send the request, read the response, convert JSON into Python data. Then explain why that order matters.

Program Writing

Write complete programs for the following tasks.

1. Delivery Stamp Write a program called `delivery_stamp.py` that asks the user for an item name, stores the current date and time, and prints a formatted message such as `Milk logged at 2026-04-27 14:30`.

2. Lucky Shelf Write a program called `lucky_shelf.py` that stores five shelf names in a list, randomly selects one shelf, and prints the shelf that will be audited today.

3. Sales Summary Write a program called `sales_summary.py` that stores a list of transaction totals and prints both the mean and the mode.

4. JSON Reader Write a program that converts the following JSON text into Python data and prints the values of `name` and `role`:

profile.json

```
'{"name": "David", "role": "clerk", "active": true}'
```

Mixed Challenge Problems

1. Daily Audit Report Write a program that imports `datetime`, `random`, and `statistics`, stores five transaction totals in a list, randomly selects one employee from a list of names, prints today's date, and prints the average transaction total.

2. JSON + Statistics A JSON string contains daily sales:

sales.json

```
'{"sales": [10, 20, 20, 40, 10, 20]}'
```

Write a program that converts the string into Python data, extracts the sales list, and prints the mean and the mode. Then explain how the same program could be extended with `datetime` to timestamp the report.

Chapter 5

Exceptions

ASSUME

Mr. Henderson is leaning over a terminal, watching over the HGI system you built. He looks up as you enter.

Mr. Henderson: "David, come in! I was just looking at the market trend reports your system generated. That API integration and the statistical analysis? Absolute game-changers. We've saved more in the last quarter than we did in the previous two years combined.

David: "I am glad to hear that, Mr. Henderson. I've been focusing on making the data more dynamic and secure, just like we discussed."

Mr. Henderson: "And it shows. But David, as we scale up to international shipping, our systems are interacting with more 'unknowns.' Last night, the system crashed because the API sent a piece of data we didn't expect. It wasn't a logic error, it was an interruption. We can't afford to have the engine stall every time something unexpected happens."

David: "I agree. The system needs to know how to react when things go wrong outside our control."

Mr. Henderson: "Exactly. You've proven you can build the machine and secure the vault. Now, I need you to make it 'unbreakable.' Because of your leadership and technical foresight, I am officially promoting you to Senior Software Developer. Your new mission is to ensure that HGI stays standing, no matter what errors the worlds throws at it."

David: "Thank you, Mr. Henderson. I've already started reading about how to handle these 'exceptions' from a book written by two amazing guys. I'll make sure the system is ready for anything."

5.1 Yet another promotion

Congratulations! Your growth is truly remarkable. Advancing from a summer intern to a Senior Software Developer in such a short period of time is really wonderful. However, the higher you climb, the more the nature of your work changes.

Your new responsibilities:

- **Handling user mistakes:** Making sure the system doesn't crash if a user types their name where they should have typed a quantity.
- **Safe Division:** Preventing the program from stopping if it tries to calculate an average but there is no data to process.
- **Basic crash protection:** Building "safety nets" around your code so that even one part fails, the rest of the HGI system keeps running.

This is a big step up, but don't worry, you will be able to complete all these tasks by the end of this chapter. We will cover:

- **Exceptions:** Understanding how Python signals that something has gone wrong.

5.2 Exceptions

The common misconception about exceptions is that exceptions are "errors," but an exception is a runtime signal that something unexpected happened and the normal execution cannot continue in the current flow.

Perhaps the following analogy will help:

- Your program is a train moving along tracks like a normal train
- An exception is a broken path ahead.
- You either switch tracks (handle the error) or let the train crash (program terminates)

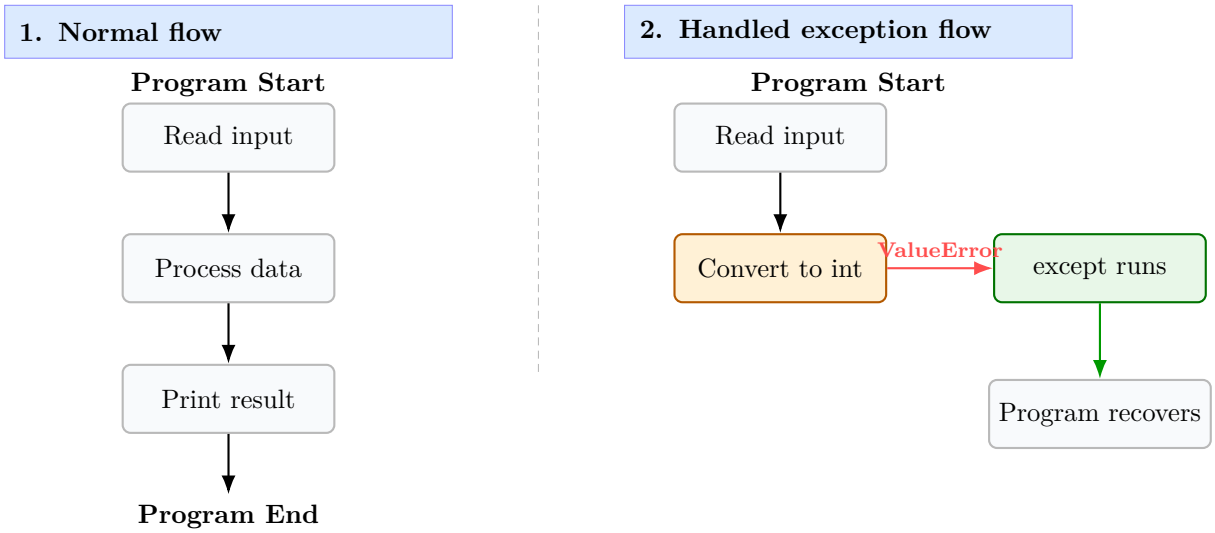


Figure 5.1: Normal program flow versus flow interrupted by an exception

Syntax errors Vs. Runtime Exceptions

- Syntax errors:
  - Happens before the program runs
  - Python refuses to even start

syntax_errors.py

```
print("Hello"
```

  - This is like submitting a form with missing fields. It never gets processed.
- Runtime Exceptions
  - Happens during execution

runtime_exceptions.py

```
x = int("abc")  # ValueError
```

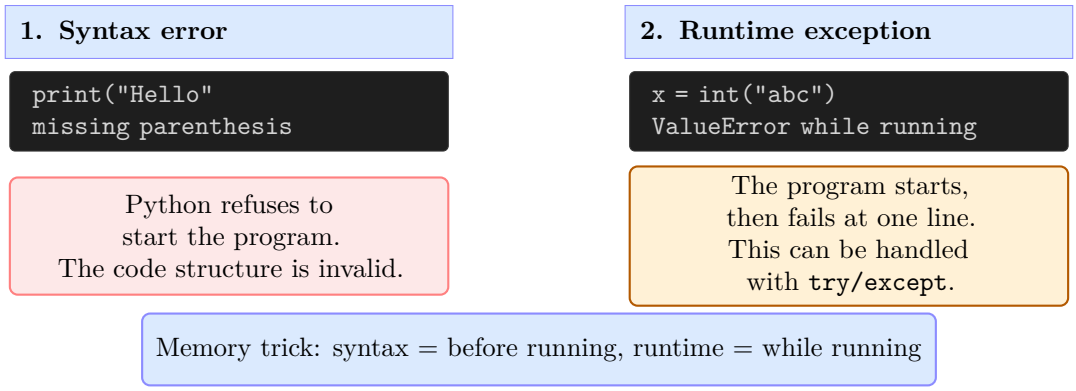


Figure 5.2: The difference between syntax errors and runtime exceptions

This is like entering the system with valid credentials but then failing during processing.

These runtime exceptions crush our programs. They may be a result of our programming mistakes, as we said before, or from the user side due to invalid input.

For example

invalid_input.py

```
x = int(input("What's x? "))
print(f"x is {x}")
```

If the user inputs strings such as "abc", then the program crashes and throws an error statement.

ValueError: invalid literal for int()

5.3 Handling Exceptions

This is a serious problem. We can't allow our program to crash every time there is an invalid input, so we have to handle it.

- **Fun fact:** "Bug" is the most famous name for exceptions. If you have ever seen tech movies from Hollywood, they always mention bugs, but why? Why, of all insects, bugs? The term Bug has been used since the dawn of tech. Even Thomas Edison used it in his papers, but the actual documented incident happened in 1947 with Grace Hopper and her team while they were working on the Harvard Mark II (not necessary for this story). The engineers saw the computer malfunction, and when they investigated, they found a real moth stuck in a relay (a physical switch) causing the program to crash, so they taped the moth into the logbook and wrote:

"First actual case of bug being found"

These engineers handled the exception by throwing out the bug, we don't have a bug in our computers, so how do we handle exceptions to prevent our computer from crashing?

5.4 `try` and `except`

To handle exceptions, we need two types of blocks of code.

- **Risky code:** code that has the potential to trigger errors
- **Recovery code:** code that handles the situation when an error occurs

There are other types of code blocks as well, but these are the most essential ones for handling exceptions.

We use two major keywords to handle exceptions:

- `try` and
- `except`

example_1.py

```
#example
try:
    # risky code
except:
    # recovery code
```

What is happening here?

- The `try` statement will contain our code that we are not so sure about or the code that may cause errors.
- The `except` statement will contain our response or our recovery code. If that error occurs, what we will do about it will be inside the except statement.

example_2.py

```
try:
    x = int(input("What's x? "))
    print(f"x is {x}")
except:
    print("That was not a number.")
```

As we saw last time, the code we put inside the try statement is our risky code, which crashes if we put in anything other than a number, such as a string or a character, so inside our except statement we put a statement that will explain what happened to the user instead of throwing a

big red and chunky error statement that the user will most probably have a hard time reading.

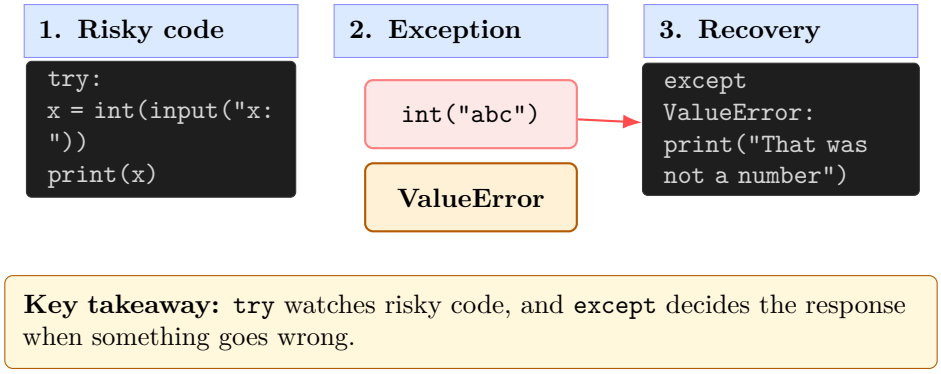


Figure 5.3: How the `try` and `except` blocks work together

But we still have a problem

- No matter what the error is, the except section always says "That was not a number", but the exception may come from something else.

For example:

example_3.py

```
try:
    x = int(input("x: "))
    y = int(input("y: "))
    print(x / y)
except:
    print("That was not a number")
```

In the above program, our program accepts two numbers to divide one by the other. Our program, like the previous one, only throws a valid error statement for `ValueError`, but we have other exceptions too.

- Our program takes x and divides it by y, so x can be any number, but y cannot be 0 because the result will be undefined, so the program throws `ZeroDivisionError`. Even if we assign y to 0, our program still says "That was not a number," but 0 is a number, so our program is throwing an invalid error statement.

- To solve this problem, we need to specify our responses to specific errors, but how do we do that?

To do that, we use keywords next to the `except`. These keywords specify the error type. Whether it is a `ValueError` or a `ZeroDivisionError`, we identify it by using these keywords.

Exception	Category	When It Happens	Example
ValueError	Logical/value error	Correct type, but invalid value	`int("abc")`
TypeError	Type mismatch	Operation applied to wrong type	`"2" + 2`
IndexError	Sequence bounds error	Index outside valid range	`[1,2][5]`
KeyError	Mapping error	Missing dictionary key	`{"a":1}["b"]`
ZeroDivisionError	Arithmetic error	Division by zero	`10 / 0`
FileNotFoundError	I/O error	File does not exist	`open("x.txt")`

Table 5.1: Common Python Exceptions

The following are some of the core exception keywords:

But how do we do this? How do we integrate our keywords into our code?

- Well that is easy, there is only one additional keyword to write.

without_keyword.py

```
#example
try:
    # risky code
except:
    # recovery code
```

This catches everything, including things you did not anticipate.

with_keyword.py

```
try:
    # risky code
except keyword:
    # recovery code
```

That is it. There is only one specific error we can handle with this code. It could be either ZeroDivisionError or ValueError.

Now you are saying:

- I expect this specific failure, nothing else.

The better version of our previous code will be:

example_with_keyword.py

```
try:
    x = int(input("x: "))
    y = int(input("y: "))
```

```
    print(x / y)
except ValueError:
    print("That was not a number")
```

So this version of our code will only handle value exceptions. This code handles inputs that are anything other than numbers. It doesn't care whether it is 0 or any other kind of number. It throws exception statements if the given input is not a number, so we achieved what we wanted in the first place, but we still have a problem.

We need our program to be immune to every exception that our first code gave us access to, but it has its consequences. Our first code had the same recovery code for every type of exception, but we solved that problem by specifying our recovery code for a specific exception.

That solution we gave is still flawed. When we solved our problem by specifying responses for errors, we lost our privilege of handling multiple exceptions, which is a major trade-off.

How do we come back from that?

We have two options:

- Going back to our first code: Our first code, even if it is flawed, had its perks, such as multiple exception handling. For a broader approach, we should optimize our recovery code to be more general. For example, look at the following code:

  option_1.py

  ```
  try:
      x = int(input("x: "))
      y = int(input("y: "))
      print(x / y)
  except:
      print("Error occurred")
  ```

- Handling every exception we expect: Here we use our "specific exception handling" code, but it requires a lot of work. In this approach, you have to write an error statement for every exception specifically. For instance:

  option_2.py

  ```
  try:
      x = int(input("x: "))
      y = int(input("y: "))
      print(x / y)
  except ValueError:
      print("Invalid number.")
  ```

```
7 except ZeroDivisionError:
8     print("Cannot divide by zero.")
```

With this approach, we have to specifically respond to every exception that we expect our program to have. In our case, we anticipate that we might encounter value errors and zero division errors, so we write recovery code tailored for each of the exceptions.

Throughout what we have learned up to now, the execution flow goes as follows:

- If input is valid:
  - `try` succeeds
  - `except` doesn't run
- If input is "abc",
  - `try` fails
  - `except` runs

Important

If the input fails by any means, Python jumps immediately to except, and Python skips the remaining try block if there is one

There Is No Dumb Question

David: Wait, if the `except` block catches the error, does the computer still show those scary red lines of code in the terminal?

Mr. Feynman: Not at all! That is the best part. By "catching" the exception, you are preventing the default crash report from appearing. Instead, the computer does exactly what you told it to do in the `except` block (like pringting a helpful message), and then continues the next line of code as if nothing went wrong.

David: What happens if an error occurs inside an `except` block.

Mr. Feynman: Then you have a new problem on your hands. If a second error happens while you are trying to fix the first one, Python will stop and report that new error. A Senior Developer tries to keep the `except` block as simple as possible to avoid a double-crash.

5.5 `else` clause

- The last time we saw the else statement, it was in conditionals, and we said that else can't be independent; without if, there is no else. It turns out else had tricks up its sleeve. I think it was feeling unwanted, so it started a side hustle and joined the exception handling team, but in my opinion, it has more use in conditionals than in exception handling.
- Else runs only if no exception occurred.

The syntax is as follows:

else_clause.py

```
try:
    # risky code
except ValueError:
    # recovery code
except ZeroDivisionError:
    # success message
```

When we look at the execution flow when we add our else statement, it is not significantly different. The execution flow goes as follows:

- If input is valid:
  - `try` succeeds
  - `except` doesn't run
  - `else` runs → prints success message
- If input is "abc",
  - `try` fails
  - `except` runs

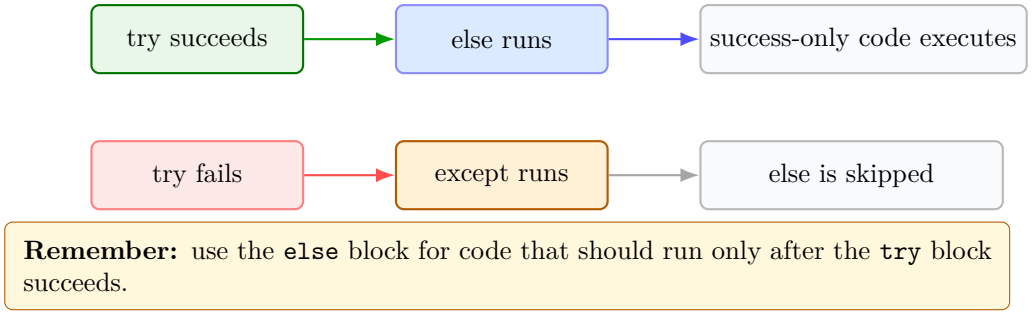


Figure 5.4: The `else` block runs only when the `try` block succeeds

The first time I looked at the else statement in exception handling, I didn't like it. I don't know if you faced the same confusion, but if you write the success statement below your risky code in your try statement, you will have an identical result. Let me show you what I mean.

example.py

```
try:
    num = int(input("Enter a number: "))
except ValueError:
    print("That is not a valid integer.")
else:
    print("Conversion successful:", num)
```

Terminal Output

```
Enter a number:  123
Conversion successful:  123
```

In our second case, our success statement is inside our try block, but the lines after our risky code only get executed if there is no exception, so it is the same as our else statement.

By now, I think you hate the else statement in exception handling, but else has another trick up its sleeve. Else is actually useful in some cases, but we are not going to see it in this volume of our book.

Important

The else block is not just for displaying an output, we can also execute different operations upon successful completion.

There Is No Dumb Question

David: So the `else` block is for code that depends on try succeeding, but isn't "risky" itself?

Mr. Feynman: Exactly. For example, in our HGI syste, you might `try` to connect to the currency API. The `else` block would then be the place where you perform the math ro update your prices. You only want to do math if the connection actually worked!

David: Can I have ans `else` block without an `except` block?

Mr. Feynman: No, Python won't allow that. An `else` cluase in exception handling specifically requires at least one `except` block to exist. It needs that "safety net" present to know that "all clear" actually means something.

5.6 `finally` clause

- The `finally` block is basically a block that always runs. Finally always runs regardless of:
  - success
  - failure
  - return
  - crash

Mostly we use it to do something that we execute no matter what the outcome is. For instance, in file execution (we will see it in our next chapter, don't worry), we open a file, we execute some operation, and no matter what the outcome of that execution is (either success or failure), we have to close the file, so with finally we will be able to close the file after our execution. It is basically our cleanup logic.

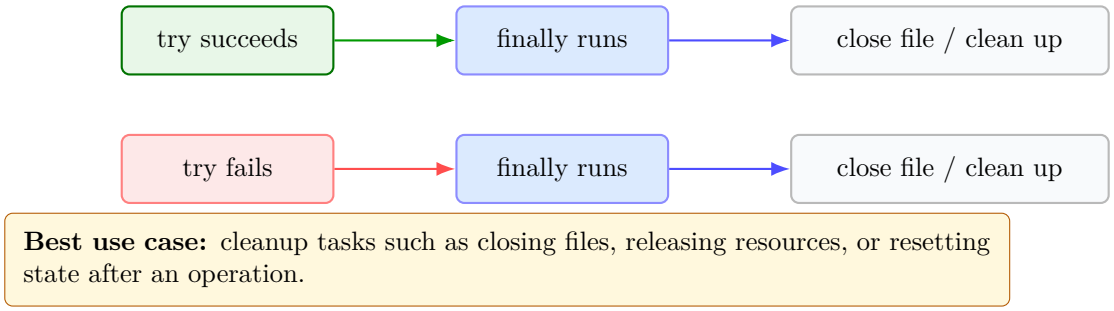


Figure 5.5: Why the `finally` block is used for cleanup

example.py

```
try:
    # risky code
except SomeError:
    # recovery code
else:
    # success statement / logic
finally:
    # clean up
```

The full exception handling statement looks like this, we can cover almost every aspect of exceptions through this statement.

real_world_example.py

```
try:
    f = open("data.txt")
    data = f.read()
except FileNotFoundError:
```

```
    print("File not found")
else:
    print("Read successful")
finally:
    print("Closing attempt")
    try:
        f.close()
    except:
        pass
```

Don't worry about the file execution code; we will see that in our next chapter, but you can get the idea of what is going on here. Our try block opens a file called data.txt and reads what is inside that file (our risky code). We might encounter some errors; for instance, we might not find the file, or even if we find it, we might not have permission, but for this case we are going to stick to `FileNotFoundError`. Then the except block handles the exception and throws an error statement. If we encounter no errors whatsoever, we print our success statement through our else block, but where are our manners? We have to close the file after operating on it, so through our finally block, we close the file.

Important

We can use exception handling statements under another exception handling statements. The operations we execute through our exception handling statements themselves might be risky, so we have to find a way to handle those bugs instead of focusing on the broad errors we might encounter.

There Is No Dumb Question

David: Does the `finally` block run even if I use a `return` statement in the `try` block?

Mr. Feynman: `finally` is incredibly loyal. Even if you tell a function to `return` and exit, Python will pause, run everything inside the `finally` block first, and then finish the return. It is the one part of your code that refuses to be ignored.

David: What if the program crushes completely because of an error it didn't catch? Does `finally` still execute?

Mr. Feynman: Yes! Even if your program is about to crash and burn because of an unhandled exception, Python will execute the `finally` block as its "last words" before the program shuts down. This ensures that no matter how bad the crash is, your cleanup tasks are finished.

David: Is there any situation where `finally` doesn't run?

Mr. Feynman: Only in extreme cases, such as when the power goes out, the computer is physically unplugged, or a command like `os._exit()` is used to terminate the Python process instantly, will the `finally` block fail to execute. In almost every practical coding scenario, however, `finally` provides a reliable guarantee.

5.7 Raising your own exceptions

- Python built-in exceptions might not be enough for our needs. You are not limited to Python's built-ins. You can raise your own errors or exceptions tailored to your needs.
  - But how? How can we do that?

There is another keyword we use for this purpose, raise.

When you are in class, if you see your teacher making mistakes, you raise your hand or your voice to point out that error. With the same methodology, if there is a situation in which our code might raise something different from what we intended.

Another vital statement we use is the conditional statement. To raise an exception, we have to check whether our condition for raising an exception is fulfilled or not. We are able to do that through conditionals. We check whether our input is out of our bounds or not and throw an exception based on the result.

x

```
 = int(input("x: "))
if x < 0:
    raise ValueError("x must be positive")
```

This program makes us able to have a bounded input. Any input that is less than 0, or any input that is negative, is regarded as invalid input.

Terminal Output

```
x:  -5 x must be positive
```

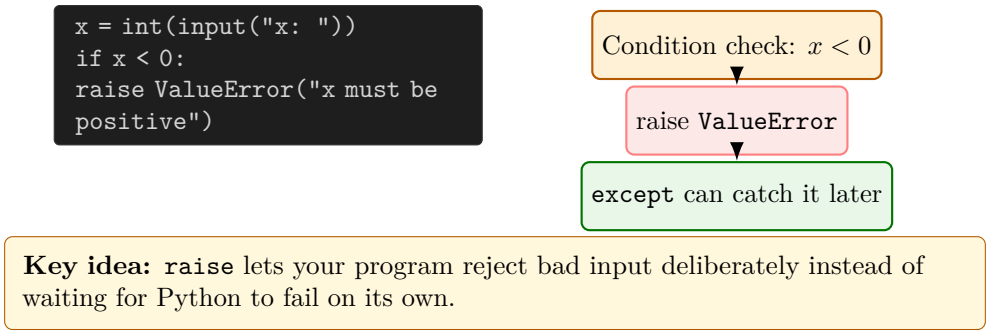


Figure 5.6: Using `raise` to create an exception deliberately

Through this mechanism, we can define our own exceptions, and handle them accurately. There is one other mechanism that we can use to execute this more efficiently, but we are not going to cover it through this volume.

Another concept you should be familiar with is the exception hierarchy. All of the keywords we mentioned before are under one parent section called `Exception`, and `Exception` is under another hierarchical parent called `BaseException`, so we can use these as keywords in our exception handling statements.

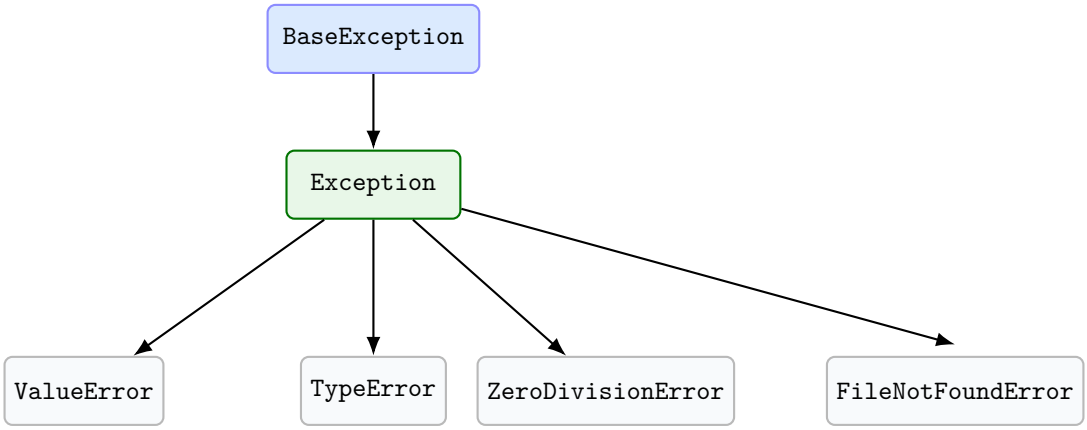


Figure 5.7: A simplified view of the Python exception hierarchy

So we can say

hierarchy_example_1.py

```
try:
    # risky code
except BaseException
    # recovery statement
```

This code will make us able to handle every error-handling mechanism Python has to offer, but it is like our previous version, which is like saying just except:. They are basically the same; they both handle every error. We can also say:

hierarchy_example_2.py

```
try:
    # risky code
except Exception:
    # recovery statement
```

This one is like the previous code, but with a bit of a limited scope. There are some errors that are covered under the `BaseException` category but not under the `Exception` category, such as `KeyboardInterrupt` (an exception raised when the user manually interrupts a running program, typically by pressing Ctrl + C in the terminal).

Up to now, our exception handling mechanisms just tell us that an error has occurred or "something failed." Instead, what if we wanted to dissect it, log it, or react based on the result?

To be able to do that, we have to capture the exception object, and what do we capture it in?

A VARIABLE

We can bind the exception to a variable, and we can inspect that variable, which represents the exception.

variable_exception_1.py

```
except ValueError as e:
```

We can assign that exception to a variable, in this case our variable e. We can perform operations on e. For instance, in our case, we can print the error statement itself instead of writing what the error is, which in a way lets us handle multiple exceptions. No matter what the exception is, we can print it out rather than writing these statements. For example:

variable_exception_2.py

```
try:
    x = int(input())
    y = 10 / x
except Exception as e:
    print("Failure:", e)
```

As you can see, instead of printing out statements ourselves, we print the error itself. That

way, we can handle multiple exceptions (all the exceptions under the `Exception` category in our previous hierarchy) with a single line while keeping our transparency to our user.

Another way we can handle multiple exceptions is by generating a common statement for multiple exceptions.

multiple_exception_1.py

```
try:
    x = int(input())
    y = 10 / x
except (ValueError, ZeroDivisionError) as e:
    print("Failure:", e)
```

Overall, you can visualize the exception handling statement through the following analogy: Imagine a restaurant:

- Customer orders food → normal flow
- Kitchen runs out of ingredients → exception
- Options:
  - Substitute dish → handled exception
  - Cancel order → unhandled exception

Your job as a programmer:

- decide the response, not avoid the failure

Important

Our final goal through exception handling is not avoiding exceptions, it is deciding the right response to our failures.

There Is No Dumb Question

David: Which error should I raise? Do I have to make up a new name?

Mr. Feynman: You can eventually create your own, but most programmers start by "borrowing" Python's existing names or keywords. If a user provides a name that is too short, you might raise a `ValueError`. If they try to access a part of the warehouse that doesn't exist, you might raise an `IndexError`. You are using a languages the com puter already understands to describe a specific problem.

David: What happens after I `raise` the exception? Does the program just die?

Mr.Feynman: Only if you fail to catch it. A common technique is to raise an error in one part of the program and catch it in another. This allows you to send a specific "self_destruct" signal with a custom message, such as `raise ValueError("Shipment quantity cannot be negative!")`, which can then be handled and displayed clearly to Mr. Henderson.

Project: Exception-Safe Inventory Terminal

Background

Mr. Henderson is upgrading HGI's warehouse input process from a fragile manual workflow to a safer digital terminal. He needs a reliable system that can accept shipment quantities, reject invalid entries, and stay running even when employees make mistakes. In this project, you will practice **exceptions**, **`try/except`**, **`else`**, **`finally`**, and **raising your own errors**. The goal is not only to stop crashes, but also to design a terminal that behaves like a real warehouse tool.

In a file named `inventory_manager.py`, write a Python program for the HGI terminal. The program should ask the employee for an incoming shipment quantity and then respond safely based on the value entered.
Your program must do the following:

- asks the employee for an incoming shipment quantity,
- uses `try` to convert the input into an integer,
- catches `ValueError` if the user types text instead of a number,
- raises a custom `ValueError` if the number entered is negative,
- uses `else` to confirm that the shipment was recorded only when everything succeeds,
- uses `finally` to print a final readiness message for the next employee.

Your terminal should feel realistic. In other words, the output should clearly tell the user whether the shipment entry was accepted or denied.
Expected behavior examples:

- If the user types `25`, the system should accept the shipment.
- If the user types `apple`, the system should explain that the value is invalid.
- If the user types `-8`, the system should explain that inventory cannot be negative.
- No matter what happens, the terminal should always end by telling the next employee that the system is ready.

Hints:

- Use `int(input(...))` inside the `try` block.
- Use `raise ValueError(...)` for the negative quantity rule.
- Put the success message in the `else` block.
- Put the cleanup or reset message in the `finally` block.

End of Unit Questions

Predict the Output

1. Predict the output:

predict_1.py

```
try:
    x = int("12")
    print(x + 3)
except ValueError:
    print("Bad input")
```

Terminal Output

2. Predict the output:

predict_2.py

```
try:
    x = int("abc")
    print(x)
except ValueError:
    print("Conversion failed")
```

Terminal Output

3. Predict the output:

predict_3.py

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Terminal Output

4. Predict the output:

predict_4.py

```
try:
    x = int("5")
except ValueError:
    print("Wrong")
else:
    print("Success", x)
```

Terminal Output

5. Predict the output:

predict_5.py

```
try:
    x = int("9")
    y = 10 / x
except Exception:
    print("Failure")
else:
    print(y)
finally:
    print("Done")
```

Terminal Output

Predict the Output

6. Predict the output:

predict_6.py

```
try:
    x = int("0")
    print(10 / x)
except ValueError:
    print("Not a number")
except ZeroDivisionError:
    print("Math error")
```

Terminal Output

7. Predict the output:

predict_7.py

```
try:
    raise ValueError("Negative value")
except ValueError as e:
    print("Caught:", e)
```

Terminal Output

8. Predict the output:

predict_8.py

```
try:
    print("A")
except Exception:
    print("B")
else:
    print("C")
finally:
    print("D")
```

Terminal Output

9. Predict the output:

predict_9.py

```
try:
    items = [1, 2, 3]
    print(items[5])
except IndexError:
    print("Bad index")
```

Terminal Output

10. Predict the output:

predict_10.py

```
try:
    x = int(input("x: "))
    if x < 0:
        raise ValueError("x must be non-negative")
    print("Accepted")
except ValueError as e:
    print(e)
```

If the user types `-2`, what is the output?

Terminal Output

Debugging

Fix the error in each code snippet and explain why it is wrong.

1.

debug_1.py

```
try
    x = int("5")
except ValueError:
    print("Wrong")
```

2.

debug_2.py

```
try:
    print(10 / 0)
except ValueError:
    print("Error")
```

3.

debug_3.py

```
try:
    x = int("abc")
except:
print("Bad input")
```

4.

debug_4.py

```
try:
    x = int("7")
else:
    print(x)
```

5.

debug_5.py

```
try:
    y = 10 / 2
except ZeroDivisionError:
    print("Zero")
finally
    print("Done")
```

6.

debug_6.py

```
try:
    raise "Wrong"
except:
    print("Caught")
```

Theory and Short Answer

1. What is the difference between a syntax error and a runtime exception?
2. Why is using plain `except:` usually weaker than using a specific exception type?
3. What is the purpose of the `else` block in exception handling?
4. Why is the `finally` block useful when working with files or database connections?
5. What does `except ValueError as e:` allow you to do that `except ValueError:` does not?
6. Explain the difference between `BaseException` and `Exception`.
7. Why is `KeyboardInterrupt` not usually something you want to catch with broad exception handling?
8. What does the keyword `raise` do?
9. Give one real-world scenario where raising your own `ValueError` makes sense.
10. Why is exception handling about choosing the right response rather than pretending failure will never happen?

Trace the Flow

For each snippet below, write which block runs: `try`, `except`, `else`, and/or `finally`.

1.

trace_1.py

```
try:
    x = int("4")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

2.

trace_2.py

```
try:
    x = int("cat")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

3.

trace_3.py

```
try:
    print(5 / 0)
except ZeroDivisionError:
    print("A")
finally:
    print("B")
```

4.

trace_4.py

```
try:
    raise ValueError("bad")
except TypeError:
    print("A")
finally:
    print("B")
```

What happens here?

5.

trace_5.py

```
try:
    x = [1, 2]
    print(x[1])
except IndexError:
    print("A")
else:
    print("B")
```

6.

trace_6.py

```
try:
    d = {"a": 1}
    print(d["b"])
except KeyError:
    print("Missing key")
finally:
    print("End")
```

Write the Response Message

For each situation, write a user-friendly message that the program should display.

1. A customer types letters instead of a shipment quantity.

2. A worker tries to divide the total value by the number of boxes, but the number of boxes is 0.

3. The program cannot find the file `prices.txt`.

4. The user enters a negative number for product count.

5. A dictionary does not contain the warehouse code the user requested.
6. The user presses `Ctrl + C` to stop the program in the terminal.

Program Writing

Write full Python programs for the following tasks.

1. Safe Average Calculator

Ask the user for two numbers. Print their average.

- Catch `ValueError` if the user types invalid input.
- Use `else` to print the average only when both inputs are valid.
- Use `finally` to print `"Calculator session ended"`.

2. Positive Age Checker

Ask the user for an age.

- Convert the input to an integer.
- Raise `ValueError` if the age is negative.
- Print a success message only if the value is valid.

3. Safe Dictionary Lookup

Create a small dictionary of product codes and prices.

- Ask the user for a code.
- Catch `KeyError` if the code does not exist.
- Print the price if the code exists.

4. File Reader Skeleton

Write a program that tries to open `notes.txt`.

- Catch `FileNotFoundError`.
- Use `else` to print `"Read successful"`.
- Use `finally` to print `"Closing attempt"`.

HGI Exception Challenges

Use exception handling ideas from this chapter to design solutions for these HGI situations.

1. Damaged Manifest Entry

An employee types `"twelve"` instead of `12` for incoming boxes. Describe:

- which exception may occur,
- which block should handle it,

- what message the user should see.

2. Shipping Division Problem
The system calculates cost per box using:

challenge_2.py

```
cost_per_box = total_cost / boxes
```

What exception must be anticipated? How should the program respond? **3. Missing Inventory File**
The overnight script tries to open `inventory_backup.txt`, but the file is missing. Explain how `try`, `except`, `else`, and `finally` could be used. **4. Invalid Custom Rule**

HGI decides that shipment quantity cannot be more than 10,000 in one manual entry. Write the conditional logic and identify which exception you would raise. **5. Multiple Exception Design**
You are writing one input program that could fail because of:

- text instead of a number,
- division by zero,
- a negative quantity.

Would you use three different `except` blocks or one broad one? Explain your design choice. **6. Final Reflection**

In two or three paragraphs, explain how exception handling helps transform a fragile program into a reliable one.

Chapter 6

Files

ASSUME

Mr. Henderson stands beside a crowded desk, one hand resting on a stack of printed shipping manifests and the other pointing at the HGI terminal. He looks up as you walk in.

Mr. Henderson: "David, perfect timing. The systems you've built have changed how we operate. Orders move faster, reports are cleaner, and every department finally trusts the numbers. But now we have a different kind of problem. We are drowning in information. Customer records, shipment logs, pricing tables, audit trails—everything has to be stored, retrieved, updated, and shared without confusion."

David: "So the next challenge is not just processing data. It is organizing it so the system can reliably keep track of everything."

Mr. Henderson: "Exactly. Right now, too much important information disappears into memory and vanishes when the program stops. That may work for a quick calculation, but it does not work for a company managing contracts, cargo, and compliance documents across the globe."

David: "Then we need a way to make data permanent. If the system can read from files and write back to them, we can preserve records, exchange information with other systems, and build tools people can depend on day after day."

Mr. Henderson: "That is the mission. And David, before we go any further, I want to make something official. Because of the way you've strengthened this company's systems, I am promoting you again. As of today, you are HGI's new IT Manager. I do not just want software that thinks, I want software that remembers. I want a leader who knows how to open the right file, save the right information, and bring order to a mountain of messy records."

David: "Thank you, Mr. Henderson. I am honored. If exceptions taught us

how to survive mistakes, then files will teach us how to preserve progress. As IT Manager, I'll make sure HGI knows how to remember."

6.1 Yet another promotion

Congratulations again. Your rise inside HGI has been fast, but every promotion changes the kind of problems you solve. Before, you mainly focused on getting a program to work while it was running. Now you are responsible for making sure the work of that program survives after it stops.

At this level, your responsibilities sound less like classroom exercises and more like real business demands:

- **Saving records:** customer names, shipments, logs, and invoices cannot disappear when a script ends.
- **Loading information back:** if the company saved yesterday's data, today's program must be able to read it again.
- **Working with structured data:** business information often lives inside CSV files, not just plain text.
- **Handling media files:** not every file is a paragraph of words; some files store images and other binary data.

By the end of this chapter, you will know how to read from files, write to files, append to existing files, use context managers safely, process CSV data, and even interact with image files through Python libraries.

6.2 What is a file?

A variable lives in memory. Once the program ends, that memory is gone. A file, however, lives on disk, which means the data can remain available long after the program exits.

That difference is everything.

If you calculate a total and only store it in a variable, it vanishes when Python stops. If you write that total into a file, you can come back tomorrow, next week, or next month and read it again.

A file is simply a named location that stores data. Python can:

- open a file,
- read what is inside it,
- write new content into it,
- append extra content to the end,
- and close the file when it is done.

Important

Memory is temporary. Files are persistent. That single distinction is one of the biggest shifts from toy programs to useful software.

6.3 Opening files

The function we use again and again in this chapter is `open`. It tells Python which file we want to work with and how we want to work with it.

open_file.py

```
name = input("What's your name? ")

file = open("names.txt", "w")
file.write(name)
file.close()
```

Let us unpack this carefully:

- `open("names.txt", "w")` means open the file `names.txt` in write mode.
- Python returns a *file object*, which we store in `file`.
- `file.write(name)` writes the user's name into the file.
- `file.close()` closes the file so Python can finish the operation cleanly.

Common file modes

The second argument we pass to `open` is called the mode.

- `"r"` — read from a file.
- `"w"` — write to a file, replacing old content if it already exists.
- `"a"` — append to the end of a file without deleting previous content.

If you forget this behavior, remember one sentence: `"w"` wipes, `"a"` adds.

6.4 Writing to files

When we use write mode, Python creates the file if it does not already exist. If the file does exist, Python starts fresh and replaces the old content.

write_example.py

```
name = input("What's your name? ")

file = open("names.txt", "w")
file.write(name)
file.close()
```

Suppose the user enters `David`. The file `names.txt` will now contain:

Terminal Output

```
David
```

Now imagine we run the same program again and enter `Maria`. The old content is overwritten.

Terminal Output

```
Maria
```

This behavior is useful when you want a clean file every time, but dangerous when you want to preserve older information.

6.4.1 New lines matter

If we keep writing names into a file without line breaks, the contents can become messy very quickly.

better_write.py

```
name = input("What's your name? ")

file = open("names.txt", "a")
file.write(f"{name}\n")
file.close()
```

Here we made two important changes:

- We changed the mode from `"w"` to `"a"`.
- We added `\n` so each new name goes on its own line.

After running the program several times, the file might look like this:

Terminal Output

```
David Maria Amina
```

Important

Text files do not automatically add line breaks for you. If you want neat rows, you must write those line breaks yourself.

6.4.2 Closing files and why it matters

In earlier chapters, we learned that some tasks need cleanup logic. Files are one of the best examples. When Python opens a file, your program is borrowing a resource from the operating system. Once you are done, that resource should be released.

That is why `file.close()` matters. It helps ensure that:

- buffered data is actually written,
- the file is properly released,
- and your program behaves politely toward the system.

Still, there is an even better way.

6.5 The `with` keyword

Python gives us a cleaner, safer pattern for working with files: the `with` statement.

with_example.py

```
name = input("What's your name? ")

with open("names.txt", "a") as file:
    file.write(f"{name}\n")
```

This is one of the most useful habits you can build.

The word `with` means: open this file, let me work with it, and automatically close it afterward.

So instead of writing:

- open the file,
- do the work,
- remember to close the file,

we let Python manage the cleanup for us.

Why `with` is preferred

- It is shorter.
- It is safer.
- It reduces the chance that you forget to close the file.
- It reads like English: *with this file open, do the following work.*

In modern Python, if you are opening a file for normal use, `with` should usually be your first instinct.

There Is No Dumb Question

David: If `with` closes the file automatically, does that mean I should never call `close()` myself?

Mr. Feynman: In ordinary file work, yes, that is exactly the idea. When you use `with`, Python handles the closing step for you. The whole point is to remove that burden from your memory and make your code safer.

6.6 Reading from files

Saving data is only half the story. The real power of files appears when we can bring that information back into a new program.

read_names.py

```
with open("names.txt", "r") as file:
    for line in file:
        print("hello,", line)
```

This code works, but the output will look a little odd because every line already ends with `\n`. The `print` function also adds a newline of its own, so the result may have extra blank space.

A cleaner version uses `strip()`:

read_names_clean.py

```
with open("names.txt", "r") as file:
    for line in file:
        print("hello,", line.rstrip())
```

The method `rstrip()` removes whitespace from the right side of the string, especially the newline at the end.

If `names.txt` contains:

Terminal Output

```
David Maria Amina
```

Then the output becomes:

Terminal Output

```
hello, David hello, Maria hello, Amina
```

6.6.1 Sorting what we read

Sometimes we do not want to print lines in the exact order stored in the file. We may want to sort them first.

sorted_names.py

```
names = []

with open("names.txt") as file:
    for line in file:
        names.append(line.rstrip())

for name in sorted(names):
    print(f"hello, {name}")
```

This introduces a useful pattern:

- create an empty list,
- read data from the file,
- clean each line,
- store it,
- and process it later.

In real software, this is common. You first gather the data, then organize it, then display or analyze it.

A small improvement

You can make the previous program even shorter:

sorted_names_short.py

```
with open("names.txt") as file:
    for line in sorted(file):
        print("hello,", line.rstrip())
```

This works because a file object can be iterated line by line, and `sorted(file)` gives us those lines in sorted order.

6.7 CSV files

A plain text file is helpful, but businesses often need more structure than a single column of names. That is where CSV files become valuable.

CSV stands for *comma-separated values*. A CSV file stores tabular data, where each line is a row and commas separate the columns.

For example, a file called `students.csv` might contain:

Terminal Output

```
name,house Harry,Gryffindor Hermione,Gryffindor Draco,Slytherin
```

This format is simple, portable, and incredibly common in the real world.

6.7.1 Reading CSV files manually

Because a CSV file is still text, Python can read it line by line just like any other file.

students_manual.py

```
with open("students.csv") as file:
    for line in file:
        name, house = line.rstrip().split(",")
        print(f"{name} is in {house}")
```

This is an important step because it shows that structured data is still just text underneath. We split each line at the comma and assign the two pieces to variables.

But there is a problem.

The first row in many CSV files is a header row. If our file begins with `name,house`, then our program will treat that header like actual data.

Also, manual splitting becomes fragile when the data itself contains commas.

Important

Manual parsing is educational because it helps you understand what a CSV file really is. But in practical work, you often want a more reliable tool.

6.7.2 Collecting rows in dictionaries

Before we use Python's CSV module, let us look at a useful pattern that appears in CS50's treatment of files: building dictionaries from data.

students_dicts.py

```
students = []

with open("students.csv") as file:
    next(file)
    for line in file:
        name, house = line.rstrip().split(",")
        student = {"name": name, "house": house}
        students.append(student)

for student in sorted(students, key=lambda student: student["name"
    ]):
    print(f"{student['name']} is in {student['house']}")
```

There are several beautiful ideas inside this short program:

- `students` is a list of dictionaries.
- Each dictionary stores a row using keys like `"name"` and `"house"`.
- `next(file)` skips the header row.
- `sorted(..., key=...)` lets us sort by a particular field.
- The `lambda` function provides the sorting rule in a compact way.

If this feels advanced, that is normal. File chapters often bring together many earlier ideas: lists, loops, dictionaries, functions, and strings.

There Is No Dumb Question

David: Why not just keep the line as a string? Why go through the trouble of making a dictionary?

Mr. Feynman: Because the moment your data has meaning, structure becomes power. A dictionary lets you say `student["name"]` instead of trying to remember that the name happens to be the first comma-separated piece. It makes your program clearer, safer, and easier to extend later.

6.7.3 Using the `csv` module

Python includes a built-in module named `csv` that is designed specifically for CSV files. This is the professional way to work with them.

students_reader.py

```
import csv

with open("students.csv") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

This will read each row as a list. For example, the row `Harry,Gryffindor` becomes something like:

Terminal Output

```
['Harry', 'Gryffindor']
```

That is already more reliable than splitting lines manually, because the `csv` module knows how CSV data is supposed to behave.

6.7.4 Using `csv.DictReader`

Even better, Python can read each row as a dictionary.

students_dictreader.py

```
import csv

students = []

with open("students.csv") as file:
    reader = csv.DictReader(file)
    for row in reader:
        students.append({"name": row["name"], "house": row["house"]})

for student in sorted(students, key=lambda student: student["name"]):
    print(f"{student['name']} is in {student['house']}")
```

This is a lovely improvement because the header row now becomes part of the logic. Instead of saying "take the first column," we say "give me the value under the `name` column."

That is exactly how human beings think about tables, and Python lets us program at that level.

Why `DictReader` feels so natural

- Column names become keys.
- Your code becomes easier to read.
- Reordering columns in the file becomes less dangerous.
- You spend less time counting column positions and more time expressing intent.

6.7.5 Writing CSV files

Reading data is important, but eventually our programs need to create structured files too. Python's `csv` module helps with that as well.

students_writer.py

```
import csv

name = input("What's your name? ")
house = input("What's your house? ")

with open("students.csv", "a", newline="") as file:
```

```
    writer = csv.writer(file)
    writer.writerow([name, house])
```

This appends a new row to the CSV file.

The argument `newline=""` may look strange at first, but it helps Python write rows cleanly, especially across different operating systems.

There is also a dictionary-based approach:

students_dictwriter.py

```
import csv

name = input("What's your name? ")
house = input("What's your house? ")

with open("students.csv", "a", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=["name", "house"])
    writer.writerow({"name": name, "house": house})
```

The dictionary version pairs nicely with `DictReader`. If your program already thinks in terms of named fields, `DictWriter` keeps that style consistent.

6.8 File names and command-line habits

As your programs grow, hard-coding file names becomes limiting. In more advanced scripts, we often receive a filename from the command line and then open that file dynamically.

Even when the filename is fixed, however, the lesson remains the same: programs become much more powerful once they can work with information stored outside the code itself.

A script with hard-coded values is isolated. A script that reads files can participate in a workflow.

6.8.1 Binary files and images

Not every file is plain text. Some files store images, audio, video, or other binary data. You cannot meaningfully read those files line by line as human-readable text.

That does not mean Python cannot work with them. It simply means we need tools that understand those formats.

In CS50's Introduction to Programming with Python, one example is working with image files by using the `PIL` library from Pillow.

shirt.py

```
from PIL import Image

shirt = Image.open("shirt.png")
```

```
photo = Image.open("person.png")

size = shirt.size
photo = photo.resize(size)
photo.paste(shirt, shirt)
photo.save("output.png")
```

Do not worry if every image method does not feel natural yet. The core idea is what matters right now:

- a file can hold binary data,
- Python can open that file through an appropriate library,
- and your program can transform it and save the result into a new file.

The same philosophy of file I/O still applies. The only difference is the kind of data being stored.

Important

Text files are only one category of files. Once you understand how opening, reading, writing, and saving work conceptually, you can transfer that understanding to many other file types.

6.9 Common mistakes beginners make with files

Let us save you from several classic frustrations:

- **Forgetting the newline:** results get glued together.
- **Using `"w"` when you meant `"a"`:** old content disappears.
- **Forgetting to strip lines when reading:** output contains ugly blank spacing.
- **Treating header rows as data:** your first record becomes `name is in house`.
- **Parsing CSV manually in complicated cases:** commas inside quoted data can break your logic.
- **Forgetting `with`:** resources may not be managed as cleanly as they should be.

Every one of these mistakes is normal. Beginners are not bad at files; they are simply meeting persistence for the first time.

6.10 Why files matter in the real world

Files are where programs meet the outside world in a durable way.

At HGI, files might store:

- customer rosters,
- shipping manifests,
- archived logs,
- product inventories,
- exchange-rate snapshots,
- and reports generated for managers.

In your own projects, files might store notes, saved games, exported data, photographs, user preferences, or experiment results.

Without files, every program begins from zero. With files, software develops memory.

A business analogy

Imagine a warehouse where every worker has perfect short-term memory but absolutely no shelves, labels, folders, or boxes. That warehouse would be chaos by the end of the day. Files are the shelves of software. They let your work be stored, found, and reused.

Project: Optimizing the HGI File System

Background

Mr. Henderson is upgrading HGI's reporting workflow from a slow manual process to a cleaner digital system. Every branch now exports shipment records into files, but the raw data is messy and takes too much time to clean by hand. In this project, you will practice **files**, **`with open(...)`**, **reading text data**, **writing report files**, and **working with CSV data through `csv.DictReader`**. The goal is not only to store information, but also to build a real reporting tool that helps HGI management make faster decisions.

In a file named `hgi_report.py`, write a Python program for the HGI reporting terminal. The program should read shipment data from a CSV file, clean the incoming information, total the quantities, and produce a polished report for management.

Your program must do the following:

- reads shipment records from a file called `shipments.csv`,
- uses `csv.DictReader` to process each row using column names,

- cleans extra spaces from branch names and item names,
- converts each quantity into an integer,
- totals all shipment quantities across the file,
- writes a clean summary into a file called `report.txt`.

Your reporting system should feel realistic. In other words, the output file should clearly show management what came in and how many total units were processed.
Expected behavior examples:

- If a row contains `North Dock, Bolts ,15`, the final report should clean the extra space around `Bolts`.
- If the file contains several shipment rows, the system should add all their quantities together.
- The final report should list each cleaned record clearly.
- The program should finish by confirming that the report was generated successfully.

Hints:

- Use `with open(...)` when reading and writing files.
- Use `csv.DictReader(file)` to work with CSV columns by name.
- Use `strip()` to remove extra spaces from text fields.
- Use `int(...)` to convert the quantity from text into a number.
- Build the report line by line, then write the total at the end.

This is the kind of improvement that saves real time inside a real company. You are no longer just storing data in files. You are using files to optimize how HGI works.

Practice Questions

Use these questions the same way you would use a workbook: think first, write your answer carefully, and only then test your code.

Predict the Output

1. Predict the output:

predict_1.py

```
with open("names.txt", "w") as file:
    file.write("David\n")
    file.write("Mina\n")

with open("names.txt") as file:
    for line in file:
        print(line.rstrip())
```

Terminal Output

```
Terminal Output
```

2. Predict the output:

predict_2.py

```
with open("data.txt", "w") as file:
    file.write("7")

with open("data.txt") as file:
    value = file.read()
    print(value * 2)
```

Terminal Output

```
Terminal Output
```

3. Predict the output:

predict_3.py

```
with open("items.txt", "w") as file:
    file.write("pen\n")
    file.write("book\n")

with open("items.txt") as file:
    print(len(file.readlines()))
```

Terminal Output

```
Terminal Output
```

4. Predict the output:

predict_4.py

```
with open("students.csv", "w") as file:
    file.write("name,house\n")
    file.write("Harry,Gryffindor\n")

with open("students.csv") as file:
    for line in file:
        print(line.rstrip().split(","))
```

Terminal Output

```
Terminal Output
```

5. Predict the output:

predict_5.py

```
names = ["Zara", "Adam", "Mina"]

for name in sorted(names):
    print(name)
```

Terminal Output

```
Terminal Output
```

6. Predict the output:

predict_6.py

```
with open("numbers.txt", "w") as file:
    file.write("1\n2\n3\n")

with open("numbers.txt") as file:
    for line in file:
        print(int(line.rstrip()) + 1)
```

Terminal Output

```
Terminal Output
```

Debugging

Fix the problem in each snippet and explain what went wrong.

1.

debug_1.py

```
with open("names.txt", "w") as file:
print("David", file=file)
```

2.

debug_2.py

```
with open("names.txt", "r") as file:
    file.write("Mina")
```

3.

debug_3.py

```
with open("names.txt", "a") as file:
    file.write("David")
    file.write("Mina")
```

4.

debug_4.py

```
import csv

with open("students.csv") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["Name"])
```

5.

debug_5.py

```
with open("numbers.txt") as file:
    for line in file:
        print(line + 5)
```

6.

debug_6.py

```
import csv

with open("students.csv", "a") as file:
    writer = csv.writer(file)
    writer.writerow("David", "Ravenclaw")
```

Theory

1. Explain the difference between data stored in a variable and data stored in a file.
2. Why is `with open(...)` usually better than calling `open()` and `close()` manually?
3. What is the difference between opening a file in `"w"` mode and opening it in `"a"` mode?
4. Why do we often use `rstrip()` when reading lines from a text file?
5. What problem can happen if you parse CSV files manually with `split(",")`? Give one realistic example.
6. Explain why `csv.DictReader` is often easier to work with than `csv.reader`.
7. What does the argument `newline=""` help prevent when writing CSV files?

8. Why can image files not usually be processed meaningfully as plain text?
9. In your own words, explain what it means to say that files make software *persistent*.
10. Why is sorting data after reading it from a file sometimes better than printing it immediately?

Code Tracing and Reasoning

For each question below, do not just write the answer. Explain the logic line by line.

1. What will the file contain after this code runs?

trace_1.py

```
with open("log.txt", "w") as file:
    file.write("Start\n")

with open("log.txt", "a") as file:
    file.write("Process\n")
    file.write("End\n")
```

2. What will be printed?

trace_2.py

```
with open("names.txt", "w") as file:
    file.write("Laila\nNoah\nOmar\n")

names = []
with open("names.txt") as file:
    for line in file:
        names.append(line.rstrip())

for name in sorted(names, reverse=True):
    print(name)
```

3. Why does this behave differently from what many beginners expect?

trace_3.py

```
with open("value.txt", "w") as file:
    file.write("8")

with open("value.txt") as file:
    x = file.read()

print(x + x)
```

4. What will `row` look like on each iteration?

trace_4.py

```
import csv

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["name", "house"])
    writer.writerow(["Cho", "Ravenclaw"])
    writer.writerow(["Cedric", "Hufflepuff"])

with open("students.csv") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Program Writing

Write complete programs for the following. Use clear variable names and good formatting.

1. `register.py`

Write a program that asks the user for a name and appends that name to a file called `register.txt`, placing each new name on its own line.

2. `rollcall.py`

Write a program that reads every name from `register.txt` and prints:

Terminal Output

```
hello, NAME
```

for each line in the file.

3. `sorted_register.py`

Write a program that reads names from `register.txt`, sorts them alphabetically, and prints them one per line.

4. `inventory.py`
Write a program that creates a file called `inventory.txt` and stores three product names chosen by the user, one per line.
5. `count_lines.py`
Write a program that opens a text file and prints how many lines it contains.
6. `guests.py`
Write a program that keeps asking the user for guest names until the user enters a blank line. Save all entered names into `guests.txt`.
7. `houses.py`
Given a CSV file with the columns `name` and `house`, write a program that prints every student's name followed by their house.
8. `writers.py`
Write a program that asks the user for a name and department, then appends that information as a new row in `employees.csv` using the `csv` module.

Applied Business Scenarios

These are meant to feel closer to the HGI world.
1. Shipment Archive
HGI receives shipment IDs throughout the day. Write a program that stores every new shipment ID entered by the user into `shipments.txt`. The program should not erase older shipment IDs.
2. Daily Revenue Reader
A file called `revenue.txt` contains one revenue amount per line. Write a program that reads all values, converts them to integers, and prints the total revenue.
3. Client Directory
A CSV file called `clients.csv` contains the columns `name`, `country`, and `tier`. Write a program that prints only the names of clients whose tier is `gold`.
4. Audit Log Formatter
A text file contains one event per line, but many lines have unwanted trailing whitespace. Write a program that reads the file and prints a cleaned version of each line.
5. Team Roster Export
Write a program that asks the user for three employee names and saves them into a CSV file called `team.csv` under a column called `name`.
6. Branch Report Merger
Suppose you have two text files, each containing one branch name per line. Describe one strategy for reading both files and printing one sorted combined list without duplicates.

Mini Challenges

These are longer and require planning before coding.

1. Top Reader

Write a program that reads a file of names and prints the alphabetically first name and the alphabetically last name.

2. Clean CSV Copier

Write a program that reads `students.csv` and writes a new file called `cleaned.csv` containing the same data but with every name stripped of extra surrounding spaces.

3. Simple Backup Tool

Write a program that asks the user for text and saves it in `notes.txt`. Then write a second program that copies the contents of `notes.txt` into `backup.txt`.

4. Image Rename Planner

You are given a folder full of image filenames stored in a text file, one per line. Explain how you would read the file, sort the filenames, and write them into a new report file with numbering.

5. Attendance Summary

A file contains one student name per line, and some names appear more than once because the student attended multiple sessions. Write a program that prints each unique student name once, in sorted order.

Reflection Questions

1. Which feels easier to you right now: reading plain text files or reading CSV files? Why?
2. What file mistake do you think you are most likely to make as a beginner: forgetting `with`, using the wrong mode, or forgetting a newline?
3. In real software, why is it dangerous to assume all incoming files are perfectly formatted?
4. If you were building HGI's internal tools, what information would you store in text files and what information would you store in CSV files?
5. How did this unit change the way you think about programs? Do they still feel temporary, or do they now feel more like systems that can remember?

Important

If you can confidently answer these questions and complete the programming tasks without copying solutions blindly, then you are doing exactly what this unit is trying to teach you. Files are not just about syntax. They are about building programs that keep knowledge

alive.